

Future Mountain Handbook

Last Updated: 2026-06-16
by David Gordon

Table of Contents

Project Overview and Design	16
Future Mountain Design Concept	16
Purpose.....	16
Experience Goals	16
Current Conceptual Model	16
Interaction Concept	17
Important Design Tensions	17
Future Mountain Architecture.....	17
Project Type.....	17
High-Level Runtime Flow	18
Main Project-Owned Code Areas.....	18
Data Sources	19
Scene and UI Structure.....	19
Scenario Assumptions Currently Embedded	20
External and Third-Party Assets.....	20
Developer Onboarding.....	20
Purpose.....	20
Access To Request.....	20
Clone The Repository	21
Local API Configuration	21
Server API Configuration.....	21
Importer Local Password File.....	22
Smoke Tests	22
Future Mountain Operations Guide.....	22
Purpose.....	22
Infrastructure Overview	23

Server Hosting.....	23
Primary Server	23
GRIT Support.....	23
GRIT Helpdesk	23
VPN Requirements	24
Administrative Access	24
Remote Desktop	24
Hosted Websites	25
Production Website.....	25
Staging Website	25
Data Portal	25
Website Administration	25
Databases	26
Source Code	26
Repository	26
Required Local Software.....	27
Build Targets And Symbols.....	27
Opening The Project	27
Running In The Editor.....	27
Web/API Data Check.....	28
Local API Testing.....	28
WebGL Build Checklist.....	29
Standalone/Local Build Checklist	29
Data Update Procedure	30
Documentation Update Procedure.....	30
Documentation	30
Disaster Recovery.....	31
Key Contacts.....	31
UCSB GRIT	31
Future Mountain Team	31
Revision History	32
SimulationSettings	32
Purpose.....	32

How It Is Accessed	32
Inspector Groups	32
Web Build Optimization	35
Remaining Scattered Flags	35
Features.....	36
Visualization	36
Camera Animation	36
User Interface Behavior	36
Technical Behavior.....	36
Animation Triggers	37
Cube Click Flow.....	37
Reset Flow.....	38
Fly Camera	38
Key Objects And Scripts	38
Current Constraints	38
Cube Design.....	39
Design Purpose.....	39
User Interface Behavior	39
Technical Structure	39
Per-Cube Ownership.....	40
Fire And Terrain.....	40
Relationship To Data	40
Current Constraints	41
Related Specs.....	41
Lighting	41
User Interface Behavior	41
Technical Behavior.....	42
Seasonal Transition.....	42
Runtime Update.....	42
Current Constraints	42
Interaction Modes	43
Normal Mode.....	43
User Interface Behavior	43

Technical Behavior	43
Key Objects And Scripts	44
Scenario Assumptions	44
Side-by-Side Mode	44
User Interface Behavior	45
Technical Behavior	45
Key Objects And Scripts	46
Current Constraints	46
Timeline	46
User Interface Behavior	46
Technical Behavior	47
Data Display	47
Mode Interaction.....	47
Key Objects And Scripts	48
Current Constraints	48
Simulation Layers	48
Fire	48
Conceptual Model	48
User Interface Behavior	48
Scheduled Fire Dates	49
Fire Ignition Flow.....	49
Data-Driven Landscape Spread.....	49
Importer Contract	50
Cube Fire Behavior.....	50
Fire Animation Architecture	51
Burned Terrain And Regrowth.....	51
Current Constraints	51
Ground Water	52
Purpose.....	52
Visual Behavior	52
Data Inputs.....	52
Technical Flow.....	52
Depth Mapping	53

Material Mapping	53
Related Systems	54
Current Constraints	54
Roots	54
Purpose.....	54
Visual Behavior	54
Data Inputs.....	55
Technical Flow.....	55
Prefabs And Scaling	56
Known Constraints.....	56
Snow.....	56
User Interface Behavior	57
Technical Behavior.....	57
Cube Snow	57
Landscape Snow.....	57
Splatmap Behavior	58
Fire Interaction	58
Current Constraints	58
Soil	58
Purpose.....	59
Visual Behavior	59
Data Inputs.....	59
Technical Flow.....	59
Surface Soil Mapping.....	60
Deep Soil Mapping	60
Current Constraints	60
Data And Messaging	60
Data Mappings.....	60
Purpose.....	60
Mapping Overview.....	61
Time Model.....	62
Landscape Water Mapping	62
Cube Stream Mapping.....	63

Cube Soil And Groundwater Mapping	63
Cube Snow Mapping	63
Landscape Snow Mapping	64
Cube Vegetation Mapping.....	64
Tree And Shrub Size Mapping	66
Litter Mapping.....	67
Net Photosynthesis Mapping	67
Unvisualized Or Partially Visualized Fields	67
Fire Mapping	69
Model/Statistics Overlay Mapping	69
Backward Compatibility Notes.....	69
Data Model	70
Purpose.....	70
Goals	70
Current Data Flow	70
Current Tables / API Resources	70
Runtime Entities	71
Current Schema Risks	76
Compatibility Protocol	76
Recommended Canonical Runtime Model.....	77
Needed Inputs To Complete This Spec	78
Open Questions.....	78
Messages	78
User Interface Behavior	78
Technical Behavior	79
Runtime Display.....	79
Timeline Integration	80
Current Constraints	80
Show Model / Data Layer	80
Purpose.....	80
User Interface Behavior	80
Current Displayed Variables	81
Technical Flow.....	81

Data-to-Bar Mapping.....	82
Timeline As Precipitation Graph	82
Currently Disabled Or Latent Data Layer Variables	82
Current Limitations	83
Future Improvement Ideas.....	83
Backward Compatibility Notes.....	83
Future Mountain API.....	84
Future Mountain API	84
Connection Strings	84
Routes	84
Central Coast Prototype DTOs	85
Local Build And Publish	86
IIS Deployment Runbook	86
Postman Collection	87
Documentation.....	87
Future Mountain API Overview.....	87
Purpose.....	87
Runtime Configuration	88
Route Families	88
Manual Testing	88
Current Constraints	89
Related Docs	89
Future Mountain API Configuration	89
Connection Strings	89
Hosting	89
CORS	90
Swagger	90
Current Cleanup Notes	90
Future Mountain API Routes	90
Big Creek Routes.....	90
Central Coast Routes	92
Development Route	94
Future Mountain API Deployment	94

Purpose.....	94
Build Output	94
Required Configuration	95
IIS Copy Workflow	95
Smoke Tests	95
Rollback	96
RHESSys Data Importer	96
RHESSys Data Importer Overview	96
Purpose.....	96
Project Location.....	96
Runtime Type.....	97
High-Level Flow	97
Main Code Areas.....	97
Current Import Categories	98
Current Database Targets.....	98
Current Operation Modes	99
Current Known Boundaries	100
Related Documentation	100
RHESSys Data Importer Building and Running	100
Purpose.....	100
Prerequisites	100
Build	101
Configuration Files	101
Working Directory	101
Interactive Import	101
Auto Import	103
Database Behavior	104
File Discovery Check	105
Recommended Current Workflow	105
Troubleshooting.....	106
Current Safety Notes.....	106
RHESSys Data Importer Data Sources.....	106
Purpose.....	107

Current Embedded Data.....	107
Aggregate Files	107
Cube Files	107
Configured Source Folders	108
Source Formats Currently Parsed.....	108
Current Source-Data Assumptions.....	109
Current Data Management Concerns	109
RHESSys Data Importer Scenario Config	109
Purpose.....	110
Current Scope	110
Top-Level Shape	110
Database Section.....	110
Input Folders	111
File Patterns	111
Column Mapping	111
Flags	112
Output Tables	112
Current Config Issues To Know	112
RHESSys Data Importer Scenario Profiles	113
Purpose.....	113
Known Profiles	113
Selection	113
Why Not Infer The Profile	114
Current Behavior.....	114
Compatibility	114
RHESSys Data Importer Scenario Upgrade Guide	114
Purpose.....	114
Short Version	115
Step 1: Create A New ScenarioConfig.....	115
Step 2: Inventory The Source Data	115
Step 3: Choose Or Add A Scenario Profile	116
Step 4: Design Raw Import Tables First	117
Step 5: Add Models And DbContext Entries	117

Step 6: Create And Export The Database Schema	117
Step 7: Add Validation Before Import	118
Step 8: Implement Importers Incrementally	118
Step 9: Keep Derived Terrain/API Data Separate	119
Step 10: Preserve Existing Behavior	119
Suggested Documentation Checklist	119
Suggested Implementation Checklist	120
Rule Of Thumb	120
RHESSys Data Importer Import Pipeline	120
Scope.....	120
Entry Point.....	120
Config Load	121
Flag Parsing.....	121
Interactive Pipeline	122
Auto Pipeline	122
Cube Import Pipeline.....	123
Legacy Cube Pipeline	124
Legacy Non-Cube Pipelines	124
Database Write Pipeline	124
Error Handling.....	125
Current Acceptance Criteria	125
RHESSys Data Importer Scenario Config Schema	125
Scope.....	125
Object Model.....	126
ScenarioConfig.....	126
DatabaseConfig.....	126
ScenarioFlags.....	127
JSON Naming.....	127
Discovery Categories	127
Central Coast Source File Roles	128
Column Mapping Contract.....	128
Current Validation Behavior	129
Current Schema Compatibility Notes	130

RHESSys Data Importer File Naming And Discovery	130
Scope.....	130
File Discovery	130
Discovery Algorithm	130
Fixed Categories	131
Current Config Patterns.....	131
Cube Filename Parsing.....	131
Legacy Aggregate Filename Parsing.....	132
Legacy Fire Filename Parsing.....	132
Legacy Terrain Filename Parsing	132
Discovery Limitations	133
Current Practical Implications	133
RHESSys Data Importer Database Write Contract.....	133
Scope.....	133
Database Provider	133
Connection String Resolution	134
DAL Class.....	134
Write Pattern	134
Current DbContexts	134
Cube Data Contract	135
Dates Contract	136
Patch Data Contract.....	136
Fire Data Contract.....	136
Terrain Data Contract.....	136
Water Data Contract	136
Current Schema Management	136
Current Write Limitations	136
Current Operational Contract	137
Big Creek Scenario	137
Big Creek Version	137
Scope.....	137
Current Scenario.....	137
Primary User Features	137

Runtime Behavior.....	138
Known Constraints.....	138
Baseline Check.....	138
Data Formats	138
Overview	139
Scenario Config	139
Warming Levels	139
Cube Data	139
Water Data	141
Patch Data	142
Fire Data	142
Terrain Data.....	142
Dates	143
Central Coast Migration Checklist.....	143
Central Coast Scenario	143
Central Coast Version.....	143
Scope.....	143
Current Scenario.....	143
Implemented So Far	144
Important Missing Data	144
Current Constraints	144
Related Docs	145
Central Coast v2 vs Big Creek v1 Data Differences	145
Purpose.....	145
Summary.....	145
Major Differences.....	145
Warming Scenario Difference	146
Patch Identity Difference	147
Riparian Difference	148
Stratum Difference.....	148
Burn Difference.....	149
Spatial Data Difference	149
Import Strategy Difference.....	149

API Prototype Compatibility	150
Compatibility Rule	150
Central Coast v2 Data Formats.....	151
Overview	151
Date Coverage	151
File Inventory	151
Spatial Identity Model.....	152
DEM / Unity Heightmap Notes	153
Cube Locations	154
Daily Aggregate File	154
Daily Cube Patch Files.....	156
Daily Overstory Stratum Files	157
Daily Understory Stratum Files	157
Monthly Basin Burn File	158
Monthly Patch Burn File	159
Monthly Stratum Carbon File	159
Patch Map Raster Source	161
Initial Import Assumptions.....	161
Open Questions.....	162
Initial Patch Mapping	162
Purpose.....	162
Source Raster	162
Import Command	163
Implementation	164
PatchData Rows	164
Coordinate System	165
Terrain Projection.....	165
Validation	166
Current Constraints	166
Initial Terrain Data	166
Purpose.....	166
Inputs	167
Import Command	167

Rectangular Grid	168
TerrainData Rows.....	168
Data Encoding	169
Aggregation	169
Generator Flow	170
Static DEM Heightmap	170
Current Constraints	171
Central Coast v2 Importer Config Design.....	171
Purpose.....	172
Config Shape	172
File Roles.....	173
Design Decisions	174
Selecting The Config.....	175
How To Run	175
Schema Setup and Migration Workflow	176
Known Follow-ups	177
Central Coast v2 Database Schema Design.....	177
Purpose.....	177
Ground Rules.....	178
Scale Split	178
Raw Import vs Precomputed TerrainData.....	178
Table Overview	179
Decision: where the daily aggregate goes.....	179
Common Columns.....	180
Dates	180
CubeData.....	180
WaterData.....	181
BurnData.....	182
StratumData.....	183
PatchData	183
ImportRun	184
Acceptance Mapping.....	184
Follow-ups	185

Future Roadmap	185
Adding Future Scenarios	185
Purpose.....	185
Current Scenario Pattern	185
High-Level Workflow	185
Scene Duplication.....	186
Terrain Duplication.....	186
Prefabs, Materials, And Shared Assets	186
SimulationSettings.....	187
API Profile And Runtime Data	188
Messages And Narrative Content	189
Visual Tuning Checklist	189
Data Contract Checklist	189
Common Risks	190
Smoke Tests	190
RHESSys Data Importer Roadmap	190
Purpose.....	190
Current Baseline	191
Near-Term Priorities	191
Medium-Term Improvements.....	193
Cleanup And Lower-Priority Work.....	194
Completed Or Improved	194
When To Revisit This Roadmap	194
Appendix / Miscellaneous.....	195
Technical Documentation Process	195
Purpose.....	195
Handbook Manifest.....	195
Building The Draft Handbook	195
Final Handbook Output	196
RHESSys Data Importer Repository Strategy.....	196
Purpose.....	196
Decision.....	196
Project And Build Files.....	197

Generated Files	198
Large Source Data.....	198
Acceptance Check.....	198

Project Overview and Design

Future Mountain Design Concept

Source: Docs/DesignConcept.md

Last updated: 2026-06-12

Purpose

Future Mountain is an interactive 3D climate and watershed visualization. It turns RHESSys model outputs into a spatial experience where visitors can see how climate warming, water availability, fire, soil, and vegetation interact across a real landscape.

The current project centers on the Big Creek watershed in the Sierra Nevada. It was developed as part of the Burn Cycle Project with artist Ethan Turpin and UCSB ecohydrologist Naomi Tague, building from the RHESSys model developed by the Tague Lab at UCSB.

Experience Goals

- Make long-term ecohydrologic model output legible to non-specialist audiences.
- Let visitors explore rather than only watch: change warming level, move through time, inspect terrain cubes, and compare scenarios.
- Connect abstract climate data to visible landscape changes: snow, groundwater depth, streamflow, vegetation growth, evapotranspiration, soil/litter, and fire.
- Support exhibition use where the experience must load quickly, run reliably, and be understandable with minimal facilitation.
- Preserve enough scientific structure that the visualization can be adapted to new scenarios, including a planned Central Coast scenario.

Current Conceptual Model

The scene combines two complementary scales:

- A full terrain view representing the watershed-level landscape.
- A set of detailed terrain cubes representing selected patches or aggregate conditions.

The aggregate cube summarizes watershed-average behavior. Individual cubes expose local variation. Side-by-side mode lets a user compare one cube against another warming scenario at the same point in simulated time.

Interaction Concept

The user can:

- Choose a warming scenario: baseline, +1 C, +2 C, +4 C, or +6 C.
- Start and pause the simulation.
- Control the speed of time.
- Scrub or jump through years on a timeline.
- Zoom into individual terrain cubes.
- Enter side-by-side mode to compare warming scenarios.
- Toggle model/statistics displays where available.

The visual language emphasizes living systems rather than charts alone. Data drives geometry, particles, terrain textures, water levels, snow coverage, plant growth, roots, litter, and fire behavior.

Important Design Tensions

- Scientific fidelity vs. exhibition clarity: the experience should show meaningful model relationships without overwhelming visitors with raw variables.
- Performance vs. density: vegetation, particles, terrain textures, and WebGL constraints require careful balancing.
- Scenario specificity vs. reuse: Big Creek assumptions are still embedded in code, data contracts, scene content, and UI. The Central Coast scenario should separate scenario configuration from reusable visualization behavior where practical.

Future Mountain Architecture

Source: *Docs/Architecture.md*

Last updated: 2026-06-15

Project Type

Future Mountain is a Unity project using Unity 2022.3.62f3. Scenario scenes are organized by scenario:

- `Assets/Scenes/BigCreekV1/BigCreekV1.unity`
- `Assets/Scenes/CentralCoastV2/CentralCoastV2.unity`

The project targets WebGL and standalone/editor workflows. Build symbols in `ProjectSettings/ProjectSettings.asset` include:

- WEB_VERSION for WebGL.
- LOCAL_VERSION for Standalone.

High-Level Runtime Flow

1. GameController coordinates scene startup, UI state, time, cube setup, timeline setup, messages, and simulation updates.
2. SimulationSettings stores tunable runtime and performance settings, including WebGL optimization multipliers.
3. WebManager fetches JSON data from the API in web/local builds.
4. LandscapeController loads and formats watershed-level water, terrain, patch, snow, and fire data.
5. CubeController loads and visualizes per-cube RHESys data for snow, groundwater, water access, streamflow, litter, photosynthesis, transpiration, and vegetation carbon.
6. UI scripts control the warming knob, time knob, pause button, timeline interactions, and message display.
7. Fire scripts under Assets/Scripts/Fire manage grid/cell fire behavior and visual fire propagation.

Main Project-Owned Code Areas

- Assets/Scripts/Controllers
 - GameController: application coordinator and simulation loop.
 - LandscapeController: large terrain state, water/patch/fire/terrain data formatting, snow/river/fire visualization.
 - CubeController: detailed cube state, data parsing, vegetation/water/soil/fire visual updates.
 - WebManager: UnityWebRequest wrapper around the Future Mountain API.
 - Tree, shrub, stream, soil, camera, and water-to-groundwater controllers.
- Assets/Scripts/Models
 - JSON/data transfer classes for cube, date, fire, terrain, water, and patch data.
 - FireModels.cs â€” FireDataPoint, FireDataPointCollection, FireDataFrame, FireDataFrameRecord
 - SnowData.cs â€” SnowDataFrame
 - WaterDataFrames.cs â€” WaterDataFrame, WaterDataMonth, WaterDataYear
 - TerrainSimulationData.cs â€” TerrainSimulationData
 - PatchDataModels.cs â€” PatchDataFrame, PatchDataMonth, PatchDataYear
 - (Pre-existing) FireData.cs, PatchData.cs, WaterData.cs, CubeData.cs, DateModel.cs, TerrainDataFrame.cs, and timeline transfer classes.
- Assets/Scripts/UI

- Timeline, warming/time knobs, pause/continue buttons, and message management.
- Assets/Scripts/Utilities
 - MathUtils.cs “shared MapValue(value, from1, to1, from2, to2) used across controllers and UI. Previously duplicated in ~10 files.
 - GameUtilities.cs, GameObjectPool.cs, GameObjectExtensions.cs “pre-existing general helpers.
- Assets/Scripts/Settings
 - SimulationSettings MonoBehaviour with inspector-tuned parameters. See [SimulationSettings.md](#) for full field reference.
- Assets/Scripts/Fire
 - SERI fire grid, cells, nodes, terrain texture integration, visual manager, and ignition helpers.

Data Sources

The current web API base URL is selected by scripting symbol:

- LOCAL_VERSION: `http://localhost:5550/api/`
- WEB_VERSION or default: `https://data.futuremtn.org/api/`

Known API paths:

- `cubedata/{patchIdx}/{warmingIdx}`
- `waterdata/{index}`
- `waterdata/total`
- `firedata/{warmingIdx}`
- `patchdata`
- `patchdata/{patchId}`
- `dates`
- `dates/{year}/{month}/{day}`
- `terraindata/{warmingIdx}`

Standalone/non-web paths still support TextAsset/Resources workflows in several places, but the current settings default toward web/API loading.

Scene and UI Structure

The scene contains:

- One large terrain/landscape.
- One aggregate cube and five individual cubes.
- Matching side-by-side cube instances.
- Loading, setup, simulation, side-by-side, and timeline UI canvases.

- Instruction/loading text stored directly in the Unity scene.
- Message data referenced as TextAssets by GameController.

Scenario Assumptions Currently Embedded

The current runtime assumes:

- Five warming levels mapped by index: 0 -> 0 C, 1 -> 1 C, 2 -> 2 C, 3 -> 4 C, 4 -> 6 C.
- Five individual cubes plus one aggregate cube.
- Big Creek-oriented API/database data.
- Fixed cube data column order through enum indices in CubeController and WebManager.
- Fixed patch and water column order through enum indices in LandscapeController.
- Terrain/fire data shapes that match the existing Big Creek data pipeline.

These assumptions should be reviewed before adding another scenario. See [AddingFutureScenarios.md](#) for the Unity-side scenario setup workflow.

External and Third-Party Assets

The project includes several Unity asset/plugin areas, including Standard Assets, Dynamic Fog, PostProcessing, Dynamic Snow System, River Auto Material, Terrain Stitch, NatCorder, Horizon[ON], NatureManufacture/Dynamic Nature assets, and utility/editor packages. These are not all necessarily active in the final scene, but they are part of the repository and should be handled carefully during refactors.

Developer Onboarding

Source: *Docs/DeveloperOnboarding.md*

Last updated: 2026-06-16

Purpose

This is the practical checklist for a new Future Mountain developer who needs repository access, server access, and working database/API configuration.

Access To Request

Ask the project lead and UCSB GRIT for:

- GitHub access to the Future Mountain repository.
- UCSB Ivanti VPN access.
- Remote Desktop access to `fm01.grit.ucsb.edu`.

- A Windows/server login account with the admin privileges needed to manage IIS for data.futuremtn.org.
- MySQL credentials for the Future Mountain databases needed for your role.

GRIT helpdesk:

<https://zammad.grit.ucsb.edu/>

Clone The Repository

Recommended local path on the Windows workstation:

```
cd D:\Git
git clone <repo-url> FutureMountain
cd D:\Git\FutureMountain
```

Open the project with Unity 2022.3.62f3.

Local API Configuration

The API project is:

Services/FutureMountainApi/FutureMountainAPI/FutureMountainAPI/

The checked-in appsettings.json should not contain real passwords. For local development, keep connection strings in a local-only config source such as appsettings.Development.json, environment variables, or user secrets.

Expected connection-string keys:

```
{
  "ConnectionStrings": {
    "BigCreekDbContext": "<Big Creek database connection string>",
    "CentralCoastDbContext": "<Central Coast database connection string>"
  }
}
```

Do not commit local password files. The repository ignores API appsettings.Development.json and appsettings.Local.json files.

Server API Configuration

The deployed API for Unity is served through:

<https://data.futuremtn.org/api/>

On the server, the deployed API folder for data.futuremtn.org must have an appsettings.json or approved server-side configuration containing production connection strings for:

- BigCreekDbContext

- `CentralCoastDbContext`

When publishing a new API build:

1. Publish locally to a folder.
2. Confirm the publish output has the correct deployment configuration.
3. Remote Desktop to `fm01.grit.ucsb.edu`.
4. Stop the IIS site/application for `data.futuremtn.org`.
5. Back up the current deployed folder.
6. Copy the publish output into the deployed folder.
7. Confirm server connection strings are still present.
8. Restart the IIS site/application.
9. Smoke test the API.

Deployment details live in:

`Docs/Services/FutureMountainApi.md`
`Specs/FutureMountainAPI/Deployment.md`

Importer Local Password File

The RHESSys importer can read an ignored local password file:

`RHESSys_Data_Importer/RHESSys_Data_Importer/appsettings.Local.json`

This is used when scenario config files intentionally omit the database password. Keep this file local only.

Smoke Tests

After configuration, verify:

`https://data.futuremtn.org/api/dates`
`https://data.futuremtn.org/api/waterdata/total`
`https://data.futuremtn.org/api/centralcoast/dates`
`https://data.futuremtn.org/api/centralcoast/waterdata/total`

Then open the intended Unity scene and confirm the simulation loads data from the expected API profile.

Future Mountain Operations Guide

Source: *Docs/Operations.md*

Purpose

This document describes the operational infrastructure required to maintain and support the Future Mountain project.

It is intended for developers, graduate students, researchers, and technical staff who may need to administer servers, websites, databases, or related services.

This document focuses on project operations rather than software architecture or scientific modeling.

Infrastructure Overview

Future Mountain consists of several web applications, databases, simulation datasets, and Unity applications hosted through UCSB infrastructure.

Primary responsibilities include:

- Website administration
- Database maintenance
- User account management
- Server access and troubleshooting
- Deployment of web applications
- Data management and backups
- Coordination with UCSB GRIT support staff

Server Hosting

Primary Server

Hostname

fm01.grit.ucsb.edu

IP Address

128.111.100.115

This server is managed through UCSB GRIT (Geographic Information and Remote Sensing Technology).

The server hosts the Future Mountain web applications and associated services.

GRIT Support

GRIT Helpdesk

<https://zammad.grit.ucsb.edu/>

GRIT should be contacted for:

- Server outages
- Authentication problems
- Authorization and permissions issues
- VPN access problems
- Remote Desktop access issues
- Operating system or infrastructure failures
- Server maintenance requests

When reporting issues, include:

- Date and time of occurrence
 - Error messages
 - Screenshots if available
 - Steps required to reproduce the issue
-

VPN Requirements

Administrative access to project infrastructure requires connection through the UCSB Ivanti VPN.

Without VPN access, administrative services may be inaccessible.

Examples include:

- Remote Desktop access
 - Internal server management
 - IIS administration
 - Database administration
 - Internal web applications
-

Administrative Access

Remote Desktop

Administrative tasks are typically performed through Remote Desktop access to:

`fm01.grit.ucsb.edu`

Requirements:

- UCSB Ivanti VPN connection

- Authorized user account
- Administrative privileges on the server

Common administrative tasks include:

- Website deployment
 - IIS configuration
 - Log inspection
 - Application troubleshooting
 - Database maintenance
-

Hosted Websites

Production Website

<https://futuremtn.org>

Primary public-facing Future Mountain website.

Staging Website

<https://staging.futuremtn.org>

Used for testing and validation before production deployment.

Data Portal

<https://data.futuremtn.org>

Provides access to simulation data and related resources.

Website Administration

Website administration may include:

- Creating new websites
- Updating IIS configuration
- Managing SSL certificates
- Deploying application updates
- Reviewing server logs

- Managing application pools

Administrative access to the server is required.

Databases

Future Mountain utilizes MySQL databases to store simulation outputs and related application data.

Database documentation is maintained separately in:

Docs/DataDictionary.md

Specs/DataModel.md

Database administration tasks may include:

- Backups
- Schema updates
- User account management
- Data imports
- Performance troubleshooting

Source Code

Source code repositories are maintained through GitHub.

Developers should use Git for:

- Version control
- Code reviews
- Documentation updates
- Issue tracking

Project documentation should be committed alongside source code whenever possible.

Repository

- Repository root: D:\Git\FutureMountain on this workstation.
- Unity solution: FutureMountain.sln.
- Big Creek scene: Assets/Scenes/BigCreekV1/BigCreekV1.unity.
- Central Coast scene: Assets/Scenes/CentralCoastV2/CentralCoastV2.unity.
- Unity version: 2022.3.62f3.
- Scenario config reference: ScenarioConfig_BigCreek.json.

The repository contains Unity source/assets plus documentation in Docs/ and feature specs in Specs/.

Required Local Software

- Unity Editor 2022.3.62f3.
- Unity WebGL build support if building the online version.
- Git.
- A browser for WebGL smoke testing.
- Optional: MySQL Workbench 8 for schema inspection/export.
- Optional: local Future Mountain API/database stack if testing LOCAL_VERSION.

Build Targets And Symbols

The project currently uses scripting define symbols in ProjectSettings/ProjectSettings.asset.

Known symbols:

- WEB_VERSION for WebGL.
- LOCAL_VERSION for Standalone/editor-local API testing.

Current API base URLs in Assets/Scripts/Controllers/WebManager.cs:

- LOCAL_VERSION: `http://localhost:5550/api/`
- WEB_VERSION or default: `https://data.futuremtn.org/api/`

SimulationSettings.BuildForWeb is currently true by default. In web builds, SimulationSettings.OptimizeForWeb() reduces vegetation density and increases carbon factors to improve performance.

Opening The Project

1. Open the repository folder in Unity Hub.
2. Use Unity 2022.3.62f3.
3. Open the scenario scene you are testing, such as Assets/Scenes/BigCreekV1/BigCreekV1.unity.
4. Wait for Unity to finish importing assets and compiling scripts.
5. Confirm that the scene loads without missing-script errors in the Console.

Running In The Editor

The project can run in the Unity Editor, but current runtime settings are oriented toward API-backed data loading.

Before pressing Play, confirm:

- The active scene is the scenario scene you intend to test.

- The intended scripting symbol is active for the target being tested.
- The API target is reachable if using web/API data.
- The Console is visible for data-loading errors.

Basic run check:

1. Press Play.
2. Confirm the intro/loading flow appears.
3. Start the simulation.
4. Confirm terrain, aggregate cube, and individual cubes load.
5. Confirm the Timeline appears and the simulation advances through dates.

Web/API Data Check

For the deployed Big Creek API, these endpoints are expected by Unity:

- `https://data.futuremtn.org/api/cubedata/{patchIdx}/{warmingIdx}`
- `https://data.futuremtn.org/api/waterdata/{index}`
- `https://data.futuremtn.org/api/waterdata/total`
- `https://data.futuremtn.org/api/firedata/{warmingIdx}`
- `https://data.futuremtn.org/api/patchdata`
- `https://data.futuremtn.org/api/patchdata/{patchId}`
- `https://data.futuremtn.org/api/dates`
- `https://data.futuremtn.org/api/dates/{year}/{month}/{day}`
- `https://data.futuremtn.org/api/terraindata/{warmingIdx}`

If the app stalls during loading, first check browser/Unity Console logs for failed API requests, JSON parse failures, or missing fields.

API route and deployment documentation:

- [Developer Onboarding](#)
- [Future Mountain API](#)
- [API Route Spec](#)
- [API Deployment Spec](#)

Local API Testing

LOCAL_VERSION points Unity to:

`http://localhost:5550/api/`

Use this path when testing a local API/database copy. The local API should match the same endpoint and JSON field contract as the deployed API unless Unity code is being intentionally updated.

Before testing locally:

- Start the local API.
- Confirm the local database is loaded with the expected scenario data.
- Visit one or two API endpoints directly in a browser to confirm JSON responses.
- Confirm CORS/browser access if testing a WebGL build against the local API.

WebGL Build Checklist

Before building:

- Confirm active build target is WebGL.
- Confirm `WEB_VERSION` is present for WebGL scripting define symbols.
- Confirm the intended scenario scene is included in Build Settings.
- Confirm `SimulationSettings.BuildForWeb` is true.
- Confirm the deployed API is reachable.
- Confirm no local-only API URL is selected.

Build smoke test:

1. Launch the built WebGL output through a local web server or deployment staging URL.
2. Confirm loading completes.
3. Confirm the landscape river, cubes, vegetation, snow behavior, timeline, and messages are visible.
4. Change warming level.
5. Pause/resume time.
6. Click the Timeline to jump to another year.
7. Enter Side-by-Side Mode on a cube and compare two warming levels.
8. Toggle Show Model/Data Layer.
9. Verify fire behavior around Big Creek fire years.

Standalone/Local Build Checklist

Before building:

- Confirm active build target is Standalone.
- Confirm `LOCAL_VERSION` is present for Standalone scripting define symbols if testing a local API.
- Confirm a local API is running at `http://localhost:5550/api/`, or adjust the code/config intentionally.
- Confirm the intended scenario scene is included in Build Settings.

Smoke test the same primary features as WebGL, with extra attention to any local/non-web paths such as background snow and legacy data-loading behavior.

Data Update Procedure

Current Big Creek runtime data is API/database-backed. The raw Big Creek source data files are not currently present in this Unity repository.

For any scenario data update:

1. Confirm source files and units with the data/modeling contact.
2. Update importer configuration and importer code as needed.
3. Import into a staging database.
4. Export or document the resulting schema changes.
5. Verify API responses for every endpoint Unity consumes.
6. Update Unity DTOs only if the API contract changes.
7. Update [DataDictionary.md](#), [Data Model Spec](#), and [Data Mappings Spec](#).
8. Run the WebGL/editor smoke tests before replacing production data.

For Central Coast, new fields should be added with an explicit compatibility plan so Big Creek can continue to run unchanged or through a clear adapter.

Documentation Update Procedure

When behavior changes, update the nearest feature spec in Specs/.

When data contracts change, update:

- [DataDictionary.md](#)
- [DataFormats.md](#)
- [Data Model Spec](#)
- [Data Mappings Spec](#)

When build, deployment, access, or runbook details change, update this operations document.

Documentation

Project documentation is maintained within the repository under:

Docs/

Examples include:

- Architecture documentation
- Data model documentation

- RHESys integration documentation
- Vegetation visualization specifications
- Fire system documentation
- Operations documentation

Documentation stored in Git is considered the authoritative source.

Disaster Recovery

If project services become unavailable:

1. Verify VPN connectivity.
 2. Verify server availability.
 3. Review application and IIS logs.
 4. Confirm database connectivity.
 5. Contact GRIT if server-level issues are suspected.
 6. Document findings and corrective actions.
-

Key Contacts

UCSB GRIT

Primary support organization responsible for server infrastructure and related services.

Helpdesk:

<https://zammad.grit.ucsb.edu/>

Future Mountain Team

Project-specific contacts should be maintained here.

Future updates should include:

- Principal Investigators
 - Graduate Researchers
 - Developers
 - System Administrators
-

Revision History

Date	Author	Notes
YYYY-MM-DD	Initial Author	Initial operations guide created

SimulationSettings

Source: Docs/SimulationSettings.md

Last updated: 2026-06-16

Purpose

SimulationSettings (Assets/Scripts/Settings/SimulationSettings.cs) is a Unity MonoBehaviour attached to the **Settings** GameObject in the scene. It is the single Inspector-accessible location for tuning simulation behaviour without recompiling. All controllers obtain a reference to it at startup and read from it at runtime.

How It Is Accessed

GameController finds it in Awake() with:

```
settings = GameObject.Find("Settings").GetComponent<SimulationSettings>();
```

LandscapeController finds it using:

```
settings = FindObjectOfType<SimulationSettings>();
```

Other controllers (CubeController, etc.) receive a reference through method arguments or by finding it the same way. GameController asserts that the reference is not null on startup, so the Settings GameObject and component must always be present in the scene.

Inspector Groups

Data

Field	Default	Meaning
CubeDataOnly	false	Skip the full landscape simulation and use test precipitation data. Useful for rapid iteration on cube visuals.
apiProfile	BigCreek	Scene-level API route profile. BigCreek uses legacy /api/... routes; CentralCoast prepends /api/centralcoast/... for web requests.

The Big Creek scene should use `apiProfile = BigCreek`. The Central Coast v2 scene should use `apiProfile = CentralCoast`. This setting affects cube data, dates, water data, fire data, patch data, terrain data, and timeline water-data requests because they all flow through WebManager.

BuildForWeb (Compile-Time, Not Inspector)

BuildForWeb is **not** an inspector toggle. It is a read-only C# property whose value is determined at compile time: - WEB_VERSION or LOCAL_VERSION build symbol â†’ true - No symbol (Editor) â†’ false

This controls whether data is loaded from the web API (true) or from local TextAsset/Resources files (false). It is used extensively in GameController to switch between `UpdateDataFromWeb()` and `FindParameterRanges()` paths.

Fire

Field	Default	Meaning
AutoPauseOnFire	false	Pause the simulation automatically when a fire starts.
MinFireFrameLength	6	Minimum number of simulation frames a fire occupies when auto-pause is off.
MaxFireLengthInSec	3	Fire display duration in seconds when auto-pause is on.
ImmediateFireTimeThreshold	10	Time steps above which fire ignition skips spread animation and burns immediately.

Population

Field	Default	Meaning
MinFrontTrees	2	Minimum trees placed at the front of each cube.
MaxTrees	40	Maximum grown trees per cube. Scaled down by <code>WebBuildMaxVegMultiplier</code> in web builds.
MaxShrubs	100	Maximum grown shrubs per cube. Scaled similarly.
WebBuildMaxVegMultiplier	0.55	Multiplier applied to <code>MaxTrees</code> and <code>MaxShrubs</code> to

Field	Default	Meaning
		reduce vegetation density in web builds.

Distribution

Controls spatial placement of trees and shrubs within each cube (padding, zone depth, preference percentages).

Carbon

Carbon-per-metre factors for trees, shrubs, roots, and the aggregate cube. Higher values â†’ fewer visible plants for the same carbon value. Scaled by WebBuildCarbonMultiplier for web builds.

Field	Default
TreeCarbonFactor	0.027
RootsCarbonFactor	0.009
ShrubCarbonFactor	0.004
CubeATreeCarbonFactor	0.018
CubeARootsCarbonFactor	0.0033
CubeAShrubCarbonFactor	0.005
WebBuildCarbonMultiplier	3.0

Emission

Particle emission rate factors for tree and shrub evapotranspiration particles.

Geometry

Tree and shrub height/width scale ranges and variability.

Roots

Root height/width scale ranges, spread speed, size ratio, Y-offset factor, and width variability.

Time

Field	Default	Meaning
TreeGrowthSpeedFactor	0.00033	Per-frame growth increment for trees.
TreeDeathSpeed	0.1	Speed at which trees die.
DeadTreeShrinkFactor	0.066	Per-frame shrink rate for dead tree litter.

Side-by-Side Mode

Field	Default	Meaning
-------	---------	---------

Field	Default	Meaning
SideBySideModeXOffsetAggregate	100	Lateral offset for side-by-side aggregate cube (metres).
SideBySideModeXOffset	80	Lateral offset for side-by-side sample cubes (metres).

UI

Field	Default	Meaning
MessageFramesLength	90	How many frames a message display lasts.
MessageMinLength	8	Minimum message display time in seconds.

Debugging

These flags gate Debug.Log output throughout the controllers. All default to false.

Field	Controls
DebugGame	General flow logging in GameController and LandscapeController.
DebugModel	Graph/data-layer display logging in CubeController.
DebugFire	Fire ignition, regrowth, and patch burn logging in LandscapeController.
DebugDetailed	High-frequency per-frame or per-data-row logging in LandscapeController. Very verbose – only enable briefly.

GameController reads these through its DebugLevel(int level) helper.

LandscapeController reads them directly via its settings reference obtained in Awake().

Web Build Optimization

OptimizeForWeb() is called automatically from Start() whenever the WEB_VERSION scripting symbol is defined. It applies WebBuildMaxVegMultiplier and WebBuildCarbonMultiplier to the population and carbon fields so that vegetation counts are appropriate for WebGL performance budgets. **Do not call this manually** except in automated tests.

Remaining Scattered Flags

The following debug booleans still exist as local hardcoded fields in their respective files and are not yet driven by SimulationSettings. They are all false by default and function as dead code unless manually changed in source:

Flag	Location
debugCubes, debugAggregate, debugDetailed	CubeController.cs
debugFire, debugFrame, debugMessages	GameController.cs
debug, debugDetailed	WebManager.cs

Additionally, `debugOutputPath = "/Users/davidgordon/Desktop/debug.txt"` is duplicated verbatim in both `GameController.cs` and `CubeController.cs`. It points to a hardcoded developer-specific path and is effectively dead code in any other environment.

Features

Visualization

Camera Animation

Source: *Specs/CameraAnimation.md*

Last updated: 2026-06-16

User Interface Behavior

Camera animation is used when the user focuses on a cube, exits a cube focus, or enters Side-by-Side Mode. In normal simulation view, clicking the aggregate cube or one of the five sample cubes starts a predefined camera zoom toward that cube. While zoomed in, the zoom-out button is shown unless the experience is in Side-by-Side Mode.

The user can also enable fly-camera mode. Fly mode disables the Animator-driven camera path and lets the user move the camera manually with mouse and keyboard input. Turning fly mode off re-enables the Animator and starts the normal reset zoom flow.

Keyboard shortcuts still exist for development/testing:

- A: zoom to aggregate cube.
- F, E, D, C, B: zoom to sample cubes 1 through 5.
- Space: reset zoom when already zoomed.
- Fly mode uses mouse look plus W, A, S, D, Q, E, Shift, and Control for movement speed and vertical movement.

Technical Behavior

`CameraController` is attached to the scene camera and drives the camera Animator. At startup it:

1. Gets the camera Animator.
2. Enables the Animator.

3. Reads `moveLength` from the first clip in the runtime animator controller.
4. Asserts that the zoom-out button and Side-by-Side toggle objects are assigned.

The controller uses three main runtime state flags:

- `moving`: prevents overlapping camera animations.
- `zoomed`: records whether the camera is currently in a focused position.
- `fly`: switches between Animator-driven camera motion and manual fly-camera input.

`pauseState` is a small handoff enum read by `GameController.RunGame()`. Camera animations set it to pause before triggering movement and to unpause after the animation coroutine waits for `moveLength`.

Animation Triggers

Normal zoom triggers:

- `ZoomAggregateCube`
- `ZoomCube1`
- `ZoomCube2`
- `ZoomCube3`
- `ZoomCube4`
- `ZoomCube5`

Side-by-Side zoom triggers:

- `SBS_Cube0` for the aggregate cube.
- `SBS_Cube1` through `SBS_Cube5` for sample cubes.

Reset trigger:

- `ResetZoom`

Idle state:

- `ResetPosition()` directly plays `Idle`.

Cube Click Flow

`GetMouseInput()` raycasts from the main camera on left click. If the hit object has tag `AggregateCube`, it calls:

`StartZoomIntoCube(-1)`

For other tagged cube objects, it reads the final character of the tag, converts it into a zero-based cube index, and calls:

`StartZoomIntoCube(idx)`

`StartZoomIntoCube(cubeIdx)` disables the Side-by-Side toggle, hides the model data layer, chooses the appropriate trigger, pauses simulation time through `pauseState`, starts the Animator trigger, and launches `ZoomingIn()`.

If the Side-by-Side toggle is armed and the app is not already in Side-by-Side Mode, the same method calls `GameController.EnterSideBySideMode()` before triggering the `SBS_Cube*` animation.

Reset Flow

`StartResetZoom()` restores normal UI state, pauses simulation time, triggers `ResetZoom`, and launches `ZoomingOut()`. After the wait, `ZoomingOut()` clears `moving`, sets `zoomed = false`, and requests `unpause`.

`GameController.ResetCameraZoom()` also uses `CameraController.ResetPosition()` to force the Animator back to `Idle` during broader app resets.

Fly Camera

`SetCameraFlyMode(true)` saves the current transform, copies current Euler rotation into the fly-camera rotation fields, disables the Animator, and leaves the camera in place for manual control.

`SetCameraFlyMode(false)` re-enables the Animator, hides the zoom-out button, restores the Side-by-Side toggle, and calls `StartResetZoom()`.

Manual fly-camera rotation is clamped to:

- X rotation: 0 to 180.
- Y rotation: 100 to 240.

Key Objects And Scripts

- `CameraController`: input handling, Animator triggers, fly-camera mode, and pause/unpause handoff.
- `GameController`: reads `CameraController.pauseState`, controls UI visibility, enters/exits Side-by-Side Mode, and performs full camera resets.
- `Camera Animator Controller`: owns the named animation clips and transitions.
- `Scene cube tags`: `AggregateCube` and numbered cube tags used to infer cube identity.

Current Constraints

- Camera animation assumes one aggregate cube plus five sample cubes.
- Trigger names are hard-coded in `CameraController` and must match the camera Animator Controller exactly.
- `moveLength` is taken from the first animation clip, based on the assumption that all camera animation clips have the same length.

- Cube identity is inferred from tag names rather than scenario data.
- Click-to-reset while zoomed is disabled; reset is currently through UI or keyboard path.
- Fly-camera rotation limits are hard-coded for the current scene framing.

Cube Design

Source: Specs/CubeDesign.md

Last updated: 2026-06-16

Design Purpose

Terrain cubes are the close-up, inspectable units of the Future Mountain experience. The large landscape shows watershed-scale change, while cubes make model processes visible at a human-readable scale: vegetation growth, snow, streamflow, groundwater, roots, soil state, fire effects, and scenario comparison.

The original concept design suggested a stronger object-like cube treatment, including an almost glass enclosure around each cube. The current implementation does not use visible glass walls or a strong transparent container. The visual direction so far has favored open terrain blocks with visible ecological systems over a literal display-case treatment. This keeps the cubes readable, reduces material and transparency complexity, and avoids adding visual clutter around vegetation, particles, snow, water, and fire.

User Interface Behavior

In Normal Mode, the scene presents one aggregate cube and five individual sample cubes alongside the large landscape. Users can click a cube to zoom in and inspect it. In Side-by-Side Mode, a selected cube can be compared against its matching side cube under another warming scenario.

Each cube is intended to feel like a small self-contained terrain system rather than a simple chart. The cube surface, vegetation, roots, snow, water, particles, and fire state all update over simulation time.

Technical Structure

Each active cube is built around its own `CubeController`. The controller owns or coordinates the cube's model data, terrain state, vegetation objects, water and soil visuals, fire behavior, labels, and statistics/model-data display.

The cube hierarchy is scenario-authored in Unity rather than generated entirely from code. A typical cube setup includes:

- A Unity Terrain or terrain-like object for the cube surface.
- A cube-specific `TerrainData` asset.

- A CubeController.
- A soil/groundwater visual subtree.
- Stream/water objects driven by cube flow data.
- Vegetation containers for trees, shrubs, grass, litter, and roots.
- Particle systems for precipitation, groundwater movement, and evapotranspiration.
- A cube-specific fire manager object.
- Labels or UI targets used by messages, camera focus, and model data display.

The scene also contains side-cube variants used by Side-by-Side Mode. These are normally hidden or inactive in Normal Mode, but they still need their own terrain assets and scenario-specific setup because they become active during comparison.

Per-Cube Ownership

Each cube should own the scene assets that can change during simulation or scenario tuning. In practice, this means each cube should have its own terrain, fire manager, and mutable visual setup. Side cubes should be treated the same way, even when they are hidden by default.

This is especially important for scenario duplication. Unity TerrainData, materials, prefabs, terrain layers, and fire settings can be shared accidentally between scenes. If a new scenario edits a shared asset, that change can bleed back into older scenarios. When a cube needs scenario-specific tuning, duplicate the relevant asset and reassign the cube to the duplicate.

Fire And Terrain

Cube fire is separate from landscape fire. The large landscape has its own fire manager and terrain state, while each burnable cube uses its own fire manager and cube-local terrain update path. This allows cube vegetation and cube terrain to respond to fire without requiring the large landscape's fire grid to map perfectly onto cube-local geometry.

The current cube fire behavior is less spatially data-driven than the landscape fire behavior. Cube fire primarily uses cube model data to decide vegetation loss and visual fire impact, while the cube fire grid itself burns locally.

Relationship To Data

Cube design is tightly coupled to CubeData. The following fields are among the main drivers of cube visuals:

- snow: cube snow surface and precipitation-to-groundwater effect.
- qout: stream height and flow presentation.
- depthToGW: groundwater/soil wetness cue.
- vegAccessWater: surface soil water availability.

- leafC*, stemC*, rootC*: tree, shrub, and root size/count behavior.
- transOver and transUnder: evapotranspiration particle behavior.
- fire and burn-related data: vegetation loss and burned terrain state.

The cube is therefore not just a miniature terrain mesh. It is an authored container for many simultaneous data mappings.

Current Constraints

- The current experience assumes one aggregate cube plus five individual sample cubes.
- Side-by-Side Mode assumes corresponding side-cube instances exist for the aggregate cube and each sample cube.
- Cube identity is still scene/tag driven in several places rather than fully scenario-data driven.
- The glass/display-case concept remains a design option, not an implemented visual feature.
- Transparent cube enclosures would need careful testing with vegetation, particles, snow, water, fire, selection rays, WebGL performance, and camera framing before adoption.
- Scenario duplication must include cube terrains, side-cube terrains, and cube-specific fire/visual assets where those assets will be edited.

Related Specs

- Specs/NormalMode.md
- Specs/SideBySideMode.md
- Specs/CameraAnimation.md
- Specs/DataMappings.md
- Specs/Fire.md
- Docs/AddingFutureScenarios.md

Lighting

Source: Specs/Lighting.md

Last updated: 2026-06-12

User Interface Behavior

Lighting changes as simulation time advances. The sun angle and intensity shift seasonally, giving a visual cue for time of year. This is especially important because users can accelerate time or jump through the timeline.

There is no direct user-facing sun control in the current version. Lighting is tied to simulation date.

Technical Behavior

Lighting is controlled by GameController.

Key fields:

- `sunLight`: directional light.
- `summerLightIntensity` = 1.525f
- `winterLightIntensity` = 1.3f
- `summerAltitudeAngle` = 71.35f
- `summerAzimuthAngle` = 132.65f
- `winterAltitudeAngle` = 29.48f
- `winterAzimuthAngle` = 181.19f

The code comments note that these angles came from SunCalc. They are treated as representative seasonal solar positions.

Seasonal Transition

`InitSunTransition()` sets the starting transition direction based on the simulation start month/day and the solstices:

- June 21: summer reference.
- December 21: winter reference.

`UpdateSunTransition()` computes the current day of year, decides whether the simulation is moving from summer to winter or winter to summer, then linearly interpolates:

- Altitude angle.
- Azimuth angle.
- Light intensity.

The final light transform is:

```
sunLight.transform.localEulerAngles = (altitude, azimuth, existingZ)
```

The code adds 180f to azimuth when applying the orientation, then wraps if it exceeds 360 degrees.

Runtime Update

`RunGame()` calls `UpdateLighting()` during simulation updates and also between simulation frames when seasons are displayed. This keeps lighting visually moving even when the simulation timestep is not advancing every rendered frame.

Current Constraints

- Angles are hard-coded for the Big Creek experience.

- Seasonal interpolation is based on solstice endpoints, not full astronomical recalculation per date/location.
- Future scenarios should decide whether to keep this stylized seasonal lighting or compute location-specific sun paths from scenario latitude/longitude.

Interaction Modes

Normal Mode

Source: Specs/NormalMode.md

Last updated: 2026-06-12

User Interface Behavior

Normal Mode is the main simulation view. It shows the full Big Creek landscape, the aggregate cube, and five individual terrain cubes. The user can start the simulation, select a warming scenario, pause or resume time, control time speed, inspect cubes, show or hide model/statistics overlays, show or hide narrative messages, and use the timeline.

The primary warming control is a single knob with five discrete values:

- 0 C
- +1 C
- +2 C
- +4 C
- +6 C

When the simulation starts, the intro panel hides, loading UI appears, runtime data is loaded, and the user is asked to continue into the main experience. Cubes animate into view after loading. In normal mode, clicking a cube zooms the camera toward that cube unless the Side-by-Side toggle is armed.

Technical Behavior

GameController is the coordinator for Normal Mode. It owns current time, current warming index, cube arrays, side-cube arrays, UI canvases, messages, timeline, lighting, and fire scheduling.

Startup flow:

1. `StartSimulationRun()` hides setup UI and reads the selected warming index from `WarmingKnobSlider`.
2. Web builds call `LandscapeController.LoadLandscapeDataForWarmingIdx(warmingIdx)` and `WebManager.GetDataDates()`.

3. `FinishStarting()` initializes each cube, side cube, aggregate cube, messages, fire dates, timeline, and initial sun position.
4. `HandleContinueButtonPressed()` transitions from loading into the interactive simulation.

Runtime flow:

1. `RunGame()` advances `timeIdx` by `timeStep` while not paused.
2. `UpdateSimulation()` updates the current `DateModel`, checks fire ignition, updates the landscape, updates cubes, updates messages, updates aggregate cubes, and records the last update time.
3. `UpdateLighting()` updates seasonal sun angle and intensity between simulation frames.
4. `UpdateUIText()` writes the current date to the timeline label.

Normal Mode uses the main cube array (`cubes`) and aggregate cube. Side cube instances are normally hidden but are initialized so Side-by-Side Mode can activate them quickly.

Key Objects And Scripts

- `GameController`: mode state, simulation clock, UI, fire scheduling, timeline integration.
- `LandscapeController`: full landscape terrain, river, fire, snow, and patch/water data.
- `CubeController`: per-cube data and visual simulation.
- `CameraController`: click/zoom behavior and fly camera mode.
- `WarmingKnobSlider`: discrete scenario selector.
- `TimeKnobSlider`: time-step/speed selector.
- `TimelineControl`: annual timeline and click targets.
- `UI_MessageManager`: story/fire messages and cube label highlights.

Scenario Assumptions

Normal Mode currently assumes one aggregate cube plus five individual cubes. It also assumes the five Big Creek warming levels and a Big Creek data/API contract. Future scenarios should make these values scenario-configurable.

Side-by-Side Mode

Source: Specs/SideBySideMode.md

Last updated: 2026-06-12

User Interface Behavior

Side-by-Side Mode lets the user zoom in on one cube and compare two warming scenarios for that same cube at the same simulation time.

The intended interaction is:

1. User enables the Side-by-Side toggle.
2. User clicks an aggregate or individual cube.
3. Camera zooms to a Side-by-Side framing.
4. Other cubes hide or shrink away.
5. Two versions of the selected cube are shown.
6. Two warming knobs appear, one for the left/original cube and one for the right/comparison cube.
7. The regular warming knob hides.
8. User can change each cube's warming scenario independently.
9. Timeline, pause, messages, lighting, and simulation time continue to apply.

The comparison cube defaults to a different warming index from the original: if the original is baseline, the comparison starts at +1 C; otherwise it starts at baseline.

Technical Behavior

Side-by-Side Mode is controlled by `GameController.sideBySideMode` and `GameController.sbsIdx`.

Entry flow:

1. `CameraController.StartZoomIntoCube(cubeIdx)` checks whether Side-by-Side should be entered.
2. It triggers a Side-by-Side camera animation, such as `SBS_Cube0` for aggregate or `SBS_Cube1` through `SBS_Cube5` for individual cubes.
3. `GameController.EnterSideBySideMode(idx)` stores `sbsIdx`, sets `sideBySideMode = true`, hides other cubes, chooses the active main cube and matching side cube, shows two Side-by-Side warming knobs, hides the regular warming knob, and enables the Side-by-Side canvas.
4. `FinishEnteringSideBySideMode()` starts simulation on each selected cube at the current `timeIdx`, assigns the message manager, calls `CubeController.EnterSideBySide()`, and optionally initializes vegetation for the comparison cube.

Changing either Side-by-Side warming knob calls:

- `GameController.SetSBSSWarmingLevel(newIdx, newDegrees, isComparedCube)`

That method resets the targeted cube, sets its warming index, and restarts its simulation at the current shared `timeIdx`.

Exit flow:

1. `ExitSideBySideMode(immediate)` sets `sideBySideMode = false`.
2. The active comparison cube is stopped.
3. Side cubes and Side-by-Side warming knobs are hidden.
4. The regular warming knob is restored.
5. Main cubes are reset to the global warming index.
6. Statistics panels are hidden/reset.
7. If exit is not immediate, cubes are shown again and camera reset animation runs.

Key Objects And Scripts

- `GameController`: owns `sideBySideMode`, `sbsIdx`, entry/exit flow, and per-side warming changes.
- `CameraController`: decides whether a cube click means zoom-only or Side-by-Side entry.
- `CubeController.EnterSideBySide()`: prepares cube-specific UI/statistics and cube position/state.
- `WarmingKnobSlider`: has Side-by-Side flags and calls `SetSBSWarmingLevel()`.
- Side cube scene objects: one duplicate for each cube and one duplicate aggregate cube.

Current Constraints

The feature assumes exactly five individual cubes plus one aggregate cube. The `WarmingKnobSlider` code comments note that Side-by-Side knob behavior still needs generalization. Future scenario work should make cube identity and comparison defaults data-driven.

Timeline

Source: Specs/Timeline.md

Last updated: 2026-06-16

User Interface Behavior

The timeline is visible during the simulation and works in both Normal Mode and Side-by-Side Mode. It shows annual precipitation as a bar graph, displays the current simulation date, and includes icon rows for fire years and message years.

User interactions:

- Hovering over timeline bars changes bar highlighting.

- Clicking a timeline bar jumps the simulation to that year.
- Clicking a fire icon jumps to the fire year.
- Clicking a message icon jumps to the message year.

The timeline does not currently jump to the exact day of a message or fire icon. It sets the simulation to January 1 of the selected year through `GameController.SetTimePosition()`.

Technical Behavior

`TimelineControl` creates and updates the timeline. It stores a `clickedID` year offset that `GameController.RunGame()` reads and consumes.

Timeline creation paths:

- `CreateTimeline(...)`: non-web/local path, using formatted `WaterDataYear` objects.
- `CreateTimelineWeb(...)`: web path, using `PrecipByYear[]` from `waterdata/total`.
- `CreateTestTimeline(...)`: fallback/test path when landscape simulation is off.

For web builds:

1. `GameController.GetTimelineWaterDataFromWeb()` calls `WebManager.GetTimelineWaterData()`.
2. `SetTimelineWaterData()` deserializes the response into `TimelineWaterData`.
3. `TimelineControl.CreateTimelineWeb()` instantiates bars and icons.

`WebManager` applies the active `SimulationSettings.apiProfile` to this request, so Big Creek uses `/api/waterdata/total` and Central Coast uses `/api/centralcoast/waterdata/total`.

Each year is represented by a `graphBarPrefab` named `Point_{index}`. Fire icons are named `Fire_{year}`. Message icons are named `Message_{year}_{warmingDegrees}`.

Data Display

Annual precipitation determines bar height. In both local and web paths, the maximum precipitation used for scaling is computed from the loaded yearly precipitation values. The web path no longer uses the old hard-coded 2582f maximum.

Fire and message years are passed in from `GameController`:

- `fireYears` comes from scheduled fire dates.
- `messageYears` comes from loaded message files filtered by active warming degrees.

Mode Interaction

Timeline time is global. In Normal Mode, a timeline jump updates the main landscape, aggregate cube, and visible cubes. In Side-by-Side Mode, the same global `timeIdx` is used

for both comparison cubes, preserving same-time comparison while warming levels can differ.

Key Objects And Scripts

- **TimelineControl**: graph creation, icon creation, hover/click handling, current-year highlighting.
- **GameController**: reads `clickedID`, converts selected year to `timeIdx`, updates landscape and cubes.
- **WebManager**: fetches web precipitation totals.
- **TimelineWaterData** and **PrecipByYear**: web timeline DTOs.

Current Constraints

- Timeline click resolution is year-level.
- Icon naming encodes year and warming degrees.
- Message icon lookup uses the message manager and active warming degree conventions.

Simulation Layers

Fire

Source: Specs/Fire.md

Last updated: 2026-06-16

Conceptual Model

RHESSys fire dates are intentionally set by the scenario. In the current Big Creek version, the fires are scheduled on:

- July 15, 1969
- November 20, 1988

Those dates are not emergent from climate or vegetation data inside the Unity simulation. They are scenario events.

However, once a scheduled fire begins, its extent/severity/spread is data-driven. Big Creek fire spread differs by warming scenario because the loaded fire data for each warming index defines which patches/cells burn and in what order/intensity.

User Interface Behavior

When time reaches a scheduled fire:

- The large landscape can ignite a data-driven fire pattern.

- Relevant cubes can ignite if they correspond to affected patches or are explicitly configured to burn.
- The fire may auto-pause the simulation if `AutoPauseOnFire` is enabled.
- Fire icons appear on the timeline at fire years.
- Fire messages can appear through the message system.
- Burned terrain changes appearance, then may visually regrow/recover over time.
- Vegetation in cubes can burn, die, shrink, or be removed based on the fire effect on visualized biomass.

Scheduled Fire Dates

`GameController.SetupFires()` currently hard-codes two `Vector3(month, day, year)` dates:

- (7, 15, 1969)
- (11, 20, 1988)

It converts those dates into `timeIdx` values using `GetTimeIdxForDay()`, stores them in `fireFrames`, and sends the dates to `LandscapeController.SetupFires()`.

The code comments mark this as a future refactor target: fire dates should come from scenario data/configuration.

Fire Ignition Flow

`GameController.UpdateFireIgnition()` checks the current `timeIdx` against `fireFrameSet`. If the time step skips over a fire frame, it also scans the interval between the last and current time index so fires are not missed at higher simulation speeds.

When a fire should start:

1. `pausedAuto` is set if `SimulationSettings.AutoPauseOnFire` is true.
2. `LandscapeController.IgniteTerrain(date, timeStep, autoPause, maxFireLength, fireFrameIdx)` ignites the large landscape fire.
3. Cube controllers decide whether to ignite through `CubeController.ShouldBurnFireOnDate(date)`.
4. `CubeController.IgniteTerrain(fireTimeIdx, useThresholds)` ignites cube terrain and vegetation.
5. Aggregate cube ignition is checked separately.

Data-Driven Landscape Spread

Landscape fire data is loaded by warming index:

- Web path: `WebManager.RequestFireData(warmingIdx)` calls `firedata/{warmingIdx}`.

- Local/file path: Resources fire data can be loaded from `FireData/fireDataList_{warmingIdx}`.

`LandscapeController.FinishUpdateFireDataFromWeb()` deserializes API records into `FireDataFrame` objects. Each frame contains date/grid metadata and a list/grid of `FireDataPoint` values.

`SERI_FireManager.Initialize(...)` receives fire frames and creates one `SERI_FireGrid` per fire frame. In data-controlled mode, `SERI_FireGrid` uses `FireDataPoint` and `FireDataPointCollection` data to decide active cells, patch IDs, spread values, and burn order.

Importer Contract

`FireData` means Unity-compatible fire playback frames. It should contain event dates, landscape fire grid dimensions, and serialized `FireDataPoint` values with patch/zone id, spread, and iter/order. This is the data used for instantaneous fire animation after a scheduled ignition.

`BurnData` is separate. In Central Coast v2, monthly RHESys burn values from `bm.csv` and `spatial_data_point_patchvar.csv` are imported with `--burn` into `BurnData`. These rows can inform terrain state, but they are not fire spread/iter playback frames.

The Central Coast v2 importer now has a `--fire` scaffold. The scenario config reserves two file roles for future fire-frame sources:

- `fireFrameSpreadIter`: event rows with date, patch/zone id, spread, and iter/order.

That role is empty in the current Central Coast sample bundle because no Central Coast fire-frame source has been provided yet. Central Coast fire-frame generation should reuse existing `PatchData` as the landscape patch/zone grid map; `PatchData` is currently derived from `patchFamilyRaster`.

Cube Fire Behavior

Cube fires are less spatially data-driven than landscape fires. The cube fire grid is ignited immediately because cube-level spread data is not currently used. The data-driven part is the amount of vegetation to remove:

- `SetVegetationToDieFromFire()` compares current visualized carbon against data values at the fire time index.
- It calculates `shrubsToKill` and, for two-layer vegetation cubes, `firsToKill`.
- `SetShrubsToBurn()` ignites shrub `SERI_FireNodeChain` objects.
- `SetTreesToBurn()` ignites selected `FirController` fire node chains.
- Once burning ends, cube terrain is set to a burned splatmap.

Fire Animation Architecture

Main fire scripts:

- `SERI_FireManager`: parent coordinator attached under terrain/cube fire manager objects. Stores terrain references, terrain texture rules, fire prefab, grid size, cell size, fire grids, and terrain restoration data.
- `SERI_FireGrid`: creates a grid of fire cells over a terrain, tracks burning cells, starts cells, updates combustion, and handles data-controlled spread.
- `SERI_FireCell`: one cell in the grid. Instantiates pooled fire prefabs, holds fuel/combustion state, sets visual state, and extinguishes/returns pooled objects.
- `SERI_FireVisualManager`: controls particle systems on each fire prefab. It switches heat-up, ignition, and extinguish visual states, randomizes fire, and scales particle parameters.
- `SERI_FireNodeChain`: attached to burnable vegetation objects. It links one or more `SERI_FireNode` objects and can destroy/replace vegetation after fire.
- `SERI_FireNode`: fire node on vegetation; consumes fuel and controls local fire visuals.
- `SERI_FireTerrainTexture`: terrain texture/fire fuel settings.
- `SERI_FireGrassRemover`: removes/replaces grass details after burning.
- `SERI_FireIgniter` and `SERI_FireBox`: legacy/adapted helpers from the Fire Propagation System asset.

Scene fire manager objects are named by cube/terrain, such as `FireManager_A` through `FireManager_F` and `FireManager_T`.

Burned Terrain And Regrowth

Landscape fire state is tracked with:

- `terrainBurning`
- `terrainBurnt`
- `recentFire`
- `fireRegrowthStartTimeIdx`
- `fireRegrowthLength`

After fire stops, `SetToBurnt()` marks the terrain burned and stops grid fires.

`UpdateTerrain()` then blends burned/unburned state through a `regrowthAmount`. Snow and burned texture weights interact in the landscape splatmap.

Current Constraints

- Fire dates are hard-coded in Unity.
- Big Creek has exactly two scheduled fire dates.

- Landscape fire spread is data-driven by warming index; cube spread is visually immediate and biomass-driven.
- The fire system is adapted from a third-party Fire Propagation System and has project-specific SERI modifications.
- Future scenarios need explicit fire-date configuration and a concrete fire-frame source format that can populate FireData.

Ground Water

Source: Specs/GroundWater.md

Last updated: 2026-06-12

Purpose

Ground water is a cube-level visual cue for depth to groundwater. It shows hidden water availability below the terrain surface and helps explain why vegetation may grow, decline, or experience water stress.

The current implementation represents groundwater as colored water-pocket objects embedded in the soil portion of each cube.

Visual Behavior

Groundwater objects are named GroundWater1, GroundWater2, and so on under a cube's Soil object. They do not physically fill upward or move during playback. Instead, each object changes color and material wetness according to the current mapped groundwater level.

The intended visual effect is:

- Wet/filled pockets when groundwater is high enough for that pocket.
- Dry or less saturated pockets when groundwater is too deep.
- Smooth color blending near the transition.

Data Inputs

Groundwater visualization uses:

- depthToGW: depth to groundwater from cube model data.

CubeController loads this into DepthToGW and passes it to SoilController.

Technical Flow

SoilController.Start() discovers groundwater pocket objects by repeatedly looking for child names:

GroundWater1
GroundWater2
GroundWater3
...

For each pocket it stores:

- Renderer.
- Material.
- Local Y position.

It initializes each groundwater material with:

```
_Metallic = 0  
_Smoothness = 0.5
```

During playback, `CubeController.UpdateVegetationBehavior()` calls:

```
soilController.UpdateSimulation(timeIdx, timeStep, WaterAccess, DepthToGW)
```

`SoilController.UpdateSimulation()` then calls `UpdateGroundwater()`.

Depth Mapping

`UpdateGroundwater()` normalizes the current `depthToGW` against:

- `DepthToGWMin`
- `DepthToGWMax`

It maps that value into a normalized groundwater position from 0 to 0.75. Each groundwater object maps its own local Y position between:

- `deepSoilBottomYPos`
- `deepSoilTopYPos`

The difference between mapped groundwater level and object position determines whether the object is displayed as wet, dry, or in transition.

Material Mapping

Groundwater uses two HSV-defined colors:

- `gwWet`
- `gwDry`

The code converts those to Unity colors at startup. During updates:

- Objects clearly above/beyond the mapped level are assigned the dry color and higher metallic value.

- Objects clearly below/within the mapped level are assigned the wet color and lower metallic value.
- Objects near the threshold blend between wet and dry colors.

This gives the appearance of pockets filling or draining without moving geometry.

Related Systems

Groundwater is visually related to the precipitation-to-groundwater effect described in `Snow.md`. Cube snow can activate `RainToGW_Prefab`, while groundwater pockets show the current mapped groundwater-depth state.

These are separate systems:

- Snow/rain-to-groundwater is a particle/pathway effect.
- Groundwater pockets are persistent soil objects whose material state changes.

Current Constraints

- Groundwater pockets do not physically fill, deform, or move.
- The visual mapping is relative to min/max values calculated from loaded cube data.
- The color comments in `SoilController` are partly inconsistent with the wet/dry variable names; behavior should be verified visually before changing colors.
- The current system is cube-local and does not render watershed-scale groundwater on the large landscape.

Roots

Source: *Specs/Roots.md*

Last updated: 2026-06-12

Purpose

Roots are the below-ground companion visualization for tree growth. They make vegetation carbon visible as more than trunks and leaves, and help communicate that climate, water access, and biomass changes affect the hidden root system as well as the visible canopy.

Roots are currently tied to fir/tree-capable cubes. They are not a separate user interface control in the current version.

Visual Behavior

Root objects appear under fir trees in terrain cubes. As trees become established and grow, the active root prefab and its scale change to represent increasing root depth and spread.

Visually, roots should read as below-ground root structures, not as dead trees, branch clusters, or above-ground vegetation. Runtime root transforms should preserve this orientation even when tree objects are randomly rotated.

Data Inputs

Root behavior is driven by cube vegetation data:

- `rootCOver`: overstory root carbon.
- `rootCUnder`: understory root carbon, loaded where available but less directly represented by current tree root visuals.

For tree-capable two-layer vegetation cubes, overstory root carbon is the main root signal. Aggregate cubes use the same overstory/understory-style fields with aggregate-specific scaling factors.

Technical Flow

`CubeController` reads root data into:

- `RootsCarbonOver`
- `RootsCarbonUnder`
- `RootsCarbonOverMin`
- `RootsCarbonOverMax`
- `RootsCarbonUnderMin`
- `RootsCarbonUnderMax`

When firs are created, `CubeController.InstantiateTreeFromPrefab()` creates the tree object, tree LOD groups, dead-tree LOD group, and one root object per root prefab. The root objects are named `Roots_0`, `Roots_1`, and so on so `TreeController` can find and switch between them.

`FirController.UpdateSimulation(...)` receives root carbon with transpiration, leaf carbon, and stem carbon. It compares the model value to the currently visualized root carbon and calls `GrowRoots()` when the visual root system needs to catch up.

`TreeController` owns root state:

- `rootsHeightScale`
- `rootsWidthScale`
- `rootsFullHeightScale`
- `rootsFullWidthScale`
- `rootsPrefabIdx`
- `curRootsObject`

`TreeController.UpdateRootsPrefab()` chooses the closest root prefab for the current depth. `SetCurrentRootsPrefab()` deactivates the previous root object and activates the new one. `UpdateRootsLODsScale()` scales the active root LOD children.

Prefabs And Scaling

Root meshes come from:

Assets/Prefabs/Vegetation/Components/Roots/Roots_Incremental/

The current implementation uses incremental root prefabs named `Roots_001` through `Roots_010`. Their default renderer bounds are measured during `TreeController.InitializePrefabs()` and stored as root prefab depths and widths.

Root scale settings live in `SimulationSettings`:

- `MinRootsFullHeightScale`
- `MaxRootsFullHeightScale`
- `MinRootsFullWidth`
- `MaxRootsFullWidth`
- `RootsSpreadSpeedFactor`
- `RootsSizeRatio`
- `RootsYOffsetFactor`
- `RootsWidthVariability`
- `RootsCarbonFactor`
- `CubeARootsCarbonFactor`

Web optimization can multiply root carbon factors through `SimulationSettings.OptimizeForWeb()`.

Known Constraints

- Roots are implemented as discrete prefab swaps plus scale changes, not procedural geometry.
- The mapping is an approximate visual representation of root carbon, not a literal root architecture model.
- Current root carbon visualization is strongest for overstory/tree roots.
- Root orientation is sensitive to prefab rotations and parent transforms. Runtime code should instantiate roots as local children of their tree and set local transform intentionally.

Snow

Source: *Specs/Snow.md*

Last updated: 2026-06-12

User Interface Behavior

Snow appears on the landscape and cubes as a visible indicator of changing water/climate conditions. In Big Creek, snow is central to the experience because the Sierra Nevada watershed depends heavily on snowpack dynamics.

This system is likely Big Creek-specific, but it is important to preserve because future mountain or high-elevation scenarios may reuse it.

Technical Behavior

Snow is implemented in two related ways:

- Cube snow uses each cube's snow data value and a per-cube `SnowManager`.
- Landscape snow uses terrain splatmaps and, in some modes, background `SnowManager` objects.

The project uses `SnowManager` components from the included Dynamic Snow/NatureManufacture-related assets.

Cube Snow

`CubeController` reads snow from cube data:

- Web/API path: `CubeData.snow`.
- TextAsset path: `DataColumnIdx.Snow` or `AggregateDataColumnIdx.Snow`.

It tracks `SnowAmountMin` and `SnowAmountMax` from data ranges. Runtime snow value is mapped into a `SnowManager.snowValue`, clamped to a visual range.

Cube snow also interacts with precipitation-to-groundwater visual effects:

- `UpdatePrecipToGW(float snowValue)` activates the rain/snow-to-groundwater prefab when snow value is greater than zero.

Landscape Snow

`LandscapeController` owns:

- `snowManager` for `SnowManager_Landscape`.
- `snowManagerBkgd` for `SnowManager_Background`.
- `snowWeightFactor`.
- `backgroundSnowFactor`.
- `averageSnowAmount`.
- `SnowAmountMax`.
- `AvgSnowAmountMin` and `AvgSnowAmountMax`.

Scripting symbols affect background snow:

- LOCAL_VERSION: backgroundSnowOn = true.
- WEB_VERSION: backgroundSnowOn = true.
- Default: backgroundSnowOn = false.

When background snow is on, landscape simulation can load terrain data through `terraindata/{warmingIdx}` and build monthly/current splatmaps.

`FinishUpdateTerrainDataFromWeb()` converts API terrain data into `TerrainDataFrame` objects and then into splatmaps.

Splatmap Behavior

Landscape terrain uses splatmap texture weights to represent:

- Unburned/no snow.
- Unburned/snow.
- Burned/snow.
- Burned/no snow.

In non-web/local paths, `BuildTerrainSplatmapForDay()` interpolates between monthly patch data, maps snow to a normalized weight, and blends snow with burned/regrowth state.

In web/background-snow paths, `UpdateTerrain()` looks up precomputed monthly splatmaps, interpolates them by day/month/year, and applies the result to `terrain.terrainData.SetAlphamaps(...)`.

Fire Interaction

Snow and fire share terrain texture blending. Burned terrain can still show snow through burned-snow texture weights. During post-fire recovery, `regrowthAmount` gradually shifts weights back toward unburned terrain.

Current Constraints

- Landscape web snow behavior is split by `backgroundSnowOn`, and WebGL currently enables the terrain-data snow path.
- Some background snow update code is commented out.
- A hard-coded landscape start year of 1942 appears in the web/background snow month-index calculation.
- Snow texture layer meaning is implicit in code comments and terrain setup.
- Future scenarios should explicitly decide whether they use cube snow only, full landscape snow, no snow, or a replacement seasonal surface system.

Soil

Source: *Specs/Soil.md*

Last updated: 2026-06-12

Purpose

Soil is the cube-level visual layer that shows near-surface wetness and deeper soil moisture state. It helps connect abstract RHESSys water variables to visible ground conditions around vegetation.

In the current version, soil is primarily a material/shader-driven visual system rather than a geometry-changing system.

Visual Behavior

Soil wetness is communicated through shininess/reflectivity:

- Surface soil objects become glossier as vegetation-accessible water increases.
- The front deep-soil plane changes its material metallic value based on groundwater depth.

This makes wetter soil read as darker, shinier, or more reflective depending on the material and lighting setup.

Data Inputs

Soil visualization uses cube data fields:

- `vegAccessWater`: vegetation-accessible water in surface soil.
- `depthToGW`: depth to groundwater.

The `soil` data field is loaded and tracked in data models, but this audit did not find it driving a current direct visual effect. Current soil visuals are water-focused.

Technical Flow

`CubeController` reads water values into:

- `WaterAccess`
- `DepthToGW`

During startup and playback, `CubeController` calls:

```
soilController.UpdateParams(timeStep, WaterAccess, DepthToGW)
soilController.UpdateSimulation(timeIdx, timeStep, WaterAccess, DepthToGW)
```

`SoilController` finds its child renderers at startup:

- `FrontPlane` for deep/front soil.
- `SurfaceSoil1`, `SurfaceSoil2`, etc. for shallow surface soil patches.
- `GroundWater1`, `GroundWater2`, etc. for groundwater pockets.

It stores each renderer's material so it can update shader parameters during the simulation.

Surface Soil Mapping

`SoilController.UpdateSimulation()` maps `waterAccess` from `[WaterAccessMin, WaterAccessMax]` into:

- `minShallowSoilShininess`
- `maxShallowSoilShininess`

It writes the mapped value to each surface soil material:

`_Glossiness`

Higher `vegAccessWater` therefore makes shallow soil appear wetter or shinier.

Deep Soil Mapping

`SoilController.UpdateSimulation()` also maps groundwater-related state into a deep-soil shininess value and writes it to the front soil material:

`_Metallic`

This is a visual cue for deeper wetness rather than a literal soil-carbon display.

Current Constraints

- The current code maps deep-soil shininess using `waterAccess` while checking `depthToGW` bounds. This may be intentional visual tuning or a historical bug; future refactors should confirm the desired mapping.
- Soil carbon is loaded but not strongly visualized in the current implementation.
- Soil materials depend on Unity shader parameters such as `_Glossiness` and `_Metallic`; changing material/shader types can break the wetness cue.
- The behavior is cube-local and does not directly change the large landscape terrain.

Data And Messaging

Data Mappings

Source: *Specs/DataMappings.md*

Last updated: 2026-06-12

Purpose

This spec explains how Future Mountain data fields become 3D visual behavior over time. It complements [DataModel.md](#), which describes the shape of the data. This document

focuses on runtime interpretation: which values drive which objects, materials, particles, terrain textures, UI displays, and time-based behaviors.

Mapping Overview

Data source	Field(s)	Primary visual target	Update style
waterdata	qBase, qWarm1, qWarm2, qWarm4, qWarm6	Large landscape river height/face	Direct mapped value by warming index
cubedata	qout	Cube stream height/face	Direct mapped value, log-scaled
cubedata	snow	Cube snow surface and precipitation-to-groundwater particle effect	Accumulating/melting value over time
patchdata / terraindata	snow, terrain splatmap data	Large landscape snow texture/surface	Splatmap blending/interpolation
cubedata	depthToGW	Cube groundwater color/wetness	Range-mapped material color/metallic
cubedata	vegAccessWater	Cube surface soil wetness	Range-mapped material glossiness
cubedata	transOver, transUnder	Tree/shrub evapotranspiration particles	Particle emission rate
cubedata	leafC*, stemC*, rootC*	Tree count, tree size, shrub count/size, roots	Feedback system over time; full regeneration on jumps
cubedata	litter	Litter/dead vegetation state	Partial/unfinished; litter objects shrink over time
cubedata	netpsn	Model/statistics data	Stored/ranged, limited visible mapping
firedata	fire frames, fire points, spread/iter	Landscape fire grid/cells and burned terrain	Data-driven fire spread after scheduled ignition
dates	year, month, day, id	Time progression, timeline, sun position, event	Global clock/date index

Data source	Field(s)	Primary visual target	Update style
		matching	

Time Model

GameController owns the global simulation index, `timeIdx`. During playback, `RunGame()` increments `timeIdx` by `timeStep` when the simulation is not paused and not auto-paused during fire. `UpdateSimulation()` then applies the current date to the landscape, cubes, messages, timeline, fire, and lighting.

Timeline jumps use a different path. When the user clicks a timeline year, `TimelineControl.clickedID` is read by GameController, which calls `SetTimePosition(selectedYearIdx)`. That sets `timeIdx` to January 1 of the selected year, then forces relevant systems to re-read data for the new time.

This distinction matters most for vegetation:

- Playback: compare current visual vegetation against current model data and grow/kill incrementally.
- Timeline jump: reset the cube and regenerate vegetation from data for the destination time.

Landscape Water Mapping

Large landscape river flow comes from `waterdata`.

The current model stores one streamflow value per warming level:

- `QBase`
- `QWarm1`
- `QWarm2`
- `QWarm4`
- `QWarm6`

`WaterDataFrame.GetStreamflowForWarmingIdx(warmIdx)` maps warming index to the correct streamflow field. `LandscapeController.UpdateData()` stores the current value in `StreamflowLevel`.

`LandscapeController.UpdateRiver()` maps `StreamflowLevel` into:

- `riverObject.transform.localPosition.y`
- `riverFaceObject.transform.localPosition.y`
- `riverFaceObject.transform.localScale.y`

The mapping uses `MapValue(StreamflowLevel, RiverLevelMin, RiverLevelMax, ...)` and clamps to scene-configured min/max heights/scales.

Cube Stream Mapping

Cube stream flow comes from `cubedata.qout`, stored as `CubeController.StreamHeight`.

`CubeController.UpdateStream()` maps it through a logarithmic scaling step:

```
streamPos = MapValue(Log(StreamHeight) * 20, StreamHeightMin, StreamHeightMax, 0, 1)
```

Then it maps `streamPos` to:

- `streamObject.transform.localPosition.y`
- `streamFaceObject.transform.localScale.y`

This means cube streams are not a purely linear visualization. The log scaling makes smaller changes more visible across a broad streamflow range.

Cube Soil And Groundwater Mapping

Cube soil uses:

- `vegAccessWater -> WaterAccess`
- `depthToGW -> DepthToGW`

`CubeController.UpdateVegetationBehavior()` calls:

```
soilController.UpdateSimulation(timeIdx, timeStep, WaterAccess, DepthToGW)
```

`SoilController` maps `WaterAccess` to shallow soil material glossiness. Higher water access makes surface soil appear wetter/shinier.

`DepthToGW` is mapped against groundwater object vertical positions. Groundwater objects fade between wet and dry colors based on whether the mapped groundwater level is above or below each object. This is a visual depth cue rather than literal geometry movement.

Cube Snow Mapping

Cube snow uses `cubedata.snow`, stored as `SnowAmount`.

On simulation start, the current snow amount is mapped into `snowManager.snowValue`. During playback, snow behaves like an accumulated visual state:

1. Existing `snowValue` melts by `snowMeltRate * sqrt(timeStep)`.
2. Current or recent `SnowAmount` data is mapped from data min/max into the visual snow range.
3. At small time steps, mapped snow is added gradually.
4. At larger time steps, combined snow over the skipped interval is averaged and amplified/replaced so fast playback remains readable.
5. `snowManager.snowValue` is updated.

6. If snow is present, `UpdatePrecipToGW(snowValue)` activates the precipitation-to-groundwater particle effect.

The precipitation-to-groundwater effect is controlled by `WaterToGWController`, which maps snow value and time step into particle emission rate, speed, lifetime, and trail lifetime.

Landscape Snow Mapping

Landscape snow is more complex and likely Big Creek-specific.

In local/non-web paths, `LandscapeController.BuildTerrainSplatmapForDay()` uses monthly patch snow values. For each patch:

1. Current and next month snow values are normalized against `SnowAmountMax`.
2. Snow is interpolated by day-of-month.
3. Snow weight is converted to terrain texture weights.
4. Patch extents map patch IDs to alphamap locations.
5. Splatmap weights are written into the terrain alphamap.

The terrain texture layers are used as:

- Unburned/no snow.
- Unburned/snow.
- Burned/snow.
- Burned/no snow.

In web/background-snow paths, terrain data can come from `terraindata/{warmingIdx}` as precomputed splatmap frames. `UpdateTerrain()` interpolates between monthly splatmaps and applies them to the terrain.

Fire and snow share the same splatmap blending. If a patch is burned, the snow/no-snow weight is blended with burned/recovered texture state.

Cube Vegetation Mapping

Vegetation is the most sophisticated mapping system. It does not simply set object scales from data every frame. Instead, it maintains a living 3D population that approximates model carbon values over time.

Data Roles

For one-layer vegetation cubes (Veg1):

- `leafCOver + stemCOver` represents shrub/grass biomass.
- `transOver` drives shrub evapotranspiration particle emission.
- `rootCOver` is available but tree roots are mainly relevant for tree-capable cubes.

For two-layer vegetation cubes (Veg2):

- leafCOver + stemCOver represents overstory tree biomass.
- rootCOver represents overstory roots.
- transOver drives tree evapotranspiration.
- leafCUnder + stemCUnder represents understory shrub/grass biomass.
- transUnder drives shrub evapotranspiration.

For aggregate cubes (Agg):

- Overstory/understory carbon fields are used similarly.
- NetTranspiration is mapped from aggregate transpiration.
- Aggregate-specific carbon/emission factors are used.

Carbon-to-Object Conversion

CubeController.Initialize() calculates average carbon represented by a visual object:

- treeAverageCarbonAmount
- treeAverageRootCarbonAmount
- shrubAverageCarbonAmount
- grassAverageCarbonAmount

These depend on SimulationSettings factors, tree/shrub full-size ranges, full tree height, and root depth. Web builds modify some values to reduce vegetation density.

The scene can then compare:

model carbon at current time
vs.
carbon currently represented by visible objects

Current visualized tree carbon comes from TreeController.GetCarbonAmount(), which uses tree height scale, full tree height, and tree carbon factor. Current visualized shrub carbon comes from shrub renderer height times shrub carbon factor. Root carbon is summed from living fir root state.

Initial Generation

GrowInitialVegetation() converts the current data row into initial object counts.

For Veg1:

shrubsToGrow = round((stemCOver + leafCOver) / shrubAverageCarbonAmount)

For Veg2:

treesToGrow = round((stemCOver + leafCOver) / treeAverageCarbonAmount)
shrubsToGrow = round((stemCUnder + leafCUnder) / shrubAverageCarbonAmount)

It then grows firs, shrubs, and initial grass patches. This path is used when starting a non-web simulation and when a cube is reset/regenerated for a timeline jump.

Playback Update

During playback, `UpdateVegetationBehavior(timeIdx, timeStep)` updates the current data values, then calls `UpdateVegetation()` unless the terrain is currently burning.

`UpdateVegetation()` is a feedback controller:

1. If pending kill counts exist, kill a fir, shrub, or grass patch.
2. Compute target model carbon for the current data row.
3. Compute current visualized carbon in the scene.
4. If visual carbon is too low, grow new objects.
5. If visual carbon is too high, mark objects to die.
6. Continue growth animations for existing roots, shrubs, and grass.

The tolerance is generally half an average object:

```
if visual < target - averageObjectCarbon / 2: grow
if visual > target + averageObjectCarbon / 2: kill
```

Growth is throttled by wait times such as `firGrowthWaitTime` and `shrubGrowthWaitTime` so vegetation does not spawn all at once during normal playback.

Timeline Jump / Regeneration

When a user clicks the timeline, `GameController.SetTimePosition()` changes `timeIdx` and calls `UpdateVegetationFromData()` on active cubes.

`UpdateVegetationFromData()`:

1. Calls `ResetCube()`.
2. Reads the data row for the new `timeIdx`.
3. Calls `GrowInitialVegetation()`.

This is intentionally different from playback. A large time jump should not animate decades of growth/death one event at a time; it reconstructs a plausible vegetation state directly from the destination data.

Tree And Shrub Size Mapping

Tree count is driven primarily by carbon, but individual tree size is also animated. Trees have randomized full height/width scales within `SimulationSettings` ranges. Tree prefabs and root prefabs are selected based on current height/depth, and LOD groups are scaled as growth changes.

Tree evapotranspiration maps to particle emission. `FirController.UpdateSimulation()` receives transpiration and sets emission rate. It also updates growth or death state.

Shrubs are simpler. Their count is driven by understory or one-layer carbon. Their evapotranspiration particle emission is driven directly from `TransUnder`, `TransOver`, or aggregate net transpiration depending on cube type.

Litter Mapping

`cubedata.litter` is stored as `Litter`, but the current visible mapping appears incomplete. `UpdateLitter()` collects objects tagged as litter and shrinks existing litter over time using `SimulationSettings.DeadTreeShrinkFactor`. `GetLitterAmountVisualized()` currently returns `0f`, so this mapping should be treated as partially implemented.

Net Photosynthesis Mapping

`cubedata.netpsn` is stored as `NetPhotosynthesis` and included in data range calculations. The current visible mapping is limited; several UI/statistics lines for net photosynthesis are commented out. Treat `netpsn` as available model data with incomplete or disabled visualization in the current version.

Unvisualized Or Partially Visualized Fields

This section is provisional because the original raw Big Creek source data is not present in this repo or in the configured sibling `../Data` folder on this workstation. It is based on the Unity code, runtime DTOs, and `ScenarioConfig_BigCreek.json`.

Clearly Visualized

These fields currently have direct or substantial visual mappings:

- `cubedata.snow`: cube snow and precipitation-to-groundwater particles.
- `cubedata.depthToGW`: cube groundwater/soil visualization.
- `cubedata.vegAccessWater`: cube surface-soil wetness.
- `cubedata.qout`: cube stream height/face scale.
- `cubedata.transOver`, `cubedata.transUnder`: evapotranspiration particles.
- `cubedata.leafCOver`, `stemCOver`, `rootCOver`, `leafCUnder`, `stemCUnder`, `rootCUnder`: tree/shrub/root population, growth, size, and fire biomass loss.
- `waterdata.qBase`, `qWarm1`, `qWarm2`, `qWarm4`, `qWarm6`: landscape river height/face by warming scenario.
- `waterdata.precipitation`: timeline bar height and message/story context.
- `patchdata.snow`: landscape snow splatmap generation in local/non-web paths.
- `patchdata.spread`, `patchdata.iter`: fire frame generation in legacy/local paths.
- `firedata._dataList`: data-controlled fire grid cells/spread.
- `terraindata._dataList`: precomputed terrain/snow splatmaps when background snow terrain data is enabled.

Partially Or Weakly Visualized

These fields are loaded or ranged but appear to have limited current visual effect:

- `cubedata.netpsn`: stored as `NetPhotosynthesis` and ranged, but UI/statistics visualization is mostly commented out.
- `cubedata.litter`: stored and `UpdateLitter()` shrinks existing litter objects, but `GetLitterAmountVisualized()` returns `0f`, so model litter does not appear to fully drive litter object creation.
- `cubedata.evap`: present in the DTO/column mapping, but this audit did not find an active visual mapping. Evaporation appears folded into labels such as `evapotranspiration` rather than separately visualized.
- `cubedata.soil`: present in DTO/column mapping, but this audit did not find an active visual mapping. `SoilController` has a `soilCarbon` field, but current update calls use water access and groundwater depth.
- `cubedata.heightOver`, `heightUnder`: present in DTO/column mapping. Current vegetation size is primarily derived from carbon factors and generated object geometry rather than directly setting tree/shrub height from these fields.
- `waterdata.index`: used as a record/index value, not a visualization.
- `dates.id`, `dates.date`, `dates.year`, `dates.month`, `dates.day`: drive time, timeline, messages, events, and lighting, but are metadata/control data rather than ecological visual variables.

Configured But Not Runtime-Visualized In Unity

`ScenarioConfig_BigCreek.json` includes source mappings that are useful to the importer/database but not directly visualized by the Unity runtime as standalone variables:

- `basin.wy / waterYear`: importer/database metadata. Unity consumes calendar date and streamflow/precipitation instead.
- `climate.date`, `day`, `month`, `year`: source climate metadata.
- `climate.precip`, `tmax`, `tmin`, `tavg`, `vpd`: configured for import, but this audit did not find direct Unity visualization of climate fields as separate variables. Their effects may be implicit in RHESSys outputs such as snow, streamflow, transpiration, vegetation carbon, and fire spread.

Unknown Until Schema/Data Samples

The following need confirmation from MySQL schema export and old/new source snippets:

- Whether imported tables include additional columns not represented in current Unity DTOs.
- Whether Central Coast v2 adds variables that should be visualized or stored only for provenance/model completeness.
- Whether climate source fields should remain import-only or become user-visible explanatory layers.

- Whether `height_*`, `evap`, `soil`, `netpsn`, and `litter` should be promoted into stronger visual roles or removed from runtime payloads if unused.

Fire Mapping

Fire is covered in detail in [Fire.md](#), but from a data-mapping perspective:

- Fire dates are scheduled scenario events.
- Fire spread/severity after ignition is driven by `firedata` frames.
- Landscape fire data maps to `SERI_FireGrid` cells and burned terrain splatmap state.
- Cube fire uses fire-time vegetation carbon differences to decide how many shrubs/trees to burn.

Fire also modifies later mappings by setting burned terrain state, killed vegetation, and litter/dead tree behavior.

Model/Statistics Overlay Mapping

See [ShowModelDataLayer.md](#) for the feature-level behavior and future improvement notes.

The model display compares data values against visualized values.

`CubeController.UpdateStatistics()` maps:

- Data net transpiration into `netTransSlider`.
- Data stem + leaf carbon into `plantCarbonSlider`.
- Visualized transpiration into `netTransSliderDebug`.
- Visualized tree + shrub carbon into `plantCarbonSliderDebug`.

Several additional sliders for snow, net photosynthesis, water access, and groundwater depth are present or referenced but commented out.

Backward Compatibility Notes

For Central Coast or future scenarios, each new field should declare a visual role:

- Direct geometry mapping.
- Material/color mapping.
- Particle mapping.
- Terrain/splatmap mapping.
- Population feedback mapping.
- UI/statistics only.
- Metadata/no direct visualization.

Adding a new data field should not automatically require a new visual behavior. If a field is not used visually yet, it should be documented as available but unmapped. Big Creek

should continue to provide the existing mapped fields, or a scenario adapter should provide default values.

Data Model

Source: *Specs/DataModel.md*

Last updated: 2026-06-12

Purpose

This spec defines the current Future Mountain data model from the Unity/runtime point of view and outlines how the model should evolve for the Central Coast scenario while preserving Big Creek compatibility. For how these fields drive the 3D scene over time, see [DataMappings.md](#).

This first draft is based on the Unity repo, `ScenarioConfig_BigCreek.json`, runtime DTO classes, and API call sites. It is not yet an authoritative database schema spec. To complete it, we need a MySQL schema export and, ideally, the importer repo or importer output examples.

Goals

- Document the Big Creek data contract as it exists today.
- Identify which fields drive visual behavior in Unity.
- Support an expanded Central Coast data model without breaking Big Creek.
- Establish a compatibility protocol for adding fields, tables, endpoints, and derived values.
- Keep importer, MySQL schema, API DTOs, and Unity DTOs aligned.

Current Data Flow

flowchart LR

```
A["RHESSys and scenario source files"] --> B["Importer / preprocessing"]
B --> C["MySQL database"]
C --> D["Future Mountain API"]
D --> E["Unity WebManager"]
E --> F["Unity DTOs and controllers"]
F --> G["Landscape, cubes, fire, snow, timeline, messages"]
```

The Unity project currently consumes API JSON for web builds and also retains older TextAsset/Resources parsing paths. The API and TextAsset paths do not fully share one centralized schema definition.

Current Tables / API Resources

`ScenarioConfig_BigCreek.json` lists these output tables:

- cubedata

- patchdata
- terraindata
- firedata
- waterdata
- dates

WebManager currently calls these API resources:

- cubedata/{patchIdx}/{warmingIdx}
- waterdata/{index}
- waterdata/total
- firedata/{warmingIdx}
- patchdata
- patchdata/{patchId}
- dates
- dates/{year}/{month}/{day}
- terraindata/{warmingIdx}

Runtime Entities

Scenario

Current scenario config exists as `ScenarioConfig_BigCreek.json`, but Unity does not appear to load it directly at runtime.

Current Big Creek scenario properties:

- Scenario id/name: BigCreek
- Warming levels: 0 C, 1 C, 2 C, 4 C, 6 C
- Fire enabled: true
- Vegetation layers: 2
- Database name: bigcreek_rhessys

Future scenarios should make scenario identity and schema version first-class runtime data.

Recommended fields:

- scenarioId
- displayName
- region
- description
- schemaVersion
- apiBaseUrl
- availableWarmingLevels

- dateRange
- features
- dataContractVersion

Dates

Unity DTO: DateModel

Fields:

- id
- date
- year
- month
- day

Unity uses dates to convert between calendar dates and timeIdx. Many simulation systems depend on stable date indexing, so backward compatibility requires Big Creek date ids to remain stable or a compatibility map to be provided.

Cube Data

Unity DTO: CubeData

Fields:

- id
- dateIdx
- warmingIdx
- patchIdx
- snow
- evap
- netpsn
- depthToGW
- vegAccessWater
- qout
- litter
- soil
- heightOver
- transOver
- heightUnder
- transUnder
- leafCOver
- stemCOver

- rootCOver
- leafCUnder
- stemCUnder
- rootCUnder

Unity visual uses include:

- snow: cube snow surface and precipitation-to-groundwater effect.
- depthToGW: groundwater depth display/soil visualization.
- vegAccessWater: soil water access.
- qout: stream height/outflow.
- litter: litter/burned/ground state.
- netpsn: model/statistics display and ranges.
- transOver, transUnder: evapotranspiration particles/statistics.
- leafC*, stemC*, rootC*: vegetation growth, roots, and fire biomass removal.

Water Data

Unity DTO: WaterData

Fields:

- index
- year
- month
- day
- qBase
- qWarm1
- qWarm2
- qWarm4
- qWarm6
- precipitation

Unity visual uses:

- Landscape river/streamflow.
- Timeline annual precipitation.
- Warming-specific streamflow comparison.

Current concern: the schema encodes warming levels as separate columns (qBase, qWarm1, etc.). Future scenarios may need a normalized form such as (scenarioId, dateIdx, warmingIdx, q, precipitation) so additional climate cases do not require new columns.

Patch Data

Unity DTOs:

- PatchDataRecord
- PatchPointCollection
- PatchPoint
- PatchDataList

Fields include:

- id
- patchID
- _data
- points
- location
- fireLocation
- alphamapLoc
- utm

Patch data links model patches to Unity landscape coordinates, fire grid cells, terrain alphamap positions, and UTM/spatial position.

Fire Data

Unity DTOs:

- FireDataFrameJSONRecord
- TimelineFireData
- Runtime nested fire classes in LandscapeController: FireDataPoint, FireDataPointCollection, FireDataFrame, FireDataFrameRecord

Fields include:

- id
- warmingIdx
- year
- month
- day
- gridHeight
- gridWidth
- _dataList

Important model distinction:

- Fire dates are scheduled scenario events.

- Fire spread/severity after ignition is data-driven.

For Big Creek, scheduled fire dates are currently hard-coded in Unity as July 15, 1969 and November 20, 1988. These should move into scenario data before Central Coast work.

Terrain Data

Unity DTOs:

- TerrainDataFrame
- TerrainDataFrameJSONRecord
- TimelineTerrainData

Fields include:

- id
- warmingIdx
- year
- month
- gridSize
- pixelGrainSize
- decimalPrecision
- _dataList

Unity uses terrain data for precomputed/interpolated splatmaps, including snow and burned/unburned texture states.

For Central Coast v2, preserve this meaning. TerrainData should remain the precomputed Unity/API-facing large-landscape frame, not a raw RHESys CSV import table. The Central Coast raw stratum carbon file is long-format source data:

`month + year + zoneID + patchID + stratumID -> totalc, total_plantc`

It must be transformed after import into precomputed TerrainData frames using the patch map (zoneID footprints), vegetation/carbon values, burn values, and scenario metadata. Unity should consume the precomputed TerrainData shape rather than millions of raw stratum rows.

Timeline Water Data

Unity DTO:

- TimelineWaterData
- PrecipByYear

Fields:

- year

- precipitation

This is a lightweight yearly aggregate for timeline rendering.

Current Schema Risks

- Warming levels are hard-coded as five indices and, in water data, as distinct columns.
- Fire dates are hard-coded in Unity instead of scenario data.
- Some payload fields store nested JSON strings (`_data`, `_dataList`) rather than normalized relational records.
- TextAsset parsing depends on positional column enums.
- Runtime DTOs, database schema, API JSON, and importer mappings are not generated from one source of truth.
- Big Creek-specific dates, terrain assumptions, cube count, and snow assumptions are scattered across code.

Compatibility Protocol

Version Every Contract

Add explicit versions:

- Scenario version.
- Database schema version.
- Importer version.
- API contract version.
- Unity-supported minimum/maximum contract version.

Unity should fail clearly if the API reports an incompatible contract.

Add Fields Without Breaking Big Creek

Rules:

- New API fields must be optional for old scenarios.
- Big Creek responses should continue to include all current fields with the same names and meanings.
- Unity should ignore unknown fields.
- Unity should supply defaults for missing optional fields.
- Renames should be additive first: add the new field, keep the old field, migrate consumers, then deprecate later.

Prefer Named, Normalized Scenario Data

For new scenario data, prefer structures like:

scenarioId
dateIdx
warmingIdx or climateCaseId
patchIdx
variableName
value
unit

or table-specific normalized columns, rather than adding one column per warming level.

Preserve Big Creek Adapter Behavior

If the Central Coast schema changes substantially, create an adapter layer:

- Big Creek adapter maps current tables/API fields to the canonical runtime model.
- Central Coast adapter maps the new schema to the same canonical runtime model.
- Unity controllers consume canonical runtime objects, not raw database-shaped DTOs.

Recommended Canonical Runtime Model

The next refactor should introduce an internal scenario data model separate from API DTOs:

- ScenarioDefinition
- ClimateCase
- DateRecord
- CubeTimeSeries
- LandscapePatchFrame
- WaterFrame
- FireEvent
- FireSpreadFrame
- TerrainSurfaceFrame
- VariableDefinition

VariableDefinition should include:

- id
- displayName
- unit
- sourceColumn
- defaultValue
- visualRole
- minMaxStrategy
- isRequired

Needed Inputs To Complete This Spec

A MySQL Workbench 8 export would be very helpful. Best export format:

- CREATE TABLE statements for all Future Mountain tables.
- Indexes, primary keys, foreign keys, and unique constraints.
- Column types, nullability, defaults, and comments if present.
- A small anonymized/sample row set for each table if possible.
- Stored procedures/views if the API depends on them.

Also helpful:

- API response examples for each endpoint listed above.
- Importer repo or importer README.
- Central Coast proposed column list, including units and whether each column should drive visuals/UI.
- Any existing API/backend models if the API is a separate .NET/Node/Python project.

Open Questions

- Should future climate cases remain numeric warming degrees, or become named scenario cases?
- Are fire event dates part of RHESys metadata, scenario curation, or importer config?
- Should patch spatial data remain JSON blobs or become normalized point rows?
- Should terrain/fire _dataList blobs remain for performance, or be generated/cache artifacts outside the canonical database?
- Does Big Creek need to run indefinitely on the old API, or can it be migrated behind an adapter?

Messages

Source: *Specs/Messages.md*

Last updated: 2026-06-12

User Interface Behavior

Messages are short narrative or explanatory text boxes that appear during simulation when the current date and warming level match configured message records. They can also highlight cube labels when not in Side-by-Side Mode.

There are two message categories:

- General story/model messages.
- Fire messages.

Messages can be shown or hidden through story/message controls. The timeline can also show message icons at years where messages exist.

Technical Behavior

GameController references two TextAsset fields:

- messagesFile
- fireMessagesFile

During `FinishStarting()`, it calls `LoadMessagesFile(file, fire)` for each file and passes the parsed message lists into `UI_MessageManager.Initialize()`.

Message file record format is currently parsed as:

1. Header line with date, warming values, and cube indices.
2. Message text line.
3. Blank/separator line.

Example shape:

```
1969-07-15 0C,6C 2
Message text goes here.
```

The loader parses:

- Date as year-month-day.
- Warming list as values ending in C.
- Cube list as comma-separated integer indices.
- Message text as the next line.

It converts the date to `timeIdx` using `GetTimeIdxForDay()`.

Runtime Display

`UI_MessageManager.UpdateSimulation(...)` checks both general messages and fire messages during each simulation update. A message appears if:

- Its date/time window applies.
- Its warming list includes the current warming degrees.
- It is not already displayed.
- There is an available message box.

The message panel has four message boxes: `MessageBox1` through `MessageBox4`. Text is displayed through `TextMeshProUGUI` child objects.

When not in Side-by-Side Mode, message cube indices can show associated cube labels. Labels are hidden again when the message hides.

Timeline Integration

GameController builds messageYears from loaded messages whose warming degrees apply. TimelineControl displays message icons for those years. Clicking a message icon jumps to that year.

Current Constraints

- Message format is custom plain text and positional.
- Warming matching uses degrees, while some method/field names still say index.
- Timeline click jumps to the message year, not exact message date.
- Cube labels are suppressed in Side-by-Side Mode.
- Future scenarios should move message content into scenario-specific files with documented format validation.

Show Model / Data Layer

Source: *Specs/ModelDataLayer.md*

Last updated: 2026-06-12

Purpose

The Show Model feature is the cube-side visual data layer: the bar-style graphs that appear next to cubes. Its purpose is to let users compare the current RHESSys model values with the living 3D visualization, so the experience is not only scenic or illustrative but also connected back to the underlying data.

This feature is related to, but separate from, the Timeline. The Timeline is also graph-like: it renders annual precipitation as bars and overlays fire/message markers. Show Model is a cube-level data display for the current simulation moment.

User Interface Behavior

The user enables the layer with the ShowModelDataToggle UI control.

In Normal Mode, the layer appears for:

- The aggregate cube.
- Each active detailed terrain cube.

In Side-by-Side Mode, the layer appears in the left and right side-by-side statistics areas so the selected cube and comparison cube can be evaluated against different warming scenarios.

The current UI is best understood as a set of bars or slider-like readouts, not as a full graphing system. It shows current-state values. It does not show a full time-series line chart, historical trace, uncertainty range, or selectable axis.

Current Displayed Variables

The active current-version display focuses on:

- Net transpiration.
- Plant carbon, calculated as stem carbon plus leaf carbon.

`CubeController.UpdateStatistics()` maps current data values into UI sliders:

- `netTransSlider`: model net transpiration.
- `plantCarbonSlider`: model stem carbon plus leaf carbon.

The same method also computes visualized comparison values:

- `netTransSliderDebug`: transpiration currently represented by visible tree and shrub particle emission.
- `plantCarbonSliderDebug`: plant carbon currently represented by visible trees and shrubs.

Those debug/comparison sliders are initialized inactive in `SetupStatisticsPanel()`, so the current public UI may not expose them as a polished user-facing comparison. They are still important technically because they show the intended comparison model: data value vs. current 3D representation.

Technical Flow

`GameController.UpdateModelDisplayFromToggle()` reads the Show Model toggle, sets `displayModel`, and calls either `ShowStatistics()` or `HideStatistics()`.

`GameController.ShowStatistics()` enables the proper statistics UI for the current mode:

- Normal Mode: aggregate cube plus active cubes.
- Side-by-Side Mode: `cubeSBSModeStatsLeft` and `cubeSBSModeStatsRight`.

`GameController.HideStatistics()` disables the same display objects.

During simulation playback, `GameController.RunGame()` updates the statistics when either of these is true:

- `displayModel` is enabled.
- `sideBySideMode` is active.

Each cube then runs `CubeController.UpdateStatistics()`, which:

1. Reads the current model values for the cube's current `timeIdx`.
2. Chooses the correct overstory, understory, or aggregate fields based on cube data type.
3. Maps values from scenario min/max data ranges into slider min/max UI ranges.

4. Optionally computes the current visualized equivalents from objects already in the scene.

Data-to-Bar Mapping

Net transpiration uses:

- `transOver` for one-layer vegetation cubes.
- `transOver` + `transUnder` for two-layer vegetation cubes.
- `netTranspiration` for aggregate cubes.

Plant carbon uses:

- `leafCOver` + `stemCOver` for one-layer vegetation cubes.
- `leafCOver` + `stemCOver` + `leafCUnder` + `stemCUnder` for two-layer vegetation cubes.

The visualized comparison values are calculated from the scene:

- Visualized transpiration sums tree transpiration effects and shrub particle emission rates.
- Visualized plant carbon sums tree carbon and shrub carbon inferred from current object sizes.

This means the layer can reveal when the 3D representation is still catching up to the model data during animated playback.

Timeline As Precipitation Graph

The Timeline is a separate graphing feature. `TimelineControl.CreateTimeline()` and `CreateTimelineWeb()` instantiate one bar per year and map precipitation to bar height.

For local/non-web data, the maximum precipitation is calculated from loaded `WaterDataYear` values. For web data, the maximum precipitation is calculated from the returned `PrecipByYear[ ]` values from the active scenario's `waterdata/total` endpoint.

Timeline bars are not a general Show Model graph. They are specifically annual precipitation bars plus event markers for fires and messages.

Currently Disabled Or Latent Data Layer Variables

The code contains commented or inactive references for additional statistics:

- Snow amount.
- Net photosynthesis.
- Water access.
- Groundwater depth.

These variables should be treated as candidates for future data-layer work rather than confirmed active user-facing graphs in the current version.

Current Limitations

- The display is bar-based, not an actual line graph or multi-year chart.
- The visible UI does not clearly expose all computed data-vs-visual comparison values.
- Units, legends, and labels are limited.
- The graph ranges are scenario-dependent and rely on min/max values calculated from loaded data.
- The layer compares current values only; it does not preserve a visible history of how a cube changed over time.
- Some potentially useful data fields are loaded but not actively graphed.

Future Improvement Ideas

Future versions could turn this into a more complete visual analytics layer:

- Replace or augment bars with actual graphs.
- Show model value and visualized value together as paired traces.
- Add variable selectors for water, snow, vegetation, fire, and soil metrics.
- Add units, legends, and short labels that match RHESSys/source data names.
- Support time-window graphs for the selected cube instead of current-frame values only.
- Share graph infrastructure with the Timeline so precipitation, fire markers, and cube variables can be compared more consistently.
- Allow scenario-specific graph definitions so Big Creek remains compatible while Central Coast can add new fields.

Backward Compatibility Notes

For Big Creek, keep the current Show Model defaults working even if the graph system is refactored.

For Central Coast and later scenarios, each graphable variable should declare:

- Data field name.
- Display label.
- Units.
- Min/max or normalization rule.
- Whether it is compared against a visualized scene-derived value.
- Whether missing data should hide the graph, show a disabled state, or use a default.

This will let new data columns be added without breaking the existing Big Creek visualization.

Future Mountain API

Future Mountain API

Source: Docs/Services/FutureMountainApi.md

Last updated: 2026-06-16

The API project is stored at `Services/FutureMountainApi/`. It was moved from the former standalone `FutureMountainAPI` repository with `git subtree` to preserve project history.

The imported history was sanitized before import:

- `appsettings.json` and `appsettings.Development.json` entries containing database credentials were removed from historical commits.
- Hardcoded connection-string credentials in historical source files were redacted.
- The current API startup code reads named connection strings from configuration instead of using checked-in connection strings.

Use a local configuration source for the connection string, such as `.NET` user secrets, environment variables, or an ignored `appsettings.Development.json`.

Connection Strings

The API serves legacy Big Creek data and Central Coast prototype data from separate databases.

```
{
  "ConnectionStrings": {
    "BigCreekDbContext": "<legacy Big Creek database>",
    "CentralCoastDbContext": "<Central Coast database>"
  }
}
```

`BigCreekDbContext` is the connection-string key used by the existing legacy contexts (`CubeDataDbContext`, `WaterDataDbContext`, `FireDataDbContext`, `TerrainDataDbContext`, `PatchDataDbContext`, and `DateDbContext`). The C# context class names are intentionally unchanged.

`CentralCoastDbContext` is used by the new Central Coast API context and points at the separate Central Coast schema/database produced by the importer.

Routes

Legacy Big Creek routes remain unchanged:

```
/api/CubeData/...  
/api/WaterData/...  
/api/PatchData/...  
/api/TerrainData/...  
/api/Dates/...
```

Central Coast prototype routes are scenario-explicit under the same `/api` prefix:

```
/api/centralcoast/CubeData/...  
/api/centralcoast/WaterData/...  
/api/centralcoast/PatchData/...  
/api/centralcoast/TerrainData/...  
/api/centralcoast/Dates/...
```

A future cleanup may add `/api/bigcreek/...` aliases while keeping the original Big Creek routes for backwards compatibility.

Central Coast Prototype DTOs

Central Coast database rows preserve the Central Coast source structure, which does not exactly match the Unity runtime JSON contract used by Big Creek. The prototype API therefore shapes Central Coast rows into Unity-friendly responses at the route boundary.

Explicit prototype DTO classes live in:

```
Services/FutureMountainApi/FutureMountainAPI/FutureMountainAPI/Models/Central  
Coast/CentralCoastPrototypeDtos.cs
```

Current explicit DTOs:

- `CentralCoastCubeDataPrototypeDto`
- `CentralCoastPatchDataPrototypeDto`

Examples of prototype mappings:

- Central Coast `zoneID` is exposed as legacy-style `patchIdx/patchID`.
- Central Coast `litterc` and `soilc` are exposed as `litter` and `soil`.
- Central Coast `Qout` is exposed as `qout`.
- Central Coast `overstory` and `understory` `netpsn` values are summed into legacy-style `netpsn`.
- Central Coast `canopy` and `ground evaporation` are summed into legacy-style `evap`.

`WaterData` and `TerrainData` currently reuse existing API response models (`WaterDataFrame` and `TerrainDataFrameJSONRecord`) with Central Coast query projections. Richer Central Coast v2 production endpoints can later expose native fields such as `scenarioRunId`, `zoneID`, `patchID`, `basinID`, `hillID`, and `stratum` identifiers without changing these prototype contracts.

Local Build And Publish

Project folder:

Services/FutureMountainApi/FutureMountainAPI/FutureMountainAPI/

Build:

```
dotnet build
```

Publish to a local folder:

```
dotnet publish -c Release -o C:\tmp\FutureMountainApiPublish
```

Before copying the publish output to the server, confirm the published configuration contains valid production connection strings:

```
{
  "ConnectionStrings": {
    "BigCreekDbContext": "<production Big Creek database>",
    "CentralCoastDbContext": "<production Central Coast database>"
  }
}
```

The required keys are `BigCreekDbContext` and `CentralCoastDbContext`.

`BigCreekDbContext` is required at startup. `CentralCoastDbContext` is optional in code, but Central Coast routes require it to be configured.

Do not commit production credentials. For deployment, use the server-side configuration mechanism already approved for the host, or update the `appsettings.json` in the local publish folder before copying it to IIS.

IIS Deployment Runbook

The deployed API is served by IIS for:

<https://data.futuremtn.org/api/>

Manual publish workflow:

1. Publish the API locally with `dotnet publish -c Release`.
2. Confirm the publish folder contains the expected build output and deployment `appsettings.json`.
3. Log in to `fm01.grit.ucsb.edu` over Remote Desktop.
4. Open IIS Manager.
5. Stop the IIS website or application that serves `data.futuremtn.org`.
6. Back up the current deployed API folder.
7. Copy all files from the local publish folder into the server folder for `data.futuremtn.org`, replacing the old application files.

8. Confirm `appsettings.json` in the deployed folder has the production connection strings.
9. Start the IIS website or application.
10. Smoke test the API endpoints below.

If deployment fails, restore the backed-up server folder and restart the IIS site/application.

Smoke-test URLs:

```
https://data.futuremtn.org/api/dates
https://data.futuremtn.org/api/waterdata/total
https://data.futuremtn.org/api/cubedata/-1/0
https://data.futuremtn.org/api/centralcoast/dates
https://data.futuremtn.org/api/centralcoast/waterdata/total
```

Use the Unity WebGL build only after the API smoke tests succeed.

Postman Collection

A Postman collection for manual API testing is included at:

`Services/FutureMountainApi/Future Mountain.postman_collection.json`

Import this collection into Postman to run the common Big Creek and Central Coast API requests against either a local API instance or the deployed `data.futuremtn.org` API. Keep exported Postman environments free of passwords, connection strings, cookies, and tokens before committing them.

Documentation

Additional API specs live in:

`Specs/FutureMountainAPI/`

Future Mountain API Overview

Source: `Specs/FutureMountainAPI/Overview.md`

Last updated: 2026-06-16

Purpose

This spec documents the current Future Mountain ASP.NET Core API. The API serves Unity runtime data for the deployed Big Creek experience and the Central Coast v2 prototype.

Project path:

`Services/FutureMountainApi/FutureMountainAPI/FutureMountainAPI/`

The API targets `.NET 8` and uses ASP.NET Core controllers with EF Core and Pomelo/MySQL.

Runtime Configuration

The API reads named connection strings from configuration:

Key	Purpose
BigCreekDbContext	Legacy Big Creek tables and required startup connection
CentralCoastDbContext	Central Coast v2 prototype schema/database

Legacy Big Creek DbContext class names remain unchanged:

- CubeDataDbContext
- WaterDataDbContext
- FireDataDbContext
- TerrainDataDbContext
- PatchDataDbContext
- DateDbContext

Central Coast routes use CentralCoastDbContext.

Route Families

Big Creek routes keep the legacy `/api/<Controller>` pattern:

```
/api/CubeData/...  
/api/WaterData/...  
/api/FireData/...  
/api/PatchData/...  
/api/TerrainData/...  
/api/Dates/...
```

Central Coast routes are scenario-explicit:

```
/api/centralcoast/CubeData/...  
/api/centralcoast/WaterData/...  
/api/centralcoast/PatchData/...  
/api/centralcoast/TerrainData/...  
/api/centralcoast/Dates/...
```

Unity applies this route selection through `SimulationSettings.apiProfile` and `WebManager`.

Manual Testing

A Postman collection for API smoke testing is stored at:

`Services/FutureMountainApi/Future Mountain.postman_collection.json`

The collection is intended for quick local and deployed API checks. Any Postman environment files committed with it should contain only non-secret example values.

Current Constraints

- There is no explicit `/api/bigcreek/...` route family yet.
- Central Coast responses are prototype Unity-compatible DTOs, not native full Central Coast scientific records.
- FireData routes exist for Big Creek only. Central Coast fire spread playback data has not been provided yet.
- Swagger is enabled only in development environment.
- CORS currently allows any origin.

Related Docs

- `Docs/Services/FutureMountainApi.md`
- `Specs/FutureMountainAPI/Routes.md`
- `Specs/FutureMountainAPI/Deployment.md`
- `Docs/AddingFutureScenarios.md`
- `Docs/RHESSysDataImporter/ScenarioUpgradeGuide.md`

Future Mountain API Configuration

Source: *Specs/FutureMountainAPI/Configuration.md*

Last updated: 2026-06-16

Connection Strings

The API uses ASP.NET Core configuration and named connection strings.

Required keys:

Key	Used by
BigCreekDbContext	Legacy Big Creek controllers and startup server-version detection
CentralCoastDbContext	Central Coast prototype controllers

BigCreekDbContext must be non-empty or startup throws. CentralCoastDbContext is registered only when configured, but Central Coast routes require it.

Hosting

Program.cs currently configures Kestrel with:

`http://*:13198`

IIS fronts the public `https://data.futuremntn.org/api/` site/application.

CORS

The current CORS policy allows:

- any origin
- any header
- any method

This supports WebGL/API access but should be revisited if the API becomes publicly broader than the Future Mountain runtime.

Swagger

Swagger services are registered, but Swagger UI is enabled only when the app environment is development.

Current Cleanup Notes

- Some old SQL Server package references and comments remain.
- WeatherForecastController remains from the ASP.NET template and is not part of the Unity runtime API.
- Production credentials must not be committed to source control.

Future Mountain API Routes

Source: *Specs/FutureMountainAPI/Routes.md*

Last updated: 2026-06-16

Manual request examples are also available in the Postman collection:

Services/FutureMountainApi/Future Mountain.postman_collection.json

Big Creek Routes

CubeData

Base route:

/api/CubeData

Routes:

Route	Purpose
GET /api/CubeData	All cube rows
GET /api/CubeData/{id}	One row by database id
GET /api/CubeData/{patchIdx}/{warmingIdx}	Cube rows for patch/cube and warming
GET	Cube rows for one date index

Route	Purpose
<code>/api/CubeData/{patchIdx}/{warmingIdx}/{dateIdx}</code>	
GET	
<code>/api/CubeData/{patchIdx}/{warmingIdx}/{dateIdxStart}/{dateIdxEnd}</code>	Cube rows in a date-index range

WaterData

Base route:

`/api/WaterData`

Routes:

Route	Purpose
GET <code>/api/WaterData</code>	All water rows
GET <code>/api/WaterData/{index}</code>	One water frame by index
GET <code>/api/WaterData/{startIdx}/{endIdx}</code>	Water frames in an index range
GET <code>/api/WaterData/max/{warmingIdx}</code>	Maximum streamflow for warming index
GET <code>/api/WaterData/min/{warmingIdx}</code>	Minimum precipitation for warming index
GET <code>/api/WaterData/total/{year}</code>	Total precipitation for one year
GET <code>/api/WaterData/total</code>	Annual precipitation totals for timeline
GET <code>/api/WaterData/maxtotal</code>	Maximum annual precipitation total

FireData

Base route:

`/api/FireData`

Routes:

Route	Purpose
GET <code>/api/FireData</code>	All fire frames
GET <code>/api/FireData/{warmingIdx}</code>	Fire frames for warming index

PatchData

Base route:

`/api/PatchData`

Routes:

Route	Purpose
-------	---------

Route	Purpose
GET /api/PatchData	All patch records
GET /api/PatchData/{patchId}	Patch records for one patch id

TerrainData

Base route:

/api/TerrainData

Routes:

Route	Purpose
GET /api/TerrainData	All terrain frames
GET /api/TerrainData/{warmingIdx}	Terrain frames for warming index
GET /api/TerrainData/{startIdx}/{endIdx}	Terrain frames in id/index range

Dates

Base route:

/api/Dates

Routes:

Route	Purpose
GET /api/Dates	All date rows
GET /api/Dates/{id}	Date row by id/date index
GET /api/Dates/{year}/{month}/{day}	Date row matching calendar date

Central Coast Routes

Central Coast prototype routes mirror the Unity-facing Big Creek route shapes where practical.

Central Coast CubeData

Base route:

/api/centralcoast/CubeData

Routes:

Route	Purpose
GET /api/centralcoast/CubeData	All Central Coast cube rows shaped as prototype DTOs
GET /api/centralcoast/CubeData/{id}	One row by database id

Route	Purpose
GET /api/centralcoast/CubeData/{patchIdx} /{warmingIdx}	Rows for zone/patch identity and warming
GET /api/centralcoast/CubeData/{patchIdx} /{warmingIdx}/{dateIdx}	Rows for one date index
GET /api/centralcoast/CubeData/{patchIdx} /{warmingIdx}/{dateIdxStart}/{dateIdx End}	Rows in a date-index range

Central Coast WaterData

Base route:

/api/centralcoast/WaterData

Routes:

Route	Purpose
GET /api/centralcoast/WaterData	All water frames
GET /api/centralcoast/WaterData/{index}	One water frame by index
GET /api/centralcoast/WaterData/total	Annual precipitation totals for timeline
GET /api/centralcoast/WaterData/maxtotal	Maximum annual precipitation total

Central Coast PatchData

Base route:

/api/centralcoast/PatchData

Routes:

Route	Purpose
GET /api/centralcoast/PatchData	All patch-map footprint rows shaped as prototype DTOs
GET /api/centralcoast/PatchData/{patchId}	Patch-map footprint rows for one zone/patch id

Central Coast TerrainData

Base route:

/api/centralcoast/TerrainData

Routes:

Route	Purpose
GET /api/centralcoast/TerrainData	All Central Coast terrain frames
GET /api/centralcoast/TerrainData/{warmingIdx}	Terrain frames for warming index

Central Coast Dates

Base route:

/api/centralcoast/Dates

Routes:

Route	Purpose
GET /api/centralcoast/Dates	All date rows
GET /api/centralcoast/Dates/{id}	Date row by id/date index
GET /api/centralcoast/Dates/{year}/{month}/{day}	Date row matching calendar date

Development Route

The default ASP.NET template WeatherForecast controller still exists:

GET /WeatherForecast

It is not part of the Unity runtime contract.

Future Mountain API Deployment

Source: Specs/FutureMountainAPI/Deployment.md

Last updated: 2026-06-16

Purpose

This spec records the current manual deployment workflow for the Future Mountain API on the data.futuremtn.org IIS site/application.

The operational runbook is also summarized in:

Docs/Services/FutureMountainApi.md

Build Output

Publish from:

Services/FutureMountainApi/FutureMountainAPI/FutureMountainAPI/

Command:

```
dotnet publish -c Release -o C:\tmp\FutureMountainApiPublish
```

The publish folder should contain the compiled API, dependencies, web.config, and deployment configuration.

Required Configuration

The deployed app must have:

```
{
  "ConnectionStrings": {
    "BigCreekDbContext": "<production Big Creek database>",
    "CentralCoastDbContext": "<production Central Coast database>"
  }
}
```

BigCreekDbContext is required during startup. CentralCoastDbContext is required for Central Coast routes.

Do not commit production credentials. Put production connection strings into the publish folder or server-side configuration using the approved deployment practice for the host.

IIS Copy Workflow

Manual deployment steps:

1. Publish locally.
2. Confirm production connection strings are present in the publish output or server-side configuration.
3. Remote Desktop to fm01.grit.ucsb.edu.
4. Open IIS Manager.
5. Stop the data.futuremtn.org website/application.
6. Back up the current deployed folder.
7. Copy all files from the publish folder into the deployed data.futuremtn.org folder.
8. Confirm deployed appsettings.json or server config still has production connection strings.
9. Start the website/application.
10. Smoke test Big Creek and Central Coast endpoints.

Smoke Tests

Minimum endpoint checks:

```
https://data.futuremtn.org/api/dates
https://data.futuremtn.org/api/waterdata/total
https://data.futuremtn.org/api/cubedata/-1/0
```

`https://data.futuremtn.org/api/centralcoast/dates`
`https://data.futuremtn.org/api/centralcoast/waterdata/total`

After endpoint checks pass, run a Unity/WebGL smoke test.

Rollback

If the new deployment fails:

1. Stop the IIS website/application.
2. Restore the backed-up deployed folder.
3. Confirm configuration is still present.
4. Start the website/application.
5. Re-run the smoke-test endpoints.

RHESSys Data Importer

RHESSys Data Importer Overview

Source: Docs/RHESSysDataImporter/Overview.md

Last updated: 2026-06-16

Purpose

The RHESSys Data Importer is a standalone .NET console utility embedded in the Future Mountain repository under:

`RHESSys_Data_Importer/`

Its current purpose is to import RHESSys-derived Big Creek v1 and Central Coast v2 data into MySQL databases used by the Future Mountain API and Unity runtime.

This documentation describes the current checked-in importer behavior only. It does not define a future generalized importer model.

Project Location

Solution:

`RHESSys_Data_Importer/RHESSys_Data_Importer.sln`

Opening the solution in Visual Studio requires Visual Studio 2022 or newer. The importer targets .NET 8, and older Visual Studio versions may not load the solution correctly.

Project:

`RHESSys_Data_Importer/RHESSys_Data_Importer/RHESSys_Data_Importer.csproj`

Main entry point:

RHESSys_Data_Importer/RHESSys_Data_Importer/Program.cs

Runtime Type

The importer is a .NET 8 console application.

The project file currently targets:

`<TargetFramework>net8.0</TargetFramework>`

The project uses Entity Framework Core with Pomelo/MySQL for database writes. Older SQL Server references remain in comments and package references, but the active DbContext configuration uses MySQL.

High-Level Flow

1. Program.cs starts the importer.
2. The importer looks for ScenarioConfig_BigCreek.json in the current working directory.
3. If the scenario config exists, it is loaded through ScenarioConfigLoader.
4. The database connection string is built from the scenario config and passed to ConnectionHelper.
5. FileDiscovery searches configured input folders for known data categories.
6. In interactive mode, WizardRunner prompts the user for config, database, data type, and confirmation choices.
7. In auto mode, command-line flags select the data categories to import.
8. Big Creek v1 paths primarily use TextFileInput and RHESSysDAL.
9. Central Coast v2 paths use CentralCoastImporter and CentralCoastDAL.

Main Code Areas

Area	Purpose
Program.cs	Startup, config loading, flag parsing, interactive/auto mode dispatch
Configuration/ IO/FileDiscovery.cs	Scenario config DTOs and JSON loading File discovery from configured folders and patterns
Wizard/WizardRunner.cs	Interactive console wizard
Parsing/ColumnMapper.cs	Header-to-model column mapping for config-aware cube imports
TextFileInput.cs	Parsing and import orchestration for cube, dates, water, fire, patch, and terrain data
DAL/	Connection helper, EF Core DbContexts,

Area	Purpose
Models/	and write methods
Migrations/	Database/import model classes
data/	EF migration artifacts for some contexts
	Embedded sample/source data currently present in the importer tree

Current Import Categories

The current config/discovery model recognizes these categories:

- dates
- cube
- patch
- terrain
- fire
- burn
- water
- climate
- stratum

Big Creek v1 still has cube as the most complete config-driven wizard path. Central Coast v2 has implemented paths for dates, cube patch/stratum updates, water, burn, patch-map, stratum carbon, and terrain generation. Climate remains a recognized placeholder.

Current Database Targets

The Big Creek v1 scenario config lists these output tables:

- cubedata
- patchdata
- terraindata
- firedata
- waterdata
- dates

Each major table currently has a dedicated EF Core DbContext.

Central Coast v2 uses a dedicated schema/database and additional tables such as BurnData, StratumData, and ImportRun.

Current Operation Modes

Interactive Mode

Interactive mode is the default when `--auto` is not provided.

If `ScenarioConfig_BigCreek.json` is found, the importer opens the wizard. The wizard can load another config, simulate database creation, discover files, preview cube mappings, and run selected imports.

For Big Creek v1, only cube import is currently implemented in wizard mode. Other categories are shown in the wizard but report that their import is not yet implemented there.

For Central Coast v2, wizard mode supports dates, cube, water, burn, patch, stratum, and terrain. Fire-frame import is exposed only as a placeholder until a source file is configured.

Auto Mode

Auto mode is selected with:

`--auto`

Supported flags include:

- `--dryrun`
- `--force`
- `--config <path>`
- `--dates`
- `--cubes`
- `--patch`
- `--terrain`
- `--fire`
- `--burn`
- `--water`
- `--climate`
- `--stratum`

If no category flag is provided in auto mode, the importer enables cube, patch, terrain, fire, water, dates, and stratum. Burn is also enabled for Central Coast v2. Climate remains a placeholder.

Legacy Fallback Mode

If the default scenario config is not found, or if config-driven cube discovery finds no cube files, the importer can fall back to older hard-coded folder paths and positional parsing behavior.

Those paths are currently local workstation paths in `Program.cs`.

Current Known Boundaries

- The Unity runtime does not load this importer configuration directly.
- The importer is not a Unity project.
- The importer does not currently provide a stable API.
- The importer is not currently a universal RHESSys importer.
- The current docs and specs describe the embedded current state, not a future scenario migration.

Related Documentation

- [Building And Running](#)
- [Scenario Config](#)
- [Data Sources](#)
- [Scenario Upgrade Guide](#)
- [Import Pipeline Spec](#)
- [Scenario Config Schema Spec](#)
- [File Naming And Discovery Spec](#)
- [Database Write Contract Spec](#)
- [Roadmap](#)

RHESSys Data Importer Building and Running

Source: *Docs/RHESSysDataImporter/BuildingAndRunning.md*

Last updated: 2026-06-13

Purpose

This document describes how to build and run the current RHESSys Data Importer embedded in Future Mountain.

It is intended for developers or technical staff importing current Big Creek-style RHESSys data into a local or staging MySQL database.

Prerequisites

Required:

- .NET SDK compatible with net8.0.
- Visual Studio 2022 or newer if opening the .sln in Visual Studio. Older Visual Studio versions may not load the .NET 8 importer solution correctly.
- MySQL server reachable from the workstation.
- Source RHESSys data files in folders matching the active scenario config.
- A database user with permission to write the configured database.

Useful:

- MySQL Workbench or another MySQL inspection tool.
- A staging database for testing imports before production updates.

Build

From the importer solution folder:

```
cd RHESSys_Data_Importer
dotnet build RHESSys_Data_Importer.sln
```

The project currently builds successfully, but with nullable and cleanup warnings.

Configuration Files

Default scenario config:

RHESSys_Data_Importer/RHESSys_Data_Importer/ScenarioConfig_BigCreek.json

Optional local connection fallback:

RHESSys_Data_Importer/RHESSys_Data_Importer/appsettings.Development.json

When the scenario config exists, it takes precedence because Program.cs calls ConnectionHelper.SetOverride(config.Database.GetConnectionString()).

Working Directory

Run commands from the project folder that contains ScenarioConfig_BigCreek.json:

```
cd RHESSys_Data_Importer\RHESSys_Data_Importer
dotnet run
```

The current code checks for the default config using a relative path:

ScenarioConfig_BigCreek.json

Running from another working directory may cause config discovery to fail and trigger legacy fallback behavior.

Interactive Import

Default interactive run:

```
dotnet run
```

Expected flow:

1. The app prints the importer title.
2. It loads ScenarioConfig_BigCreek.json if present.
3. It prints the configured scenario and database.

4. It discovers matching files and prints category counts.
5. It opens the wizard.
6. The user confirms the scenario or loads another config.
7. The user chooses whether to use an existing database or simulate creation of a new database.
8. The user selects data categories.
9. The user can preview cube column mappings.
10. The user confirms import.

Current wizard limitations (Big Creek v1):

Only cube import is implemented in wizard mode for the Big Creek v1 profile. Patch, terrain, fire, water, and climate are selectable but report that the import is not yet implemented.

For the Central Coast v2 profile, the following categories are fully implemented in wizard mode:

Category	Wizard	Auto
Cube (patch + stratum)	Implemented	--cubes
Water	Implemented	--water
Burn (basin + patch monthly burn)	Implemented	--burn
Fire (Unity spread frames)	Placeholder until a fire-frame source is configured	--fire
Stratum carbon	Implemented	--stratum

Central Coast v2 Config

Central Coast v2 is selected through the interactive wizard. Run from the importer project folder:

```
cd RHESSys_Data_Importer\RHESSys_Data_Importer
dotnet run
```

The app loads ScenarioConfig_BigCreek.json first because Big Creek remains the default profile. To use Central Coast v2, choose:

[2] Load another config (enter path)

Then enter:

ScenarioConfig_CentralCoastV2.json

For a validation-only run through the same wizard path:

```
cd RHESSys_Data_Importer\RHESSys_Data_Importer
dotnet run -- --dryrun
```

Then choose option 2 and load `ScenarioConfig_CentralCoastV2.json`. The wizard prints the resolved profile (CentralCoastV2), database target (centralcoast_rhessys), discovery summary, and Central Coast validation report before any import writes.

Headless (auto mode) with an explicit config:

```
dotnet run -- --auto --config ScenarioConfig_CentralCoastV2.json
```

Dry run:

```
dotnet run -- --auto --config ScenarioConfig_CentralCoastV2.json --dryrun
```

Auto Import

Run all currently enabled auto categories:

```
dotnet run -- --auto
```

Dry run:

```
dotnet run -- --auto --dryrun
```

Cube-only dry run:

```
dotnet run -- --auto --dryrun --cubes
```

Cube-only import:

```
dotnet run -- --auto --cubes
```

Central Coast v2 Auto Dry Run

From the importer project folder:

Full dry run (all CCV2 categories, no DB writes):

```
dotnet run -- --auto --config ScenarioConfig_CentralCoastV2.json --dryrun
```

Expected console output (approximate row counts):

Category	File(s)	Expected rows
Water	cube_agg_p.csv	11,688
Cube patch	cube_p_patch1.csv + patch2.csv	116,880
Cube stratum	over/under patch1/2	233,760 updates
Burn basin	bm.csv	384
Burn patch	spatial_data_point_patchvar.csv	3,438,336
Stratum carbon	spatial_data_point_stratvar.csv	6,876,672

Category-scoped dry runs:

```
dotnet run -- --auto --config ScenarioConfig_CentralCoastV2.json --dryrun --water
dotnet run -- --auto --config ScenarioConfig_CentralCoastV2.json --dryrun --cubes
dotnet run -- --auto --config ScenarioConfig_CentralCoastV2.json --dryrun --burn
dotnet run -- --auto --config ScenarioConfig_CentralCoastV2.json --dryrun --stratum
```

Auto mode flags:

Flag	Current behavior
--auto	Enables non-interactive mode
--dryrun	Prints intended actions without DB writes in supported paths
--force	Skips validation failure abort; prints that force is enabled
--cubes	Enables cube import (CCV2: patch + stratum rows)
--patch	Enables legacy patch import
--terrain	Enables legacy terrain import
--burn	Enables Central Coast v2 monthly burn import (BurnData: basin + patch burn)
--fire	Enables configured fire-frame import. The current CCV2 sample has no fire-frame source file, so this is a clean no-op for that scenario.
--water	Enables water import (CCV2: daily aggregate)
--stratum	Enables stratum carbon import (CCV2 only)
--config <path>	Loads the specified scenario config instead of the default ScenarioConfig_BigCreek.json
--climate	Recognized as a flag, but climate import is not implemented

Database Behavior

The importer writes through EF Core DbContexts. Central Coast v2 burn, stratum, patch, and terrain data use batch inserts (chunk-based AddRange + single SaveChanges per chunk) to handle the large row counts efficiently.

Before importing:

1. Confirm the target database name.
2. Confirm that credentials in the scenario config point to staging or the intended target.
3. Confirm that existing data can be replaced or appended safely.
4. Back up any database that matters.

The current database creation helper is simulated. Choosing “Create new database” in the wizard logs what it would create but does not actually create a MySQL database.

File Discovery Check

On startup, the importer prints a discovery summary:

```
cube: N files
patch: N files
terrain: N files
fire: N files
water: N files
climate: N files
```

Warnings are printed when:

- An input folder does not exist.
- A category has no configured file pattern.
- No files match a category pattern.
- A directory search fails.

Treat these warnings as important. A successful process run can still import little or no data if discovery paths are wrong.

Recommended Current Workflow

1. Build the importer.
2. Verify the scenario config database points to staging.
3. Run a dry run with the intended category flags.
4. Review discovery counts and warnings.
5. Confirm source files and target tables.
6. Back up the target database if it already contains useful data.
7. Run the import.
8. Inspect row counts in MySQL.
9. Smoke test the API endpoints that Unity consumes.
10. Smoke test the Unity runtime against the imported database/API.

Troubleshooting

The importer says no config was found

Run from:

RHESSys_Data_Importer/RHESSys_Data_Importer/

or pass through the wizard and load a config manually.

File counts are zero

Check:

- inputFolders in the scenario config.
- Whether paths are relative to the current working directory.
- File patterns under filePatterns.
- Whether source files are in the top level of each folder.

FileDiscovery currently searches only the top level of each input folder.

Cube import uses legacy parsing

If the source header does not match configured columnMapping.cube keys, ColumnMapper has no matches. The importer logs an error and falls back to positional legacy parsing.

Wizard create-database option does not create a database

This is current behavior. DatabaseHelper.CreateNewDatabase is a placeholder that logs a simulated action.

Build succeeds with warnings

The current code has nullable warnings, unused variables, and legacy comments. These do not currently prevent build output.

Current Safety Notes

- Do not assume the importer is idempotent.
- Do not point the importer at production before validating on staging.
- Do not rely on --force as a real safety or overwrite mechanism.
- Do not assume all wizard categories are implemented.
- Do not commit local credentials.

RHESSys Data Importer Data Sources

Source: Docs/RHESSysDataImporter/DataSources.md

Last updated: 2026-06-13

Purpose

This document describes the current data files and source-data assumptions used by the embedded RHESSys Data Importer.

It covers current state only.

Current Embedded Data

The importer tree currently includes source/sample files under one shared data root:

RHESSys_Data_Importer/Data/

Current subfolders:

Data/BigCreek/aggregate/

Data/BigCreek/fire_cubes/

Data/CentralCoast/RHESSysOutput-SingleWarmIdx-6-4-2026/

The embedded files total roughly 120 MB.

Aggregate Files

Current aggregate examples:

agg_hist_fire.txt
agg_warm1_fire.txt
agg_warm2_fire.txt
agg_warm4_fire.txt
agg_warm6_fire.txt

Legacy aggregate cube parsing treats these as patch index -1.

Cube Files

Current cube examples:

p1239496_2veg_hist_fire.txt
p1239496_2veg_warm1_fire.txt
p1239496_2veg_warm2_fire.txt
p1239496_2veg_warm4_fire.txt
p1239496_2veg_warm6_fire.txt

The legacy parser infers:

- Patch id from the leading p... filename segment.
- Baseline warming index from filenames containing hist.
- Non-baseline warming index from the final warming digit before _fire.

Current warming index mapping:

Warming degrees	Warming index
0	0
1	1
2	2
4	3
6	4

Configured Source Folders

The current `ScenarioConfig_BigCreek.json` points to these relative folders:

```
../Data/BigCreek/fire_cubes/
../Data/BigCreek/aggregate/
```

Those folders are not the same as the embedded `data/` folder. When running the config-driven path, make sure the configured folders exist relative to the importer working directory.

Source Formats Currently Parsed

Cube Text Files

Cube text files are parsed either by:

- Header-based config mapping through `ColumnMapper`.
- Legacy positional parsing if mapping fails or the legacy path is used.

The parser splits lines on spaces or tabs.

Dates

Dates are imported from the first aggregate file whose filename contains `hist`. The legacy dates path reads year/month/day from fixed positions in each data line.

Water JSON

The legacy water importer expects:

```
WaterData.json
```

inside the configured water folder.

Fire JSON

The legacy fire importer enumerates files in the configured fire folder, infers warming from each filename, deserializes JSON fire frame records, and writes serialized JSON records to the database model.

Patch JSON

The legacy patch importer expects:

PatchData.json

inside the configured patch folder.

Terrain JSON

The legacy terrain importer enumerates files in the configured terrain folder and expects filenames shaped like:

terrain_warm1_1942_10_4_4.json

It extracts warming, year, month, grain size, and decimal precision from filename segments.

Current Source-Data Assumptions

- Warming levels are fixed to baseline, +1 C, +2 C, +4 C, and +6 C.
- Aggregate cube data has a slightly different positional layout than patch cube data.
- Config-driven cube import expects source file headers that can be mapped to model field names.
- Legacy cube import relies on column positions.
- File discovery searches only the top level of configured input folders.
- The importer does not currently validate a full source-data manifest before writing.

Current Data Management Concerns

- The embedded data is large enough that Git/LFS policy should be decided before committing it.
- The importer includes generated build output folders in the embedded tree; those are not source data.
- The importer includes a nested Git repository; that should be resolved before the importer is fully absorbed into the parent repository.
- Source data and test fixture data are not yet clearly separated.

RHESSys Data Importer Scenario Config

Source: *Docs/RHESSysDataImporter/ScenarioConfig.md*

Last updated: 2026-06-13

Purpose

The current importer uses a JSON scenario config to describe the active scenario, database connection, input folders, file patterns, column mappings, transform placeholders, flags, and output tables.

Default config:

RHESSys_Data_Importer/RHESSys_Data_Importer/ScenarioConfig_BigCreek.json

Current Scope

The config is used by the importer only. The Unity runtime does not currently load this file.

The current config is Big Creek-oriented and should be treated as part of the data import pipeline, not as a complete runtime scenario definition.

Top-Level Shape

Current top-level fields:

Field	Purpose
scenarioName	Human-readable scenario name
description	Scenario description
version	Config/scenario version label
database	MySQL connection settings
inputFolders	Named source data folders
filePatterns	Glob patterns by importer category
columnMapping	Source-column to importer-model mappings
transforms	Placeholder transform lists by category
flags	Scenario behavior hints
outputTables	Table names expected as import targets

Database Section

Current fields:

Field	Purpose
name	Database name
host	MySQL host
port	MySQL port
user	MySQL user
password	MySQL password

Field	Purpose
charset	MySQL charset
collation	MySQL collation

DatabaseConfig.GetConnectionString() builds:

```
server={Host};port={Port};database={Name};user={User};password={Password};charset={Charset};
```

DatabaseConfig.GetAdminConnectionString() omits the database name.

Input Folders

The current Big Creek config defines:

Key	Configured meaning
cubeData	Cube source files
patchOutput	Patch-level output
basinOutput	Basin-level output
spatialData	Spatial data
climateData	Climate data

FileDiscovery currently searches all configured folders for every category pattern. It does not bind a category to only one folder key.

Paths are normalized with Path.GetFullPath, so relative paths are resolved from the importer working directory.

File Patterns

The current config recognizes:

Category	Current pattern
cube	p*_2veg_*_.txt
patch	extent100m_*.txt
terrain	patches100m*.txt
fire	firemask*.txt
water	ext100m_*.txt
climate	clim*.txt

The importer uses Directory.GetFiles(dir, pattern, SearchOption.TopDirectoryOnly).

Column Mapping

Column mapping is currently meaningful for config-driven cube imports.

The mapping shape is:

```
{
  "columnMapping": {
    "cube": {
      "source_column_name": "targetFieldName"
    }
  }
}
```

ColumnMapper reads the first non-empty line as a header, splits it on spaces or tabs, and matches source column names case-insensitively. It then allows TextFileInput to read target fields by name.

If no mapped columns are matched, cube import logs an error and falls back to legacy positional parsing.

Flags

Current flags:

Field	Purpose
hasFire	Indicates that scenario includes fire data
vegetationLayers	Indicates expected vegetation layer count

These flags are currently light metadata. They do not yet drive a comprehensive runtime or importer strategy.

Output Tables

Current output tables:

- cubedata
- patchdata
- terraindata
- firedata
- waterdata
- dates

The list documents expected targets but does not automatically create or validate all tables.

Current Config Issues To Know

- The checked-in description contains mojibake for the temperature range text.
- Big Creek and Central Coast sample files now share the RHESSys_Data_Importer/Data/ root.
- Some config categories exist before full importer support exists.

- The config includes climate mappings, but climate import is not implemented.
- Database passwords should not be committed for real shared environments.

RHESSys Data Importer Scenario Profiles

Source: *Docs/RHESSysDataImporter/ScenarioProfiles.md*

Last updated: 2026-06-13

Purpose

This document defines the scenario-profile concept added to the importer as task CCV2-02 in *Tasks/CentralCoastV2_Importer_TaskGraph.md*.

A scenario profile is an explicit, curated identifier for a scenario's data model. It lets the importer support more than one data model side by side without breaking Big Creek v1, and without guessing the data model from table names or from which files happen to be present.

Known Profiles

Profile	Meaning
BigCreekV1	Existing Big Creek data model (legacy default).
CentralCoastV2	Central Coast v2 RHESSys-derived data model.

These are represented in code by the `ScenarioProfileKind` enum in `Configuration/ScenarioProfile.cs`.

Selection

The active profile is selected explicitly through the scenario config field:

```
{
  "scenarioName": "BigCreek",
  "scenarioProfile": "BigCreekV1"
}
```

Resolution rules (`ScenarioProfiles.ResolveOrDefault`):

- A recognized value selects that profile.
- A missing/empty value defaults to BigCreekV1.
- An unknown value defaults to BigCreekV1 and logs a warning.

Recognized aliases are case-insensitive and tolerate a few spellings, e.g. BigCreekV1 / bigcreek, and CentralCoastV2 / centralcoast.

The default of BigCreekV1 is deliberate: existing Big Creek configs that omit `scenarioProfile` keep their current behavior unchanged.

Why Not Infer The Profile

The importer must not infer the data model from:

- table names in the database,
- which CSV/text files are present,
- folder layout alone.

Inference is fragile and would let an accidental file or table silently change parsing/import behavior. Instead, the profile is declared in config and logged at startup so every run states which data model it is using.

Current Behavior

- `Program.cs` resolves and prints the active profile at startup and warns on an unknown value.
- `WizardRunner` displays the active profile during scenario confirmation.
- `ScenarioConfig_BigCreek.json` now declares `"scenarioProfile": "BigCreekV1"`.

At this stage the profile is established and surfaced, but it does not yet branch import logic. Profile-specific config, schema, model classes, and import paths for Central Coast v2 are implemented in later tasks (CCV2-03 onward). Big Creek v1 remains the only profile with an implemented import path until then.

Compatibility

- Big Creek v1 is the default; omitting `scenarioProfile` preserves existing behavior.
- Adding `CentralCoastV2` does not change any Big Creek path.
- New profiles should be added as enum values plus explicit handling, never as implicit behavior triggered by data shape.

RHESSys Data Importer Scenario Upgrade Guide

Source: *Docs/RHESSysDataImporter/ScenarioUpgradeGuide.md*

Last updated: 2026-06-14

Purpose

This guide explains the recommended path for adding a future RHESSys data format after Big Creek v1 and Central Coast v2.

The goal is to add a new scenario path beside existing ones, not to mutate older imports in place. Big Creek v1 and Central Coast v2 should continue to run with their existing configs, tables, parser assumptions, API behavior, and Unity assets unless a separate compatibility task explicitly changes them.

Short Version

Yes: start with a new ScenarioConfig.

Then decide whether the new data format can reuse an existing scenarioProfile or needs a new one. If the files, columns, grains, identifiers, or derived Unity data are meaningfully different, add a new profile rather than forcing the new data through an older importer path.

Recommended sequence:

1. Create a new scenario config JSON.
2. Inventory the source files and document their grains.
3. Decide whether a new scenario profile is needed.
4. Design raw import tables before API or Unity work.
5. Add model classes and database context mappings.
6. Create/export the target database schema.
7. Add validators and dry-run row-count checks.
8. Implement importers incrementally by data category.
9. Smoke test against a separate database.
10. Design derived API/Unity data only after raw import is proven.

Step 1: Create A New ScenarioConfig

Copy the nearest existing config:

- ScenarioConfig_BigCreek.json for Big Creek-style data.
- ScenarioConfig_CentralCoastV2.json for Central Coast-style data.

For a new format, give the config its own scenario identity:

```
{
  "scenarioProfile": "NewScenarioV3",
  "scenarioRunId": "new_scenario_v3_baseline_001",
  "warmingIdx": 0,
  "sourceRoot": "../Data/NewScenarioV3",
  "delimiter": ",",
  "files": {
    "dailyAggregate": "example_daily.csv"
  }
}
```

Use scenarioRunId for the specific run/member being imported. Use warmingIdx for Future Mountain comparison cases. If the first sample is a baseline or unknown case, use warmingIdx = 0 and document that assumption.

Step 2: Inventory The Source Data

Before writing importer code, create a data-format doc that answers:

- What files exist?
- What is each file's grain: daily, monthly, yearly, static, raster, or derived?
- What are the columns?
- Which columns identify space: cube, basin, hill, zone, patch, stratum, pixel, raster value?
- Which columns identify time?
- What date range is present?
- How many rows are expected?
- Are there multiple warming/climate/scenario members?
- Which files are raw RHESys output, and which are derived assets?

For Central Coast v2, this lives in:

`Docs/CentralCoastV2/DataFormats.md`

For a future format, create the equivalent under a new folder, for example:

`Docs/NewScenarioV3/DataFormats.md`

Step 3: Choose Or Add A Scenario Profile

Use an existing profile only if the new data matches that profile's assumptions.

Add a new profile when any of these change meaningfully:

- file naming
- column names
- temporal grain
- spatial identifiers
- raster/patch-map strategy
- table shape
- required derived data for Unity
- normalization or aggregation rules

Profiles currently include:

- BigCreekV1
- CentralCoastV2

A future profile should be explicit, such as:

`NewScenarioV3`

Do not infer a scenario from file names alone. The config should say which profile is active.

Step 4: Design Raw Import Tables First

Start with raw/staging tables that preserve the source data. Avoid jumping directly to Unity-facing tables.

For each raw table, define:

- source file
- table name
- primary key strategy
- scenario columns: scenarioRunId, warmingIdx, importRunId
- source provenance: sourceFile when useful
- indexes needed for validation and later transformations

Keep existing scenario databases separate when the table meaning changes. For example, Central Coast v2 uses its own database/settings instead of forcing new rows into Big Creek's schema.

Step 5: Add Models And DbContext Entries

Add model classes under the scenario namespace when the table shape is scenario-specific:

```
Models/CentralCoast/  
Models/NewScenarioV3/
```

Then expose them through the appropriate scenario DbContext.

Do not modify older model classes unless the task is explicitly a backward-compatible cleanup and has been reviewed as such.

Step 6: Create And Export The Database Schema

Each scenario profile should have an explicit target schema/database. Do not reuse defaultdb or an older scenario schema when the table meanings or shapes have changed.

For example:

```
futuremtn_central_coast  
futuremtn_ventana_wilderness_v3
```

The most user-friendly setup path is:

1. Create the empty schema in MySQL Workbench with **Create Schema**.
2. Open the scenario SQL export from Database/Schema/.
3. Select the new schema as the active/default schema, or add `USE <schema>;` near the top of the SQL editor.
4. Run the script.

5. Verify with `SHOW TABLES;`.

For Central Coast v2, the checked-in schema export is:

`Database/Schema/CentralCoastV2_schema.sql`

Future scenarios should add the equivalent file, for example:

`Database/Schema/VentanaWildernessV3_schema.sql`

Schema export files may include table-level character set/collation details, but they do not necessarily create the database or select it. Keep the create-schema step explicit in the runbook.

EF Core migrations can still be used to generate or evolve schema, but applying migrations directly to a production/staging server requires checking the design-time DbContext factory connection first. Prefer a reviewed SQL export for manual server setup unless deployment automation has been wired.

Step 7: Add Validation Before Import

Each new format should support dry-run validation before database writes.

Minimum validation:

- all configured files exist
- headers match expected columns
- row counts match expected counts
- date/month ranges match expected ranges
- expected spatial IDs are present
- missing/extra IDs are reported clearly

The dry run should print enough detail to catch source-data surprises before a long import starts.

Step 8: Implement Importers Incrementally

Import one category at a time. Prefer this order:

1. date/time dimension
2. daily aggregate/basin data
3. daily cube or point data
4. monthly patch/zone data
5. monthly stratum/vegetation data
6. static spatial assets such as patch maps
7. derived API/Unity data

After each category:

- run a dry run
- run a small or staging import
- compare database row counts to source row counts
- spot-check a few source rows against database rows

Step 9: Keep Derived Terrain/API Data Separate

Raw RHESSys tables are not necessarily what Unity should consume.

Use this shape:

```
raw RHESSys source tables
-> validated database import
-> precomputed API/Unity-facing data
-> Unity visualization
```

For Central Coast v2, StratumData, BurnData, and PatchData are raw or source-derived inputs. The current generator derives precomputed TerrainData frames from them. FireData is reserved for Unity fire-spread frames, not monthly RHESSys burn.

Step 10: Preserve Existing Behavior

Every upgrade should explicitly state what remains untouched:

- Big Creek v1 config and database assumptions
- Central Coast v2 config and database assumptions
- existing Unity assets
- existing API endpoints
- existing importer commands

If a shared class must change, document why and test the older scenario path.

Suggested Documentation Checklist

For a future scenario, add:

- Docs/NewScenarioV3/DataFormats.md
- Docs/NewScenarioV3/BigCreekV1Differences.md or equivalent comparison doc
- Docs/NewScenarioV3/ScenarioConfig.md
- Docs/NewScenarioV3/Schema.md
- Docs/NewScenarioV3/DerivedTerrainDataPlan.md if Unity landscape output is needed
- task graph entries under Tasks/

Update existing importer docs only for shared behavior, such as new command-line flags or new profile names.

Suggested Implementation Checklist

- Add scenario profile enum/name.
- Add ScenarioConfig_NewScenarioV3.json.
- Add model classes for new raw tables.
- Add or extend a scenario DbContext.
- Add or update Database/Schema/<ScenarioName>_schema.sql.
- Add DAL write methods.
- Add validator checks.
- Add importer methods per category.
- Wire auto mode flags.
- Wire wizard categories if interactive import is needed.
- Add dry-run examples to BuildingAndRunning.md.
- Run build.
- Run dry-run validation.
- Run staging database smoke test.

Rule Of Thumb

If the new data format changes only file paths or scenario metadata, use a new config.

If it changes columns, identifiers, table shape, temporal grain, spatial mapping, or Unity-facing derived data, use a new config plus a new scenario profile.

RHESSys Data Importer Import Pipeline

Source: Specs/RHESSysDataImporter/ImportPipeline.md

Last updated: 2026-06-16

Scope

This spec documents the current import pipeline implemented by the embedded RHESSys Data Importer.

It does not specify future scenario support or new data formats.

Entry Point

The importer starts in:

RHESSys_Data_Importer/RHESSys_Data_Importer/Program.cs

Startup responsibilities:

- Initialize import-category booleans.

- Define legacy folder paths.
- Load the requested scenario config, defaulting to `ScenarioConfig_BigCreek.json`.
- Set the active DB connection override.
- Discover files through `FileDiscovery`.
- Parse command-line flags.
- Dispatch to wizard, auto, or legacy import paths.

Config Load

Default path:

`ScenarioConfig_BigCreek.json`

`--config <path>` overrides the default path. `ScenarioConfigLoader.Load(path)` uses `Newtonsoft.Json` to deserialize the file into `ScenarioConfig`.

If the config loads:

1. The importer logs scenario and database information.
2. `ConnectionHelper.SetOverride(config.Database.GetConnectionString())` sets the connection string for subsequent `DbContext`s.
3. `FileDiscovery.FindFiles(config)` builds a category-to-files result.

If the config does not exist, the importer logs that it is running the legacy Big Creek importer.

Flag Parsing

Flags are read from:

`Environment.GetCommandLineArgs().Skip(1)`

Recognized flags:

- `--auto`
- `--dryrun`
- `--force`
- `--config <path>`
- `--dates`
- `--cubes`
- `--patch`
- `--terrain`
- `--fire`
- `--burn`
- `--water`
- `--climate`

- `--stratum`

If `--auto` is absent and a config is active, `WizardRunner.Run(activeConfig, dryrun)` is called and `Program.cs` returns afterward.

Interactive Pipeline

`WizardRunner.Run` performs:

1. Scenario confirmation.
2. Optional alternate config load.
3. Existing database vs simulated new database choice.
4. File discovery.
5. Data category selection.
6. Central Coast v2 pre-import validation when that profile is active.
7. Optional cube column-mapping preview.
8. Final confirmation.
9. Selected import execution.

Current `RunSelectedImports` implementation:

- Big Creek v1 cube: imports discovered cube files through `TextFileInput.ReadCubeDataFiles`.
- Big Creek v1 patch, terrain, fire, water, climate: logs that import is not implemented in wizard mode.
- Central Coast v2 dates: derives dates from `cubeAggregateDaily`.
- Central Coast v2 cube: imports patch rows and applies stratum values.
- Central Coast v2 water: imports daily aggregate water data.
- Central Coast v2 burn: imports basin and patch monthly burn data.
- Central Coast v2 patch: imports patch-map spatial data.
- Central Coast v2 stratum: imports stratum carbon data.
- Central Coast v2 terrain: generates terrain data from completed upstream Central Coast tables.
- Central Coast v2 fire: currently reports that no fire-frame source is configured for the checked-in scenario.

Auto Pipeline

When `--auto` is present:

1. The importer logs auto mode.
2. `--dryrun` and `--force` are logged if present.
3. Category flags are checked.
4. If any category flag exists, only those categories are enabled.

5. If no category flag exists, cube, patch, terrain, fire, water, dates, and stratum are enabled. Burn is also enabled for Central Coast v2.
6. Climate is recognized as a placeholder but not imported.

Auto cube behavior:

- If config and discovered cube files exist, import those files unless dry-run is enabled.
- If config exists but no cube files are discovered, warn and fall back to legacy cube import unless dry-run is enabled.
- If no config exists, run legacy cube import unless dry-run is enabled.

Auto patch, terrain, fire, burn, and water behavior:

- These call legacy methods in `TextFileInput` unless dry-run is enabled.

Central Coast v2 overrides this legacy behavior:

- `--dates` calls `CentralCoastImporter.ImportDates`.
- `--cubes` ensures dates exist, then calls `CentralCoastImporter.ImportCubePatchData` and `CentralCoastImporter.ImportCubeStratumData`.
- `--water` ensures dates exist, then calls `CentralCoastImporter.ImportWaterData`.
- `--fire` calls `CentralCoastImporter.ImportFireData`.
- `--burn` calls `CentralCoastImporter.ImportBasinBurnData` and `CentralCoastImporter.ImportPatchBurnData`.
- `--patch` calls `CentralCoastImporter.ImportPatchMapData`.
- `--stratum` calls `CentralCoastImporter.ImportStratumCarbonData`.
- `--terrain` calls `CentralCoastImporter.GenerateTerrainData`.
- `--fire` is reserved for Unity-compatible fire-frame playback data: event date, fire grid dimensions, patch/zone id, spread, and iter/order.
- `--burn` imports monthly RHESys burn state into `BurnData`; it is not fire playback data.
- Central Coast fire-frame generation should use existing `PatchData` as the landscape patch/zone grid map.
- The current Central Coast v2 config includes one empty placeholder file role for future fire-frame source data: `fireFrameSpreadIter`.
- Until that role points at a real source file and a concrete parser is wired for its format, `--fire` writes zero rows and logs the expected role.

Cube Import Pipeline

Config-aware cube import is implemented in:

```
TextFileInput.ReadCubeDataFiles(IEnumerable<string> files, ScenarioConfig config)
```

For each file:

1. Infer warming index from filename.
2. Infer patch index from leading p... filename segment if present.
3. Read all lines.
4. Treat the first non-empty line as a header.
5. Build a ColumnMapper from the header and `config.ColumnMapping["cube"]`.
6. If no columns map, fall back to legacy positional parsing.
7. Warn for expected mapped fields not found in the header.
8. Convert each non-header data line into `CubeDataPoint`.
9. Write each row through `RHESSYSDAL.AddDataPoint`.

Legacy Cube Pipeline

Legacy cube import is implemented in:

```
TextFileInput.ReadCubeData(string folderAggregate, string folderCubes)
```

It imports:

- Aggregate files from `folderAggregate` with `patchIdx = -1`.
- Patch/cube files from `folderCubes` with patch id inferred from filename.

Rows are parsed by fixed numeric positions.

Aggregate rows omit the `transUnder` source position used by patch cube rows.

Legacy Non-Cube Pipelines

Current legacy methods:

Method	Source expectation	Target write method
<code>ReadDates</code>	Historical aggregate text file	<code>AddDate</code>
<code>ReadWaterData</code>	<code>WaterData.json</code>	<code>AddWaterDataFrame</code>
<code>ReadFireData</code>	JSON files in fire folder	<code>AddFireDataFrame</code>
<code>ReadPatchData</code>	<code>PatchData.json</code>	<code>AddPatchData</code>
<code>ReadTerrainData</code>	Terrain JSON files with metadata in filename	<code>AddTerrainDataFrame</code>

These paths are older and less config-driven than the cube path.

Database Write Pipeline

`RHESSYSDAL` creates a new `DbContext` for each record write and calls `SaveChanges`.

Current write methods:

- AddDataPoint
- AddDate
- AddWaterDataFrame
- AddPatchData
- AddTerrainDataFrame
- AddFireDataFrame

This is simple but may be slow for large imports.

Error Handling

Current behavior is mostly console logging and catch-and-continue.

Examples:

- File-level cube processing catches exceptions and logs warnings.
- Some legacy methods catch exceptions but ignore exception details.
- AddTerrainDataFrame and AddFireDataFrame catch DB exceptions and return false.

The importer does not currently produce a structured import report.

Current Acceptance Criteria

For the current importer baseline:

- The solution builds.
- The default config can be loaded from the correct working directory.
- File discovery prints counts and warnings.
- Cube import can run in wizard or auto mode.
- Legacy non-cube paths remain callable from auto mode.
- MySQL connection strings flow through ConnectionHelper.

RHESSys Data Importer Scenario Config Schema

Source: Specs/RHESSysDataImporter/ScenarioConfigSchema.md

Last updated: 2026-06-13

Scope

This spec documents the current scenario config object model implemented in:

`Configuration/ScenarioConfig.cs`

It is descriptive, not a future schema proposal.

Object Model

Current C# classes:

- ScenarioConfig
- DatabaseConfig
- ScenarioFlags

ScenarioConfig

Property	Type	Required by current code	Notes
ScenarioName	string	Expected	Logged at startup and in wizard
Description	string	Not actively required	Descriptive metadata
Version	string	Not actively required	Descriptive metadata
Database	DatabaseConfig	Yes	Used to build connection string
InputFolders	Dictionary<string, string>	Yes for discovery	Empty/missing collection produces warning
FilePatterns	Dictionary<string, string>	Yes for discovery	Empty/missing collection produces warning
ColumnMapping	Dictionary<string, Dictionary<string, string>>	Optional	Cube mapping currently uses cube key
Transforms	Dictionary<string, List<string>>	Not actively used	Placeholder metadata
Flags	ScenarioFlags	Not actively required	Light metadata
OutputTables	List<string>	Not actively enforced	Documents expected target tables

Because nullable warnings are currently present, these properties are not initialized in constructors even though many are expected by runtime paths.

DatabaseConfig

Property	Type	Current use
Name	string	Database name in

Property	Type	Current use
Host	string	connection string Server host in connection string
Port	int	Server port in connection string
User	string	User in connection string
Password	string	Password in connection string
Charset	string	Charset in connection string
Collation	string	Present in config, not included in regular connection string

Generated connection string:

```
server={Host};port={Port};database={Name};user={User};password={Password};charset={Charset};
```

Generated admin connection string:

```
server={Host};port={Port};user={User};password={Password};charset={Charset};
```

ScenarioFlags

Property	Type	Current use
HasFire	bool	Metadata only in current code
VegetationLayers	int	Metadata only in current code

JSON Naming

The current JSON uses camelCase names such as `scenarioName`, while C# properties use PascalCase such as `ScenarioName`.

Newtonsoft.Json maps these successfully with default case-insensitive behavior.

Discovery Categories

FileDiscovery has a fixed category list:

```
cube
patch
terrain
fire
water
climate
```

These categories must be present in `filePatterns` to avoid missing-pattern warnings.

Central Coast Source File Roles

Central Coast v2 uses `sourceRoot` plus logical files roles instead of the legacy `inputFolders/filePatterns` discovery path.

Current roles include:

Role	Current meaning
<code>cubeAggregateDaily</code>	Daily aggregate cube/water source (<code>cube_agg_p.csv</code>)
<code>cubePatchDaily01</code> , <code>cubePatchDaily02</code>	Daily cube patch sources
<code>cubeStratumOver01</code> , <code>cubeStratumOver02</code>	Overstory cube stratum sources
<code>cubeStratumUnder01</code> , <code>cubeStratumUnder02</code>	Understory cube stratum sources
<code>basinMonthlyBurn</code>	Monthly basin burn source for <code>BurnData</code>
<code>patchMonthlyBurn</code>	Monthly patch/zone burn source for <code>BurnData</code>
<code>stratumMonthlyCarbon</code>	Monthly stratum carbon source for <code>StratumData</code>
<code>patchFamilyRaster</code>	TIFF patch/zone footprint source for <code>PatchData</code>
<code>fireFrameSpreadIter</code>	Placeholder for future fire event rows with date, patch/zone id, spread, and iter/order

The fire-frame role is intentionally present even though the current Central Coast bundle has no such file. It makes the intended `--fire` pipeline explicit. Monthly RHESys burn belongs to `--burn/BurnData`; Unity fire playback belongs to `--fire/FireData`.

Central Coast fire-frame generation should reuse existing `PatchData` for the landscape patch/zone grid map. `PatchData` currently comes from `patchFamilyRaster`, so a separate fire-specific patch map role is not required unless a future data bundle supplies a different fire-only spatial index.

Column Mapping Contract

Column mapping entries are source-to-target:

source header name -> importer model field name

For cube data, target field names are expected to match names requested in `TextFileInput.AddDataPointMapped`, including:

- snow
- evap

- netpsn
- depthToGW
- vegAccessWater
- Qout
- litter
- soil
- heightOver
- transOver
- heightUnder
- transUnder
- leafCOver
- stemCOver
- rootCOver
- leafCUnder
- stemCUnder
- rootCUnder

The mapper is case-insensitive for source header matching. Target lookup uses the default dictionary comparer, so target field casing must match the target strings requested by importer code.

Current Validation Behavior

The importer currently validates only lightly:

- Missing inputFolders emits a warning.
- Missing filePatterns emits a warning.
- Missing category pattern emits a warning.
- Missing input folder emits a warning.
- No matched files emits a warning.
- Missing mapped cube target fields emits warnings.

The importer does not currently:

- Validate JSON against a formal schema.
- Validate database existence before import.
- Validate target table schema before import.
- Validate source file row counts or date ranges before import.
- Validate that every output table exists.

Current Schema Compatibility Notes

The config currently describes more than the importer fully implements. For example, climate mappings and transform lists are present, but climate import and transforms are not implemented.

RHESSys Data Importer File Naming And Discovery

Source: *Specs/RHESSysDataImporter/FileNamingAndDiscovery.md*

Last updated: 2026-06-13

Scope

This spec documents the current filename assumptions and discovery behavior in the embedded importer.

File Discovery

Implemented in:

`IO/FileDiscovery.cs`

Inputs:

- `ScenarioConfig.InputFolders`
- `ScenarioConfig.FilePatterns`

Output:

- `FileDiscoveryResult.FilesByCategory`
- `FileDiscoveryResult.Warnings`

Discovery Algorithm

For each configured input folder:

1. Normalize the folder path with `Path.GetFullPath`.
2. Trim trailing directory separators.
3. Warn if the folder does not exist.

For each fixed category:

1. Read the category pattern from `FilePatterns`.
2. Search every configured folder using `Directory.GetFiles`.
3. Use `SearchOption.TopDirectoryOnly`.
4. Add matching files to the category list.
5. Normalize files with `Path.GetFullPath`.
6. De-duplicate case-insensitively.

Fixed Categories

Current fixed categories:

Category	Purpose
cube	Cube-level RHESSys output
patch	Patch-level output
terrain	Terrain/spatial terrain data
fire	Fire mask/fire output
water	Water or basin output
climate	Climate output placeholder

Current Config Patterns

Category	Pattern
cube	p*_2veg_*_*.txt
patch	extent100m_*.txt
terrain	patches100m*.txt
fire	firemask*.txt
water	ext100m_*.txt
climate	clim*.txt

Cube Filename Parsing

Config-aware cube import infers metadata from filenames.

Patch Id

If a filename starts with p, the importer:

1. Splits the filename on _.
2. Reads the first segment.
3. Skips the leading p.
4. Takes subsequent digits.
5. Parses those digits as patchIdx.

Example:

```
p1239496_2veg_hist_fire.txt -> patchIdx 1239496
```

Warming Index

If the filename contains hist, warming index is:

0

Otherwise, if the filename contains `_fire`, the importer:

1. Splits the filename at `_fire`.
2. Reads the last character before `_fire`.
3. If it is a digit, maps it through `WarmingDegreesToIndex`.

Current mapping:

Filename warming degree	Warming index
0	0
1	1
2	2
4	3
6	4
Other	-1

Legacy Aggregate Filename Parsing

Aggregate files are read from the aggregate folder.

If filename contains:

`hist`

then warming index is 0.

Otherwise the legacy parser expects the warming digit before `_fire`.

Aggregate rows are imported with:

`patchIdx = -1`

Legacy Fire Filename Parsing

Legacy fire import enumerates files in the configured fire folder and extracts warming from the last character before `_fire`.

The parsed value is used directly as `warmingIdx` in current code, rather than through `WarmingDegreesToIndex`.

Legacy Terrain Filename Parsing

Legacy terrain import expects filename segments like:

`terrain_warm1_1942_10_4_4.json`

The parser extracts:

Segment	Meaning
---------	---------

Segment	Meaning
warm1	Warming index digit
1942	Year
10	Month
4	Grain size
4	Decimal precision

Discovery Limitations

- Discovery does not recurse.
- Discovery searches every folder for every category, not a specific folder per category.
- Discovery does not validate filename metadata before import.
- Discovery does not check that all warming levels are present.
- Discovery does not check that all expected patch/cube ids are present.
- Discovery does not check row counts or date coverage.

Current Practical Implications

- Wrong working directory can make relative input folders resolve incorrectly.
- A broad pattern can pick up unintended files from any configured folder.
- Missing files produce warnings but do not necessarily stop the import.
- Existing file naming conventions are part of the current importer contract.

RHESSys Data Importer Database Write Contract

Source: Specs/RHESSysDataImporter/DatabaseWriteContract.md

Last updated: 2026-06-13

Scope

This spec documents the current database write behavior in the RHESSys Data Importer.

It does not define a future database schema.

Database Provider

All current DbContexts configure MySQL through Pomelo:

```
optionsBuilder.UseMySQL(cs, ServerVersion.AutoDetect(cs));
```

SQL Server packages and comments remain in the project, but the active write paths use MySQL connection strings.

Connection String Resolution

Implemented in:

DAL/ConnectionHelper.cs

Resolution order:

1. If `ConnectionHelper.SetOverride` has been called with a non-empty connection string, return that.
2. If `appsettings.Development.json` exists, try to read `Database.ConnectionString`.
3. Fall back to a local MySQL placeholder:

```
server=localhost;port=3306;database=bigcreek_rhessys;user=root;password=;char  
set=utf8mb4;
```

When the default scenario config loads, `Program.cs` sets the override using the config database section.

DAL Class

Implemented in:

DAL/RHESSYSDAL.cs

Current write methods:

Method	DbContext	DbSet
AddDataPoint	CubeDataDbContext	CubeData
AddDate	DatesDbContext	Dates
AddWaterDataFrame	WaterDataDbContext	WaterData
AddPatchData	PatchDataDbContext	PatchData
AddTerrainDataFrame	TerrainDataDbContext	TerrainData
AddFireDataFrame	FireDataDbContext	FireData

Write Pattern

The current DAL pattern is:

1. Construct a `DbContext`.
2. Add one record.
3. Call `SaveChanges`.
4. Return true if `SaveChanges()` reports more than zero changes.

This means large file imports call `SaveChanges` once per imported row.

Current DbContexts

DbContext	Model
-----------	-------

DbContext	Model
CubeDataDbContext	CubeDataPoint
DatesDbContext	Date
WaterDataDbContext	WaterDataFrame
PatchDataDbContext	PatchDataRecord
TerrainDataDbContext	TerrainDataFrameJSONRecord
FireDataDbContext	FireDataFrameJSONRecord

Each DbContext stores a connection string and configures itself if options have not already been configured.

Cube Data Contract

CubeDataPoint is written by config-aware and legacy cube imports.

Current importer-populated fields include:

- dateIdx
- warmingIdx
- patchIdx
- snow
- evap
- netpsn
- depthToGW
- vegAccessWater
- Qout
- litter
- soil
- heightOver
- transOver
- heightUnder
- transUnder
- leafCOver
- stemCOver
- rootCOver
- leafCUnder
- stemCUnder
- rootCUnder

Aggregate cube imports use patchIdx = -1.

Dates Contract

Dates are written through `AddDate`.

Legacy date import reads from historical aggregate text data and writes:

- year
- month
- day
- date

Patch Data Contract

Patch import reads `PatchData.json` into a dictionary of `PatchPointCollection` values.

The DAL converts each collection to `PatchDataRecord` and writes:

- patchID
- serialized patch data through `SetData`

Fire Data Contract

Fire import reads JSON fire frame records, converts them to `FireDataFrameJSONRecord`, assigns `warmingIdx`, and writes serialized frame data.

Terrain Data Contract

Terrain import reads flattened integer arrays from JSON files, extracts metadata from filenames, converts to `TerrainDataFrameJSONRecord`, and writes serialized terrain data.

Water Data Contract

Water import reads `WaterData.json`, traverses years/months/frames, increments frame index, and writes `WaterDataFrame` records.

Current Schema Management

The project includes EF migration files for cube data and dates. It does not currently provide a clear complete migration workflow for every table listed in the scenario config.

The wizard's create-database option is simulated and does not create schema.

Current Write Limitations

- No bulk insert path.
- No transaction across a whole file or import run.
- No duplicate detection.
- No truncate/replace safety mechanism.
- No import manifest table.

- No structured row-level error report.
- No complete schema validation before import.
- No production/staging guardrail.

Current Operational Contract

Before running an import, the operator is responsible for confirming:

- The database exists.
- The expected tables exist.
- The schema matches the model classes.
- The database target is staging or otherwise safe.
- Existing data has been backed up if needed.

Big Creek Scenario

Big Creek Version

Source: Docs/Versions/BigCreekVersion.md

Last updated: 2026-06-16

Scope

This document describes the current Big Creek version of Future Mountain as implemented in this Unity repository. It is a baseline for comparison as newer scenario scenes are added.

Current Scenario

- Scenario: Big Creek watershed, Sierra Nevada.
- Scientific source: RHESys-derived model output.
- Warming levels: baseline, +1 C, +2 C, +4 C, +6 C.
- Runtime data source: Future Mountain API for web builds, with legacy TextAsset/Resources loading paths still present.
- Unity scene: Assets/Scenes/BigCreekV1/BigCreekV1.unity.

Primary User Features

Feature-level specs:

- [Normal Mode](#)
- [Side-by-Side Mode](#)
- [Timeline](#)
- [Show Model / Data Layer](#)

- [Data Model](#)
- [Data Mappings](#)
- [Fire](#)
- [Messages](#)
- [Lighting](#)
- [Snow](#)
- [Roots](#)
- [Soil](#)
- [Ground Water](#)

Runtime Behavior

- The user can start the simulation, select warming level, pause/resume, use the timeline, inspect cubes, and enter Side-by-Side Mode.
- The scene contains one aggregate cube, five detailed terrain cubes, matching side-by-side cube instances, and one large watershed terrain.
- Fire, snow, river/streamflow, vegetation, roots, groundwater, and cube statistics appear consistently with the deployed Big Creek behavior.
- Web builds use `https://data.futuremtn.org/api/`.
- Local standalone builds use `http://localhost:5550/api/`.
- Web optimization reduces vegetation density and increases carbon factors in `SimulationSettings.OptimizeForWeb`.

Known Constraints

- Big Creek assumptions are still embedded in code, data, scene setup, and UI.
- Warming levels are hard-coded in multiple places.
- Several data formats are positional rather than schema-driven.
- Some non-web data loading paths remain but may not be current.
- API/database schema is external to this Unity repo.

Baseline Check

- Project opens in Unity 2022.3.62f3.
- `Assets/Scenes/BigCreekV1/BigCreekV1.unity` loads without missing scripts.
- WebGL build can load Big Creek data from the API.
- Timeline, messages, terrain, cubes, Side-by-Side Mode, fire, snow, roots, groundwater, and model data layer remain functional.

Data Formats

Source: Docs/BigCreekV1/DataFormats.md

Last updated: 2026-06-12

Overview

Future Mountain Big Creek (v1) visualizes RHESSys-derived data through Unity runtime models, API JSON responses, and legacy TextAsset parsing paths. The runtime code still contains hard-coded column order assumptions, so this document should be treated as both documentation and a migration checklist.

Scenario Config

ScenarioConfig_BigCreek.json currently defines:

- Scenario name: BigCreek
- Database: bigcreek_rhessys on local MySQL defaults.
- Input folders for cube, patch, basin, spatial, and climate data.
- File patterns for cube, aggregate cube, patch, basin, and climate inputs.
- Column mappings for cube, patch, basin, and climate inputs.
- Output tables: cubedata, patchdata, terraindata, firedata, waterdata, dates.
- Flags: hasFire: true, vegetationLayers: 2.

This file is currently documentation/config for the data pipeline. The Unity runtime does not load this scenario config directly. This format may be used/expanded in the future for a generalized scenario loader.

Warming Levels

Current warming indices:

Index	Degrees
0	0 C
1	1 C
2	2 C
3	4 C
4	6 C

The same mapping appears in UI and controller code. Future scenarios should define this once.

Cube Data

Runtime DTO: Assets/Scripts/Models/CubeData.cs

Web/API fields consumed by Unity:

- id
- dateIdx

- warmingIdx
- patchIdx
- snow
- evap
- netpsn
- depthToGW
- vegAccessWater
- qout
- litter
- soil
- heightOver
- transOver
- heightUnder
- transUnder
- leafCOver
- stemCOver
- rootCOver
- leafCUnder
- stemCUnder
- rootCUnder

Legacy/TextAsset cube columns are indexed in `CubeController.DataColumnIdx`:

Column	Meaning
date	Date index/string value in source file
snow	Snow
evap	Evaporation
netpsn	Net photosynthesis
depthtogw	Depth to groundwater
vegaccesswater	Vegetation-accessible water
qout	Streamflow/outflow
litter	Litter
soil	Soil carbon/storage
height_over	Overstory height
trans_over	Overstory transpiration
height_under	Understory height
trans_under	Understory transpiration
leafc_over	Overstory leaf carbon

Column	Meaning
stemc_over	Overstory stem carbon
rootc_over	Overstory root carbon
leafc_under	Understory leaf carbon
stemc_under	Understory stem carbon
rootc_under	Understory root carbon
year	Year
month	Month
day	Day

Aggregate cube data has a related but slightly different enum that uses a single transpiration column.

Water Data

Runtime DTO: `Assets/Scripts/Models/WaterData.cs`

Fields:

- index
- year
- month
- day
- qBase
- qWarm1
- qWarm2
- qWarm4
- qWarm6
- precipitation

Legacy/TextAsset water columns are indexed in `LandscapeController.WaterDataColumnIdx`:

- day
- month
- year
- date
- wy
- precip
- QBase
- QWarm1
- QWarm2

- QWarm4
- QWarm6

Patch Data

Patch columns are indexed in `LandscapeController.PatchDataColumnIdx`:

- month
- year
- patchID
- snow
- plantc
- spread
- iter

`ScenarioConfig_BigCreek.json` currently maps patch input columns for month, year, patchID, snow, and plantc; the runtime also expects fire-related spread and iter columns in legacy formatting paths.

Fire Data

Runtime DTO: `Assets/Scripts/Models/FireData.cs`

The API fire frame record includes:

- id
- warmingIdx
- year
- month
- day
- gridHeight
- gridWidth
- _dataList

`_dataList` is serialized JSON containing fire points. Runtime fire point classes are currently nested in `LandscapeController.cs`.

Terrain Data

Runtime DTO: `Assets/Scripts/Models/TerrainDataFrame.cs`

Terrain frame fields include:

- id
- warmingIdx
- year

- month
- gridSize
- pixelGrainSize
- decimalPrecision
- _dataList

_dataList is serialized JSON containing an integer array used to rebuild terrain/snow/texture data.

Dates

Runtime DTO: `Assets/Scripts/Models/DateModel.cs`

The API provides a date list used by `GameController` to map simulation indices to calendar dates.

Central Coast Migration Checklist

- Add any new source columns to the importer config, MySQL schema, API DTOs, and Unity DTOs together.
- Decide whether new fields are visualization-driving fields or metadata.
- Avoid relying on column position for new data where a named schema can be used.
- Define scenario-specific warming levels, dates, patch/cube counts, and labels in configuration.
- Version the data contract so Unity can detect incompatible API/importer data.

Central Coast Scenario

Central Coast Version

Source: *Docs/Versions/CentralCoastVersion.md*

Last updated: 2026-06-16

Scope

This document describes the current Central Coast v2 pre-prototype version of Future Mountain. It is a work-in-progress scenario beside Big Creek, not a full replacement for the deployed Big Creek experience.

Current Scenario

- Scenario: Central Coast v2.
- Unity scene: `Assets/Scenes/CentralCoastV2/CentralCoastV2.unity`.

- Source data: partial RHESys-derived sample bundle at RHESys_Data_Importer/Data/CentralCoast/RHESysOutput-SingleWarmIdx-6-4-2026/.
- Import profile: CentralCoastV2.
- Scenario run id: single-warming-sample.
- Current warming index assumption: `warmingIdx = 0`.
- API profile: CentralCoast, using `/api/centralcoast/...` routes.

Implemented So Far

- Dedicated Central Coast Unity scene.
- Dedicated Central Coast importer profile and database/schema path.
- Dates derived from `cube_agg_p.csv`.
- Cube, water, burn, patch mapping, stratum carbon, and monthly terrain generation import paths.
- Initial patch mapping from `Pch30rip90upRN.tiff` into `PatchData`.
- Initial monthly `TerrainData` generation from `PatchData`, `StratumData`, and `BurnData`.
- Prototype API routes under `/api/centralcoast/...`
- Central Coast-specific message resource paths in `SimulationSettings`.

Important Missing Data

The current Central Coast sample bundle does not yet include real Future Mountain warming scenarios. It is imported as one assumed baseline/member with `warmingIdx = 0`.

Fire spread playback data also does not exist yet for Central Coast. Monthly RHESys burn data is imported into `BurnData`, but that is not the same as Unity fire spread-frame data. `FireData` remains reserved for event/frame data with spread and iteration/order values once a suitable source is provided.

Current Constraints

- Warming comparison UI and Side-by-Side Mode still inherit Big Creek assumptions and need careful validation for a single-warming Central Coast sample.
- Fire simulation should not be treated as complete until Central Coast fire-frame source data exists.
- Carbon factors, vegetation density, snow, fire, and web multipliers need scenario-specific tuning in `SimulationSettings`.
- Central Coast uses rectangular terrain data (`gridWidth = 396`, `gridHeight = 301`) rather than Big Creek's square `gridSize` convention.
- Shared assets must be checked carefully so Central Coast visual tuning does not bleed into Big Creek.

Related Docs

- [Adding Future Scenarios](#)
- [Initial Patch Mapping](#)
- [Initial Terrain Data](#)
- [Central Coast Data Formats](#)
- [Central Coast vs Big Creek](#)
- [Importer Scenario Upgrade Guide](#)

Central Coast v2 vs Big Creek v1 Data Differences

Source: *Docs/CentralCoastV2/BigCreekV1Differences.md*

Last updated: 2026-06-13

Purpose

This document distinguishes the Central Coast v2 source data from the current Big Creek v1 data contract.

The goal is to preserve Big Creek v1 while adding Central Coast v2 beside it as an explicit scenario profile.

Summary

Big Creek v1 and Central Coast v2 should be treated as two known scenario/data-model profiles:

BigCreekV1

CentralCoastV2

Do not mutate Big Creek v1 tables, importer assumptions, Unity assets, or API behavior to fit Central Coast v2.

Central Coast v2 should get its own importer path and database/schema path. Compatibility should happen through explicit profile selection and later API/runtime adapters.

Major Differences

Area	Big Creek v1	Central Coast v2
Scenario members	Five warming levels are part of the current runtime assumption.	Current sample has no warming identifier and appears to be one scenario member.
Date range	Current Big Creek runtime uses its existing	Current sample covers 1987-07-01 through 2019-

Area	Big Creek v1	Central Coast v2
	historical/model date range.	06-30.
Cube file layout	Existing importer expects aggregate and per-cube text files using Big Creek naming conventions.	Current cube files contain all five cube/zone IDs in each CSV.
Cube identity	Big Creek cube identity is mostly patchIdx.	Central Coast cube identity is zoneID, with patchID members beneath it.
Patch model	Big Creek treats mapped patch IDs as direct spatial patches.	Central Coast uses patch families: zoneID maps spatially, while patchID is zoneID plus 01 or 02.
Strata	Big Creek runtime fields flatten overstory and understory values into one cube data row.	Central Coast provides separate overstory and understory stratum files.
Burn data	Big Creek has fire/burn data integrated into existing fire/terrain flows by warming index.	Current Central Coast burn data is monthly at basin and patch levels; daily cube burn is zero in the sample.
Spatial map	Big Creek patch extents are consumed as derived patch data / text-derived collections.	Central Coast provides a GeoTIFF patch-family raster that must be converted into equivalent patch collections.
Landscape scale	Big Creek current runtime uses existing terrain/assets and precomputed terrain data paths.	Central Coast uses the patch map raster to connect zoneID values to landscape footprints before precomputed TerrainData can be generated.
Import style	Big Creek importer includes legacy positional parsing and Big Creek file conventions.	Central Coast should use named CSV columns and profile-specific parsing.

Warming Scenario Difference

Big Creek v1 has a hard-coded mental model of five warming levels:

Index	Big Creek label
-------	-----------------

Index	Big Creek label
0	0 C / baseline
1	+1 C
2	+2 C
3	+4 C
4	+6 C

The current Central Coast v2 sample has no warming or climate-case field.

For initial import only, assign:

```
warmingIdx = 0
```

This preserves compatibility with Future Mountain’s comparison-oriented architecture while making the assumption explicit.

Future Central Coast data should identify scenario members through config or filename metadata, not through inference from table shape.

Patch Identity Difference

Big Creek v1:

```
patchID -> mapped patch footprint
```

Central Coast v2:

```
zoneID -> mapped patch-family footprint
patchID -> aspatial member inside patch family
stratumID -> canopy stratum inside patch member
```

Example:

```
zoneID 10071
  patchID 1007101
    stratumID 10071011 overstory
    stratumID 10071012 understory
  patchID 1007102
    stratumID 10071021 overstory
    stratumID 10071022 understory
```

The Central Coast patch map stores zoneID values. It does not distinguish patch 01 and patch 02 spatially.

For compatibility with existing patch-location runtime code, Central Coast can either:

- Store patch extents by zoneID and teach Central Coast-specific consumers to use patch families.

- Duplicate the same spatial footprint to patch 01 and patch 02 when an old-style patchID -> footprint lookup is needed.

The first option is cleaner long-term. The second option may be useful as a compatibility bridge.

Riparian Difference

In the current Central Coast sample, only zoneID = 10071 differs materially between patch 01 and patch 02.

The data-provider description says:

- Patch 01 is oak for the riparian cube and chaparral elsewhere.
- Patch 02 is chaparral.
- Both have grass understory.

The sample confirms that patch 01 and patch 02 values are identical for the other four cube locations, but differ for the riparian cube.

This means the importer must preserve patch 01 and patch 02 rows even though most locations currently duplicate values.

Stratum Difference

Big Creek v1 cube rows already contain fields like:

- heightOver
- transOver
- leafCOver
- stemCOver
- rootCOver
- heightUnder
- transUnder
- leafCUnder
- stemCUnder
- rootCUnder

Central Coast v2 provides these as separate stratum files:

- cubes_sc_over_patch*.csv
- cube_sc_under_patch*.csv

The importer can later derive Big Creek-like cube payloads from Central Coast records if needed, but the raw import should preserve the separated stratum structure.

Burn Difference

The current Central Coast daily cube files include a burn column, but the sample values are zero.

Burn is provided at monthly grain through:

- `bm.csv` for basin-level burn.
- `spatial_data_point_patchvar.csv` for patch-level burn.

That differs from Big Creek runtime expectations around fire frames and warming-indexed fire data.

Initial import should preserve monthly burn as source data. API/runtime fire behavior should be designed later.

Spatial Data Difference

Big Creek runtime consumes patch extents as `PatchPointCollection` records containing:

- data grid location
- fire-grid location
- terrain alphamap location
- UTM location

Central Coast provides the source raster needed to generate equivalent collections:

`Pch30rip90upRN.tiff`

The reusable part is the downstream `PatchPointCollection` contract. The decoder should be new because the input is GeoTIFF, not the old Big Creek text grid format.

Import Strategy Difference

Big Creek v1 should remain on its existing importer and database path.

Central Coast v2 should add:

- explicit scenario profile config
- separate connection/database settings
- new CSV readers
- raw/staging tables that preserve Central Coast identifiers
- prototype API projections that adapt Central Coast rows to the current Unity runtime contract

Recommended first-phase Central Coast tables live in the separate `centralcoast_rhessys` database, so they are not prefixed. They use the same PascalCase EF table convention as the existing importer models:

- Dates
- CubeData
- WaterData
- BurnData
- StratumData
- PatchData
- ImportRun

The table names are now fixed by `Docs/RHESSysDataImporter/CentralCoastSchema.md`. The important structural separation remains: daily per-cube data, daily basin aggregate water data, monthly burn, monthly full-landscape stratum carbon, and patch-map-derived spatial footprints stay distinct.

Central Coast now includes a derived `TerrainData` table generated from the imported source tables and patch-map spatial footprints. Big Creek `TerrainData` remains a derived monthly runtime payload for large-landscape splatmap/texture state. Central Coast terrain runtime data should continue to be kept separate from Big Creek data and served through scenario-explicit API routes.

API Prototype Compatibility

The Future Mountain API preserves the legacy Big Creek routes:

`/api/CubeData/...`

Central Coast prototype routes are added beside them:

`/api/centralcoast/CubeData/...`

The Central Coast database schema intentionally preserves Central Coast source identifiers and row shapes. The prototype API uses DTO/projection mapping to return the current Unity-friendly runtime shape where needed. For example, `zoneID` maps to legacy-style `patchIdx`, overstory and understory `netpsn` values are summed for legacy `netpsn`, and `litterc/soilc` map to `litter` and `soil`.

Future Central Coast v2 production endpoints can expose richer native fields, but the prototype routes should stay stable enough for the duplicated Central Coast Unity scene to call them without changing Big Creek.

Compatibility Rule

The guiding rule:

Big Creek v1 remains a preserved legacy scenario.
 Central Coast v2 becomes the first explicit new scenario profile.
 Shared abstractions should emerge only where the two scenarios truly overlap.

That keeps the first phase focused on importing Central Coast v2 data safely without breaking current Future Mountain behavior.

Central Coast v2 Data Formats

Source: *Docs/CentralCoastV2/DataFormats.md*

Last updated: 2026-06-13

Overview

Central Coast v2 is a newer RHESSys-derived data model than the current Big Creek v1 data used by Future Mountain.

The current sample bundle is located at:

`RHESSys_Data_Importer/Data/CentralCoast/RHESSysOutput-SingleWarmIdx-6-4-2026/`

This bundle appears to represent one scenario member only. The files do not contain a warming index, warming label, climate-case label, or filename token such as `hist`, `warm1`, `warm2`, `warm4`, or `warm6`.

For initial ingestion, treat this sample as:

```
scenarioProfile = CentralCoastV2
scenarioRun = single-warming-sample
warmingIdx = 0
```

This is an import assumption, not a confirmed scientific label. Future complete output is expected to include multiple warming or climate scenario members.

Date Coverage

Daily files cover:

1987-07-01 through 2019-06-30

That is 11,688 daily records per single-row daily series.

Monthly files cover:

1987-07 through 2019-06

That is 384 monthly records per single-row monthly series.

The date range reflects the WRF climate data window described by the data provider.

File Inventory

File	Rows	Grain	Notes
<code>cube_agg_p.csv</code>	11,688	Daily	One row per day

File	Rows	Grain	Notes
		basin/aggregate	
cube_p_patch1.csv	58,440	Daily cube patch 01	Five cube/zone rows per day
cube_p_patch2.csv	58,440	Daily cube patch 02	Five cube/zone rows per day
cubes_sc_over_patch1.csv	58,440	Daily overstory stratum, patch 01	Five cube/zone rows per day
cubes_sc_over_patch2.csv	58,440	Daily overstory stratum, patch 02	Five cube/zone rows per day
cube_sc_under_patch1.csv	58,440	Daily understory stratum, patch 01	Five cube/zone rows per day
cube_sc_under_patch2.csv	58,440	Daily understory stratum, patch 02	Five cube/zone rows per day
bm.csv	384	Monthly basin burn	One row per month
spatial_data_point_patchvar.csv	3,438,336	Monthly all-patch burn	8,954 patch rows per month
spatial_data_point_stratvar.csv	6,876,672	Monthly all-stratum carbon	17,908 stratum rows per month
Pch30rip90upRN.tiff	n/a	Raster	Patch-family map, used to derive spatial patch extents
dem30mSBFRbound.tiff	n/a	Raster	DEM/height raster for shaping the Unity Central Coast terrain

Spatial Identity Model

Central Coast v2 uses a multi-scale routing structure.

Key identifiers:

Field	Meaning
basinID	Basin identifier. Current sample uses basin 1.
hillID	Hillslope identifier.
zoneID	Patch family identifier. For the five cube files, this is the cube identifier.
patchID	Aspatial patch identifier. Encoded as zoneID

Field	Meaning
	plus 01 or 02.
stratumID	Canopy stratum identifier. Encoded as patchID plus 1 or 2.
veg_parm_ID	Vegetation parameter identifier. Current sample uses 11, 6, and 2.

Patch family examples:

```
zoneID 10071 -> patchID 1007101 and 1007102
patchID 1007101 -> stratumID 10071011 and 10071012
patchID 1007102 -> stratumID 10071021 and 10071022
```

The patch map raster stores zoneID / patch-family IDs. It does not separately map 01 and 02 aspatial patch members.

DEM / Unity Heightmap Notes

dem30mSBFRbound.tiff is the Central Coast DEM used to shape the Unity terrain. It aligns with Pch30rip90upRN.tiff:

Property	Value
Grid size	396 columns x 301 rows
Pixel scale	~30 m
Bands	1
TIFF sample type	unsigned 16-bit
Compression	none
Source value range observed	0..255
Nodata value observed	none; 65535 count was 0

Because the DEM and patch-family raster share the same 396 x 301 grid, no GIS reprojection, cropping, or alignment step is needed for the Central Coast pre-prototype. Unity terrain heightmaps are square, so the DEM must be resampled and scaled before import.

Generated prototype heightmap:

Assets/Terrains/CentralCoastV2/Heightmaps/CentralCoast_DEM_513_uint16_little_endian.raw

Generation notes:

- Source: RHESSys_Data_Importer/Data/CentralCoast/RHESSysOutput-SingleWarmIdx-6-4-2026/dem30mSBFRbound.tiff
- Output grid: 513 x 513

- Format: headerless RAW, unsigned 16-bit, little-endian / Windows byte order
- Resampling: bilinear
- Scaling: source min..max scaled to 0..65535

Unity Import Raw... settings:

Setting	Value
Depth	Bit16
Width	513
Height	513
Byte order	Windows / Little Endian
Terrain Size X	11880
Terrain Size Z	9030
Terrain Size Y	Start with 1000 or 1500 and tune visually

The source DEM values appear normalized rather than true elevation meters, so Terrain Size Y is a visual scale choice for the prototype. If the terrain appears north/south reversed relative to the patch raster or map, regenerate or import a vertically flipped heightmap.

Cube Locations

The current sample includes five cube/zone IDs.

zoneID	Label	hillID
12166	North facing	320
12771	South facing	330
6492	High elevation	332
18891	Low elevation	336
10071	Riparian	334

Only the riparian cube (zoneID = 10071) differs materially between patch 01 and patch 02 in the current daily cube and stratum files. The other four cube locations have identical patch 01 and patch 02 values in the sample.

Daily Aggregate File

File:

cube_agg_p.csv

Rows:

11,688

Columns:

Column	Notes
day	Calendar day
month	Calendar month
year	Calendar year
basinID	Basin identifier
litter_cs.totalc	Litter carbon total
burn	Daily aggregate burn output. Current sample is all zero.
soil_cs.totalc	Soil carbon total
sat_deficit_z	Saturation deficit depth
evaporation_surf	Surface evaporation
exfiltration_unsat_zone	Unsaturated-zone exfiltration
exfiltration_sat_zone	Saturated-zone exfiltration
evaporation	Evaporation
family_pct_cover	Family percent cover
streamflow	Basin streamflow
rz_storage	Root-zone storage
rain	Rain
transpiration_sat_zone	Saturated-zone transpiration
transpiration_unsat_zone	Unsaturated-zone transpiration
cs.net_psn	Canopy stratum net photosynthesis
epv.height	Effective plant height
cs.leafc	Leaf carbon
cs.leafc_store	Leaf carbon store
cs.live_stemc	Live stem carbon
cs.dead_stemc	Dead stem carbon
cs.frootc	Fine root carbon
cs.live_crootc	Live coarse root carbon
cs.dead_crootc	Dead coarse root carbon
fe.canopy_target_prop_c_consumed	Fire effects canopy carbon consumed
fe.canopy_target_prop_c_remain_adjusted	Fire effects adjusted remaining canopy carbon
fe.canopy_target_prop_c_remain_adjusted_leafc	Fire effects adjusted remaining leaf carbon

Column	Notes
rootzone.depth	Root-zone depth

Daily Cube Patch Files

Files:

cube_p_patch1.csv
cube_p_patch2.csv

Rows per file:

58,440

Each file has five rows per day, one for each cube/zone ID.

Columns:

Column	Notes
day	Calendar day
month	Calendar month
year	Calendar year
basinID	Basin identifier
hillID	Hillslope identifier
zoneID	Patch family / cube identifier
patchID	Aspatial patch member ID ending in 01 or 02
coverfract	Cover fraction
litterc	Litter carbon
burn	Daily patch burn. Current sample is all zero in cube patch files.
soilc	Soil carbon
depthToGW	Depth to groundwater
canopyevap	Canopy evaporation
streamflow	Streamflow
rootdepth	Root depth
groundevap	Ground evaporation
vegAccessWater	Vegetation-accessible water
Qin	Input flow
Qout	Output flow
rain	Rain

Daily Overstory Stratum Files

Files:

cubes_sc_over_patch1.csv
cubes_sc_over_patch2.csv

Rows per file:

58,440

Each file has five rows per day, one for each cube/zone ID.

Columns:

Column	Notes
day	Calendar day
month	Calendar month
year	Calendar year
basinID	Basin identifier
hillID	Hillslope identifier
zoneID	Patch family / cube identifier
patchID	Aspatial patch member ID
stratumID	Overstory stratum ID ending in 1
consumedCOver	Consumed overstory carbon
mortCOver	Overstory mortality carbon
netpsnOver	Overstory net photosynthesis
heightOver	Overstory height
transOver	Overstory transpiration
leafCOver	Overstory leaf carbon
stemCOver	Overstory stem carbon
rootCOver	Overstory root carbon
rootdepthCOver	Overstory root depth
veg_parm_ID	Vegetation parameter ID
laiOver	Overstory leaf area index

In the current sample, overstory veg_parm_ID = 6 appears for riparian patch 01, while overstory veg_parm_ID = 11 appears for other overstory rows.

Daily Understory Stratum Files

Files:

cube_sc_under_patch1.csv
cube_sc_under_patch2.csv

Rows per file:

58,440

Each file has five rows per day, one for each cube/zone ID.

Columns:

Column	Notes
day	Calendar day
month	Calendar month
year	Calendar year
basinID	Basin identifier
hillID	Hillslope identifier
zoneID	Patch family / cube identifier
patchID	Aspatial patch member ID
stratumID	Understory stratum ID ending in 2
consumedCUnder	Consumed understory carbon
mortCUnder	Understory mortality carbon
transUnder	Understory transpiration
netpsnUnder	Understory net photosynthesis
heightUnder	Understory height
leafCUnder	Understory leaf carbon
stemCUnder	Understory stem carbon
rootCUnder	Understory root carbon
rootdepthUnder	Understory root depth
veg_parm_ID	Vegetation parameter ID
laiUnder	Understory leaf area index

In the current sample, understory rows use veg_parm_ID = 2.

Monthly Basin Burn File

File:

bm.csv

Rows:

384

Columns:

Column	Notes
month	Calendar month
year	Calendar year
basinID	Basin identifier
burn	Monthly basin burn value

Current sample burn range:

0.000000 through 5.654753

Monthly Patch Burn File

File:

spatial_data_point_patchvar.csv

Rows:

3,438,336

This equals:

384 months * 8,954 patch IDs

Columns:

Column	Notes
month	Calendar month
year	Calendar year
basinID	Basin identifier
hillID	Hillslope identifier
zoneID	Patch family ID
patchID	Aspatial patch member ID
burn	Monthly patch burn value

Current sample includes:

- 4,477 zoneID values.
- 8,954 patchID values.
- Burn range 0.000000 through 29.174637.

Monthly Stratum Carbon File

File:

spatial_data_point_stratvar.csv

Rows:

6,876,672

This equals:

384 months * 17,908 stratum IDs

Columns:

Column	Notes
month	Calendar month
year	Calendar year
basinID	Basin identifier
hillID	Hillslope identifier
zoneID	Patch family ID
patchID	Aspatial patch member ID
stratumID	Canopy stratum ID
totalc	Total carbon
total_plantc	Total plant carbon

Current sample includes:

- 4,477 zoneID values.
- 8,954 patchID values.
- 17,908 stratumID values.

Stratum Carbon Shape

This file is a rectangular CSV, but it is in **long table** form rather than map or raster form. Each line is one monthly observation for one vegetation layer:

```
month,year,basinID,hillID,zoneID,patchID,stratumID,totalc,total_plantc
7,1987,1,324,3497,349701,3497011,256.597308,247.802126
7,1987,1,324,3497,3497012,0.188243,0.188243
```

Plain-English grain:

```
for this month
for this zoneID / patch-family
for this patchID / aspatial patch member
for this stratumID / vegetation layer
the carbon values are totalc and total_plantc
```

The shape is:

4,477 zoneIDs
* 2 patchIDs per zoneID
= 8,954 patchIDs

8,954 patchIDs
* 2 strata per patchID
= 17,908 stratumIDs

17,908 stratumIDs
* 384 months
= 6,876,672 rows

The spatial footprint is indirect. zoneID links these monthly values back to the patch-family raster (Pch30rip90upRN.tiff), whose pixels store the same zoneID values. The CSV itself is not shaped like x,y,value and is not a Unity terrain/splatmap frame.

Patch Map Raster Source

Pch30rip90upRN.tiff

The patch map is the spatial bridge between RHESys zoneID values and Future Mountain landscape footprints. It is required for Central Coast PatchData and for later precomputed TerrainData generation.

Raster properties:

Property	Value
Grid size	396 columns x 301 rows
Pixel scale	Approximately 30 m
Nodata	65535
Source metadata	GRASS GIS 7.8.3 with GDAL 3.0.4
	• Stores patch-family IDs matching zoneID. • Contains 4,477 valid patch-family IDs. • Aligns with the monthly patch and stratum CSVs. • Should be used to derive patch spatial extents for Central Coast v2.

Initial Import Assumptions

For the initial Central Coast v2 importer path:

- Preserve Big Creek v1 tables and behavior.
- Use a separate Central Coast v2 database or schema.
- Add explicit scenario profile metadata.
- Assign this sample warmingIdx = 0 until true warming/climate-case metadata is provided.
- Preserve source identifiers: zoneID, patchID, and stratumID.

- Do not flatten patch 01 and patch 02 into one record.
- Treat zoneID as the mapped spatial footprint and patchID as an aspatial member within that footprint.
- Load raw/staging tables first before designing API or Unity visualization transformations.

Open Questions

- What scientific label should this sample have: baseline, historical, control, or another climate case?
- What filename or config convention will identify future warming/climate scenario members?
- Will future bundles repeat this exact file set per warming case?
- Are units stable across all files and future scenario members?
- Which burn fields should drive user-visible fire behavior versus provenance/statistics?
- Should monthly patch/stratum landscape data be imported fully for the first phase, or staged separately because of its size?

Initial Patch Mapping

Source: Docs/CentralCoastV2/InitialPatchMapping.md

Last updated: 2026-06-16

Purpose

This document explains the patch-mapping implementation used by the current pre-prototype Central Coast v2 scenario. It replaces the earlier planning note for the now-completed patch-map workflow.

The implementation uses the partial Central Coast dataset at:

RHESSys_Data_Importer/Data/CentralCoast/RHESSysOutput-SingleWarmIdx-6-4-2026/

That sample bundle has one explicit Future Mountain warming index assumption:

- scenarioRunId = single-warming-sample
- warmingIdx = 0

The mapping described here converts the Central Coast patch-family raster into PatchData rows. Those rows are then used by the terrain generator to project monthly StratumData and BurnData values onto the Central Coast landscape grid.

Source Raster

Patch mapping starts with:

Pch30rip90upRN.tiff

The file is configured through ScenarioConfig_CentralCoastV2.json:

```
"patchFamilyRaster": "Pch30rip90upRN.tiff"
```

Observed properties:

Property	Value
Grid size	396 columns x 301 rows
Pixel scale	approximately 30 m
Nodata	65535
Valid patch-family IDs	4,477
Source metadata	GRASS GIS 7.8.3 / GDAL 3.0.4

Each non-nodata pixel stores a zoneID. In Central Coast v2, zoneID is the mapped spatial footprint. The child patchID values, usually ending in 01 or 02, are aspatial members within that footprint.

This is different from Big Creek v1, where patch identifiers were treated more directly as mapped spatial patches.

Import Command

From the importer project folder:

```
cd RHESSys_Data_Importer\RHESSys_Data_Importer
dotnet run -- --auto --config ScenarioConfig_CentralCoastV2.json --patch
```

Dry run:

```
dotnet run -- --auto --config ScenarioConfig_CentralCoastV2.json --dryrun --patch
```

The --patch Central Coast path calls:

```
CentralCoastImporter.ImportPatchMapData(config, dryrun)
```

That method decodes the raster and writes rows through:

```
CentralCoastDAL.AddPatchDataRows(...)
```

For non-Central Coast profiles, --patch continues to use the legacy Big Creek patch import path.

Implementation

ImportPatchMapData reads the TIFF with `BitMiracle.LibTiff.Classic`. The raster is a single-band unsigned 16-bit grid, so the importer reads each scanline, decodes each two-byte value, skips 65535, and groups all remaining pixel coordinates by zoneID.

Algorithm:

1. Resolve patchFamilyRaster from ScenarioConfig_CentralCoastV2.json.
2. Open Pch30rip90upRN.tiff.
3. Read raster width and height.
4. For each row:
 - For each column:
 - Read the 16-bit pixel value.
 - If value == 65535, skip it.
 - Otherwise append [col, row] to that zoneID's pixel list.
5. For each zoneID:
 - Compute pixelCount, centroid, and bounding box.
 - Serialize the footprint as JSON.
 - Add one PatchDataRow to the insert batch.
6. Batch insert all rows into PatchData.

Expected console summary:

```
[PatchData] Decoded 4,477 unique zoneIDs from 396x301 grid (... total non-nodata pixels).
[PatchData] Imported 4,477 of 4,477 source rows.
```

PatchData Rows

The Central Coast PatchData table stores one row per mapped zoneID.

Column	Meaning
scenarioRunId	Scenario member, currently single-warming-sample
importRunId	Import provenance placeholder, currently 0
zoneID	Patch-family id from the raster value
data	JSON footprint payload

The footprint JSON has this shape:

```
{
  "zoneID": 3497,
  "gridWidth": 396,
  "gridHeight": 301,
  "pixelCount": 12,
  "centroidCol": 198.3,
  "centroidRow": 142.7,
  "boundingBox": {
    "colMin": 195,
```

```

    "colMax": 202,
    "rowMin": 139,
    "rowMax": 147
  },
  "pixels": [[195, 139], [196, 140]]
}

```

The `pixels` field is the important bridge. It maps a `zoneID` from RHESSys data back to concrete output grid cells.

Coordinate System

Pixel coordinates are stored as zero-based (`col`, `row`) pairs with the origin at the upper-left corner of the TIFF:

- column increases left-to-right: 0..395
- row increases top-to-bottom: 0..300

For generated Central Coast `TerrainData`, a pixel maps to a flat row-major index:

```
index = row * 396 + col
```

No DEM is read during patch mapping. The DEM file in the same sample folder is used separately for static terrain-height workflow experiments.

Terrain Projection

Patch mapping is static and climate-independent. It does not store vegetation, carbon, snow, burn, or warming values. Instead, it lets the terrain generator project monthly RHESSys values onto the same 396 x 301 grid.

For each (`scenarioRunId`, `warmingIdx`, `year`, `month`):

For each `zoneID` in `PatchData`:

1. Read `StratumData` rows for that `zoneID`.
Aggregate mean `total_plantc` across `patchID` and `stratumID` members.
2. Read `BurnData` rows for that `zoneID` where `level = "patch"`.
Aggregate max burn across `patchID` members.
3. For each [`col`, `row`] in `PatchData.pixels`:
Write the aggregated values into the output `TerrainData` frame.

The current terrain generator is documented in `Docs/CentralCoastV2/InitialTerrainData.md`. It writes one monthly `TerrainData` row per (`scenarioRunId`, `warmingIdx`, `year`, `month`). The `flat_dataList` array uses the same row-major grid indexing described above.

Validation

The current implementation reports the decoded zone count and saved row count. For this sample bundle, both should be 4,477.

Useful validation checks:

- Decoded unique zoneID count equals 4,477.
- PatchData row count for single-warming-sample equals 4,477.
- Total stored pixel count across all rows equals the TIFF non-nodata pixel count.
- No row is written for nodata value 65535.
- All zoneID values used by StratumData exist in PatchData.
- All patch-level BurnData.zoneID values exist in PatchData.

If a zoneID appears in StratumData or patch-level BurnData but not in PatchData, that value cannot be spatially projected into TerrainData.

Current Constraints

- This mapping documents the initial partial dataset only. It should be revalidated if a later Central Coast bundle changes the raster, grid size, scenario member, or warming-case metadata.
- The raster grid is rectangular (396 x 301). Central Coast TerrainData uses explicit gridWidth and gridHeight; it should not be forced into a square Big Creek-style gridSize.
- importRunId is still a placeholder value (0) until structured import-run tracking is wired.
- PatchData stores raster pixel coordinates, not UTM coordinates. UTM/world coordinate conversion can be derived later from raster georeferencing if needed.

Initial Terrain Data

Source: *Docs/CentralCoastV2/InitialTerrainData.md*

Last updated: 2026-06-16

Purpose

This document explains the implemented Central Coast v2 TerrainData generation workflow for the current pre-prototype scenario. It replaces the earlier planning note for the now-completed terrain-frame generator.

The implementation uses the partial Central Coast dataset at:

RHESSys_Data_Importer/Data/CentralCoast/RHESSysOutput-SingleWarmIdx-6-4-2026/

That sample bundle is imported with:

- scenarioRunId = single-warming-sample
- warmingIdx = 0

The terrain generator converts imported PatchData, StratumData, and BurnData into monthly TerrainData frames that can be served through the Future Mountain API and consumed by Unity as Central Coast landscape state.

Inputs

GenerateTerrainData expects the upstream Central Coast import tables to already be populated:

Input	Role
PatchData	Static zoneID footprints from Pch30rip90upRN.tiff
StratumData	Monthly per-stratum carbon values
BurnData	Monthly basin/patch burn values; terrain uses level = "patch"

PatchData is produced by the initial patch mapping workflow documented in:

Docs/CentralCoastV2/InitialPatchMapping.md

For the current sample bundle, the generated time series is monthly and spans 384 frames for one warming case.

Import Command

From the importer project folder:

```
cd RHESSys_Data_Importer\RHESSys_Data_Importer
dotnet run -- --auto --config ScenarioConfig_CentralCoastV2.json --terrain
```

The terrain import should run after patch, burn, and stratum imports:

```
dotnet run -- --auto --config ScenarioConfig_CentralCoastV2.json --patch
dotnet run -- --auto --config ScenarioConfig_CentralCoastV2.json --burn
dotnet run -- --auto --config ScenarioConfig_CentralCoastV2.json --stratum
dotnet run -- --auto --config ScenarioConfig_CentralCoastV2.json --terrain
```

The --terrain Central Coast path calls:

```
CentralCoastImporter.GenerateTerrainData(config, dryrun)
```

Rows are written through:

```
CentralCoastDAL.AddTerrainDataRows(...)
```

Dry-run behavior is intentionally limited. The generator logs that terrain generation is skipped because it depends on database-resident PatchData and StratumData.

Rectangular Grid

The Central Coast patch map raster is rectangular:

Property	Value
Grid width	396
Grid height	301
Total cells	119,196
Pixel grain size	approximately 30 m

Big Creek TerrainData uses a single gridSize field and assumes a square grid. Central Coast rows keep backward compatibility by setting:

Field	Value
gridSize	0
gridWidth	396
gridHeight	301

Central Coast data should not be resampled into a square gridSize payload. Doing so would distort the patch-map footprints.

TerrainData Rows

Each Central Coast TerrainData row represents one monthly frame:

(scenarioRunId, warmingIdx, year, month)

Database shape:

Column	Meaning
scenarioRunId	Scenario member, currently single-warming-sample
warmingIdx	Warming case, currently 0
year	Frame year
month	Frame month
gridSize	0 for Central Coast rectangular frames
gridWidth	396
gridHeight	301
pixelGrainSize	30
decimalPrecision	4
_dataList	JSON flat float array

The model class is:

RHESSys_Data_Importer/RHESSys_Data_Importer/Models/CentralCoast/TerrainDataRow.cs

Data Encoding

_dataList is a row-major JSON float array with length:

`gridWidth * gridHeight = 396 * 301 = 119,196`

Pixel coordinates from PatchData.pixels map into _dataList with:

`index = row * gridWidth + col`

Each value encodes vegetation intensity and burn signal in one float:

`value = vegIntensity + burnSignal * 100`

Where:

- `vegIntensity` is normalized into `[0.0, 1.0]`.
- `burnSignal` is 1 when patch-level burn is positive for the zone/month, otherwise 0.

Unity or API consumers can separate the two pieces as:

`vegIntensity = value % 100`

`burnSignal = floor(value / 100)`

This follows the broad Big Creek convention of encoding terrain visual state in one terrain-frame value while still allowing Central Coast to use explicit rectangular dimensions.

Aggregation

For each monthly frame and each mapped zoneID:

Source	Aggregation
<code>StratumData.total_plantc</code>	Mean across all patchID and stratumID members for the zone/month
<code>BurnData.burn</code>	Max across patch-level rows for the zone/month

Mean `total_plantc` avoids double-counting the two patch members and over/under strata inside a zoneID. Max burn treats burn as a signal rather than an additive quantity.

The implementation computes `globalMaxPlantC` across all imported `StratumData` rows for the active (`scenarioRunId`, `warmingIdx`) and normalizes each zone/month:

`vegIntensity = clamp(meanPlantC / globalMaxPlantC, 0, 1)`

The current code uses the maximum individual `total_plantc` value as `globalMaxPlantC`.

Generator Flow

GenerateTerrainData performs these steps:

1. Load PatchData rows for scenarioRunId.
Convert each [col, row] pixel to a flat index.
2. Compute globalMaxPlantC for scenarioRunId and warmingIdx.
3. Read distinct (year, month) pairs from StratumData.
4. For each month:
 - a. Aggregate mean total_plantc by zoneID.
 - b. Aggregate max patch-level burn by zoneID.
 - c. Allocate a 396 x 301 float array.
 - d. For each zoneID footprint:
 - compute vegIntensity
 - compute burnSignal
 - write value into every pixel index for that zoneID
 - e. Serialize the array as JSON.
 - f. Add one TerrainDataRow to the batch.
5. Batch insert all generated TerrainData rows.

Expected console output includes:

```
[TerrainData] Loaded 4,477 zoneID footprints.  
[TerrainData] globalMaxPlantC = ...  
[TerrainData] 384 monthly frames to generate.  
[TerrainData] Generated 384 TerrainData rows.
```

Static DEM Heightmap

The sample folder also includes:

dem30mSBFRbound.tiff

That DEM is aligned with the patch-family raster and is used separately for the static Unity terrain-height prototype. It is not read by GenerateTerrainData.

Observed DEM properties:

Property	Value
Grid size	396 x 301
Pixel scale	approximately 30 m
Sample type	unsigned 16-bit, single band
Observed value range	0..255
Nodata pixels observed	0 using 65535 as nodata

Prototype RAW output:

Assets/Terrains/CentralCoastV2/Heightmaps/CentralCoast_DEM_513_uint16_little_endian.raw

Generation method:

- Read source TIFF as 396 x 301 unsigned 16-bit values.
- Bilinearly resample to 513 x 513 for Unity terrain import.
- Scale source min..max to 0..65535.
- Write headerless little-endian 16-bit RAW.

Unity Import Raw... settings used for the prototype:

Setting	Value
Depth	Bit16
Width	513
Height	513
Byte order	Windows / Little Endian
Terrain Size X	11880
Terrain Size Z	9030
Terrain Size Y	Start with 1000 or 1500 and tune visually

The DEM values appear normalized rather than raw meter elevations, so terrain height remains a visual tuning parameter for this prototype.

Current Constraints

- This terrain-generation workflow documents the initial partial dataset only. Revalidate it if a later Central Coast bundle changes the raster, date range, scenario member, or warming-case metadata.
- Terrain generation depends on completed PatchData, BurnData, and StratumData imports. It is not idempotent; clear or replace existing rows before rerunning against the same target if duplicate rows matter.
- The burn signal is monthly. Any persistence of burn state across later months is a Unity/runtime visualization decision, not currently encoded here.
- Central Coast API/Unity consumers must use `gridWidth` and `gridHeight` when present, falling back to Big Creek `gridSize` only for legacy square frames.

Central Coast v2 Importer Config Design

Source: *Docs/RHESSysDataImporter/CentralCoastConfig.md*

Last updated: 2026-06-13

Purpose

This document defines the scenario-config design for Central Coast v2 imports, delivered as task CCV2-03. It also records the design decisions agreed while reviewing how Central Coast v2 source files relate to the existing Big Creek v1 data model.

The config file is

RHESSys_Data_Importer/RHESSys_Data_Importer/ScenarioConfig_CentralCoastV2.json.

Config Shape

Central Coast v2 reuses the existing ScenarioConfig object plus a small set of optional fields (all unused by Big Creek v1, which leaves them null):

Field	Type	Purpose
scenarioProfile	string	Must be CentralCoastV2. Selects the data model explicitly.
scenarioRunId	string	Identifies one scenario member/run (e.g. single-warming-sample).
warmingIdx	int	Warming/climate-case index. 0 for the current sample (assumed).
sourceRoot	string	Base folder the files entries resolve against. Keeps the bundle path out of code.
delimiter	string	Field delimiter for source files (, for Central Coast CSVs).
files	object	Logical file role -> file name, resolved relative to sourceRoot.
database	object	Central Coast database connection (separate DB: centralcoast_rhessys).
outputTables	array	Target tables, named to match the existing EF/Big Creek table convention where applicable (Dates, CubeData, PatchData,

Field	Type	Purpose
		WaterData). Monthly burn uses BurnData; Unity spread frames remain FireData. See CCV2-04.

Table Naming

Tables reuse the original Big Creek EF naming convention so the schema and any later API/adaptor stay familiar: Dates, CubeData, PatchData, WaterData. Central Coast-only tables use the same PascalCase style: BurnData, StratumData, and ImportRun.

This is a Central Coast v2 decision only. Big Creek v1 remains untouched, even if an existing MySQL instance displays its tables in lowercase.

ScenarioConfig.GetSourceFilePath(role) joins sourceRoot + the file name for a role and returns a full path, or null if the role is not configured.

Reusing The Shape For Future Members

A future warming/climate member is configured by changing only:

- sourceRoot (new bundle folder),
- scenarioRunId (new member id),
- warmingIdx (real index once metadata exists).

The files role names and file names stay the same, because each member is expected to repeat the same file set. This satisfies the requirement that the sample is configured without hardcoding its folder and that future members reuse the same config shape.

File Roles

Roles are explicit and named, instead of the six fixed Big Creek discovery categories, because the Central Coast file set is split by grain (daily/monthly) and spatial level (aggregate/patch/stratum):

Role	File	Grain
cubeAggregateDaily	cube_agg_p.csv	Daily basin/aggregate
cubePatchDaily01 / cubePatchDaily02	cube_p_patch1.csv / cube_p_patch2.csv	Daily cube patch member
cubeStratumOver01 / cubeStratumOver02	cubes_sc_over_patch1.csv / cubes_sc_over_patch2.csv	Daily overstory stratum
cubeStratumUnder01 / cubeStratumUnder02	cube_sc_under_patch1.csv / cube_sc_under_patch2.csv	Daily understory stratum
basinMonthlyBurn	bm.csv	Monthly basin burn

Role	File	Grain
patchMonthlyBurn	spatial_data_point_patchvar.csv	Monthly all-patch burn
stratumMonthlyCarbon	spatial_data_point_stratvar.csv	Monthly all-stratum carbon
patchFamilyRaster	Pch30rip90upRN.tiff	Patch-family raster

Design Decisions

These decisions explain why the config looks the way it does. They resolve the question of how Central Coast files relate to Big Creek’s six fixed categories (cube, patch, terrain, fire, water, climate).

1. Cube stays one logical table; strata are columns, not separate tables

Big Creek’s cube row already flattens overstory and understory fields into one record. The Central Coast over/understory files are the same data delivered as separate source files. The importer joins them back into one cube row on (day, month, year, patchID) (overstory stratumID ends in 1, understory in 2). A wider cube table is fine and keeps the cube contract stable; Central Coast’s extra stratum fields (consumedC*, mortC*, netpsn*, lai*, veg_parm_ID, root depths) are additive optional columns. Splitting into cube_over / cube_under tables is explicitly rejected.

2. No separate water file; basin streamflow comes from the aggregate cube

The aggregate cube represents the whole watershed, so streamflow in cube_agg_p.csv is the basin streamflow that drives the large-landscape river (the role Big Creek’s WaterData filled). There is therefore no water category. Per-warming streamflow columns become per-member rows as more members arrive.

3. BurnData is monthly source burn; FireData is runtime fire spread

Burn is present spatially and monthly (bm.csv, spatial_data_point_patchvar.csv are non-zero; daily cube burn is zero in the sample). It is not Big Creek’s fire-frame/spread/iter format. Monthly per-patch burn is conceptually the burned-terrain field and will be interpolated to daily at runtime using the same mechanism as snow (see decision 7). Import preserves burn columns now; the config is built so it works once fuller data arrives.

For Central Coast v2, these files import into BurnData via --burn. FireData is reserved for Big Creek-style Unity fire playback frames: instantaneous events with gridLocation, patchId, spread, and iter. Central Coast v2 currently has no configured fire-frame source file, so --fire is a clean no-op for this sample and must not import monthly burn.

4. Patch identity: zoneID is the spatial cube identity

This is the largest model change. Big Creek used a flat patchID -> footprint. Central Coast uses a patch family:

zoneID -> patchID (01/02 aspatial members) -> stratumID (over/under)

Handling:

- zoneID takes over Big Creek's old patchIdx role for spatial identity (the patch-family raster maps zoneID to pixels; the five cube locations are zoneIDs).
- patchID (01/02) is preserved as a member dimension in staging, never flattened. Only the riparian zoneID 10071 differs between 01/02 (oak vs chaparral); the others duplicate, but both rows are kept.
- Cube rows are keyed by (zoneID, patchID). A later adapter can bridge to the legacy "one cube per location" shape.

5. No climate category

Climate was imported just-in-case in Big Creek and never consumed. The Central Coast sample has no climate files. No climate category.

6. Patch-family raster decoded into the existing PatchPointCollection contract

The .tiff patch map is used the same way Big Creek's text patch map was: to place where each cube "comes from" (opening animation) and to drive large- landscape terrain color/texture. Only the decoder is new (GeoTIFF vs text grid); the downstream PatchPointCollection contract (data-grid location, fire-grid location, alphamap location, UTM) is unchanged.

Selecting The Config

The Central Coast config is selected explicitly:

- Via the wizard's "Load another config" option, entering the path to ScenarioConfig_CentralCoastV2.json.
- The resolved profile (CentralCoastV2) is printed at startup and in the wizard.

Big Creek remains the default config loaded by Program.cs, so default behavior is unchanged. Wiring the Central Coast profile into actual import paths and staging tables is handled by CCV2-04 onward.

How To Run

The canonical run instructions live in Docs/RHESSysDataImporter/BuildingAndRunning.md, including the Central Coast v2 wizard flow and dry-run command. See the Central Coast v2 Config section there.

Schema Setup and Migration Workflow

The Central Coast v2 tables live in a dedicated schema: **futuremtn_central_coast**. Do not create or touch these tables in defaultdb (Big Creek).

One-time schema creation (recommended MySQL Workbench path)

The checked-in schema export is:

Database/Schema/CentralCoastV2_schema.sql

Use MySQL Workbench unless there is a specific reason to use the CLI:

1. Connect to the target MySQL server.
2. In the Schemas panel, choose **Create Schema**.
3. Name the schema exactly futuremtn_central_coast.
4. Click **Apply**.
5. Open Database/Schema/CentralCoastV2_schema.sql.
6. Make futuremtn_central_coast the active/default schema, or add this line near the top of the SQL editor:

USE futuremtn_central_coast;

7. Run the script.
8. Verify tables were created:

SHOW **TABLES**;

Expected tables include burndata, cubedata, dates, importrun, patchdata, stratumdata, terrairdata, waterdata, and __efmigrationshistory.

The schema export does not create the database and does not contain a USE statement, so the schema must exist and must be selected before running it.

EF Core migration alternative

The EF migration artifacts also exist under:

RHESSys_Data_Importer/RHESSys_Data_Importer/Migrations/CentralCoast/

However, dotnet ef database update --context CentralCoastDbContext uses the design-time connection in DAL/CentralCoastDbContextFactory.cs, which is local development oriented. For production/staging servers, prefer the exported SQL file above unless the design-time connection has been intentionally updated for the target server.

Import sequence (after schema exists)

1. Dry run -- validate config, file paths, column headers

dotnet run -- --auto --config ScenarioConfig_CentralCoastV2.json --dryrun

2. Patch map (spatial footprints -- fast, ~4,477 rows)

dotnet run -- --patch --config ScenarioConfig_CentralCoastV2.json

3. Burn (basin + patch monthly burn)

```
dotnet run -- --burn --config ScenarioConfig_CentralCoastV2.json
```

4. Stratum (6.9M rows -- chunked 10k, takes minutes)

```
dotnet run -- --stratum --config ScenarioConfig_CentralCoastV2.json
```

5. Terrain (reads PatchData + StratumData + BurnData -- must be Last)

```
dotnet run -- --terrain --config ScenarioConfig_CentralCoastV2.json
```

6. Full import (all of the above in dependency order)

```
dotnet run -- --auto --config ScenarioConfig_CentralCoastV2.json
```

All commands run from the RHESSys_Data_Importer/RHESSys_Data_Importer directory.

Adding future migrations

After any model change to a CCV2 entity:

```
dotnet ef migrations add <MigrationName> --context CentralCoastDbContext --ou  
tput-dir Migrations/CentralCoast  
dotnet ef database update --context CentralCoastDbContext
```

Big Creek migrations live in Migrations/ (root) and use CubeDataDbContext. Central Coast migrations live in Migrations/CentralCoast/ and use CentralCoastDbContext. Never mix them.

Known Follow-ups

- The current ColumnMapper splits on whitespace; Central Coast CSVs are comma-delimited. The delimiter config field is provided for this, and the CSV-aware parsing path is implemented with the model classes in CCV2-05.
- Detailed source-column to model-field mappings are defined alongside the model classes in CCV2-05, using the column lists in Docs/CentralCoastV2/DataFormats.md.

Central Coast v2 Database Schema Design

Source: Docs/RHESSysDataImporter/CentralCoastSchema.md

Last updated: 2026-06-13

Purpose

This document designs the Central Coast v2 import (staging) schema, delivered as task CCV2-04. It defines the tables that the Central Coast importer writes to, their columns, keys, and which source file/grain feeds each one.

It builds on the config design in `CentralCoastConfig.md` and the source-file inventory in `Docs/CentralCoastV2/DataFormats.md`.

Ground Rules

- **Separate database.** Central Coast tables live in their own database (`centralcoast_rhessys`, parallel to `bigcreek_rhessys`). Nothing here touches the Big Creek v1 database or its `CubeData`.
- **Original EF naming style.** Tables are unprefixed and use the existing Big Creek EF/PascalCase convention (`CubeData`, `WaterData`, `PatchData`, `Dates`). New tables that have no Big Creek equivalent reuse the same style (`BurnData`, `StratumData`, `ImportRun`). This applies to the new Central Coast database only; Big Creek v1 remains untouched, including any lowercase table names shown by the current MySQL server.
- **Provider/conventions match the importer.** MySQL via Pomelo, `utf8mb4`, integer identity `id` primary keys, float numeric columns, `dateIdx/` `warmingIdx` integer keys, exactly like the existing Big Creek models.
- **Multi-member ready.** Every data table carries `scenarioRunId` and `warmingIdx` so multiple scenario members can coexist in one database.
- **Provenance.** Every data table carries `importRunId` (FK to `ImportRun`) and, where a single file feeds the table, `sourceFile`.

Scale Split

Central Coast data separates cleanly into two scales, mirroring Big Creek's cube vs. large-landscape split:

- **Cube scale** (5 detailed cube locations, daily): `CubeData` + `WaterData`.
- **Landscape scale** (all ~8,954 patches / ~17,908 strata, monthly): `BurnData` (monthly burn) + `StratumData` (carbon) + `PatchData` (spatial extents).

This is why `CubeData` stays small/clean and is not overloaded with whole-landscape monthly data.

Raw Import vs Precomputed TerrainData

The tables in this document are raw imported RHESys/source tables. They are not yet the Unity-facing large-landscape payload.

For Big Creek v1, `TerrainData` is a precomputed monthly large-landscape splatmap/texture frame that Unity can load through the API and interpolate over time. Central Coast v2 should keep that same concept and name, but its `TerrainData` should be generated after import from Central Coast source tables and spatial assets:

`PatchData` geometry
+ `StratumData` monthly vegetation/carbon
+ `BurnData` monthly burn

+ scenario/warming metadata
 = precomputed Central Coast TerrainData

That TerrainData generation is a separate post-import/API-data task. It is not the same thing as importing spatial_data_point_stratvar.csv.

Table Overview

Table	Grain	Source file(s)	Notes
Dates	Daily	derived from daily files	Date dimension; dateIdx = id.
CubeData	Daily, per cube	cube_p_patch1/2, cubes_sc_over_patch1/2, cube_sc_under_patch1/2	Patch + overstory + understory merged into one row.
WaterData	Daily, basin	cube_agg_p	Aggregate/whole-watershed daily (streamflow, precip, basin summaries).
BurnData	Monthly, basin + patch	bm, spatial_data_point_patchvar	RHESSys burn state; level discriminates basin vs patch. This is not Unity fire-spread frame data.
StratumData	Monthly, per stratum	spatial_data_point_stratvar	Whole-landscape stratum carbon over time.
PatchData	Static, per patch family	Pch30rip90upRN.tif	Spatial extents (PatchPointCollection contract).
ImportRun	per run	n/a	Provenance / batch marker.

Decision: where the daily aggregate goes

The daily aggregate file cube_agg_p.csv is whole-watershed, not per-cube. Per config decision 2, its streamflow is the basin streamflow that drives the large landscape river “the role Big Creek’s WaterData filled. So it maps to WaterData, not to CubeData. This keeps CubeData purely per-cube (and honors “does not overload CubeData”) while preserving the aggregate’s richer basin columns in WaterData.

This means Central Coast WaterData differs from Big Creek WaterData: Big Creek encoded warming as columns (qBase, qWarm6); Central Coast uses one row per (dateIdx,

warmingIdx, scenarioRunId) with a single streamflow, plus basin summary columns. This is the normalized form recommended in Specs/DataModel.md.

Common Columns

Unless noted, every data table includes:

Column	Type	Meaning
id	int identity PK	Surrogate key.
importRunId	int (FK ImportRun.id)	Which import run wrote the row.
scenarioRunId	varchar	Scenario member id (e.g. single-warming-sample).
warmingIdx	int	Warming/climate-case index (0 for current sample).
sourceFile	varchar (nullable)	Source file name, where a single file feeds the table.

Dates

Date dimension built from the daily files. Same shape as Big Creek Dates; dateIdx used by daily tables equals Dates.id.

Column	Type	Source
id	int identity PK	â€”
date	datetime	from year/month/day
year	int	year
month	int	month
day	int	day

Unique index on (year, month, day).

CubeData

Daily per-cube rows for the 5 cube locations Ã— 2 patch members (01/02). Built by joining the patch hydrology file with the overstory and understory stratum files on (year, month, day, zoneID, patchID). Overstory stratumID ends in 1, understory in 2.

Keys / identity:

Column	Type	Source
common columns		
dateIdx	int (FK Dates.id)	from date

Column	Type	Source
basinID	int	basinID
hillID	int	hillID
zoneID	int	zoneID (cube identity)
patchID	bigint	patchID (â€˜01/â€˜02)

Patch hydrology (from cube_p_patch*):

coverfract, litterc, burn, soilc, depthToGW, canopyevap, streamflow, rootdepth, groundevap, vegAccessWater, Qin, Qout, rain â€” all float.

Overstory stratum (from cubes_sc_over_patch*):

stratumIDOver (bigint), consumedCOver, mortCOver, netpsnOver, heightOver, transOver, leafCOver, stemCOver, rootCOver, rootdepthCOver, laiOver (float), vegParmIDOver (int).

Understory stratum (from cube_sc_under_patch*):

stratumIDUnder (bigint), consumedCUnder, mortCUnder, transUnder, netpsnUnder, heightUnder, leafCUnder, stemCUnder, rootCUnder, rootdepthUnder, laiUnder (float), vegParmIDUnder (int).

Index on (scenarioRunId, warmingIdx, dateIdx, zoneID, patchID).

This is a superset of the Big Creek CubeData columns (heightOver, transOver, leafCOver, â€˜, transUnder, â€˜ are all present), so a later adapter can project Central Coast CubeData onto the Big Creek cube contract.

WaterData

Daily basin/aggregate rows from cube_agg_p.csv.

Group	Columns (all float unless noted)
keys	common columns, dateIdx (FK Dates.id), basinID (int)
hydrology	streamflow, rain, evaporation, evaporation_surf, exfiltration_unsat_zone, exfiltration_sat_zone, transpiration_sat_zone, transpiration_unsat_zone, sat_deficit_z, rz_storage, rootzone_depth, family_pct_cover
burn	burn
carbon	litter_cs_totalc, soil_cs_totalc,

Group	Columns (all float unless noted)
fire effects	cs_net_psn, epv_height, cs_leafc, cs_leafc_store, cs_live_stemc, cs_dead_stemc, cs_frootc, cs_live_crootc, cs_dead_crootc fe_canopy_target_prop_c_consumed, fe_canopy_target_prop_c_remain_adjusted, fe_canopy_target_prop_c_remain_adjusted_leafc

Index on (scenarioRunId, warmingIdx, dateIdx).

Column names map the source headers' dots to underscores (e.g. cs.net_psn -> cs_net_psn).

BurnData

Monthly burn at basin and patch level, combining bm.csv and spatial_data_point_patchvar.csv.

Column	Type	Source
common columns		
year	int	year
month	int	month
level	varchar(basin/patch)	which file produced the row
basinID	int	basinID
hillID	int (nullable)	hillID (patch only)
zoneID	int (nullable)	zoneID (patch only)
patchID	bigint (nullable)	patchID (patch only)
burn	float	burn

Index on (scenarioRunId, warmingIdx, year, month, zoneID, patchID).

level = 'basin' rows leave hillID/zoneID/patchID null. This is monthly data; runtime interpolates to daily using the existing snow/terrain interpolation (config decision 7). No new runtime mechanism.

FireData is reserved for Big Creek-style Unity fire playback frames: instantaneous fire events with gridLocation, patchId, spread, and iter inside _dataList. The current Central Coast v2 sample does not include that fire-frame source data, so monthly RHESys burn must not be imported into FireData.

StratumData

Monthly whole-landscape stratum carbon from `spatial_data_point_stratvar.csv` (~6.9M rows per member). Kept separate from CubeData because it spans every stratum in the watershed, not just the 5 cubes.

This table is long-format RHESys source data, not a map grid. One row means:

for this month + zoneID + patchID + stratumID,
the carbon values are `totalc` and `total_plantc`

The current sample has 6,876,672 rows:

17,908 stratumIDs * 384 months

The zoneID column links these rows back to the patch-family raster; the CSV itself does not contain pixel coordinates or Unity texture weights.

Column	Type	Source
common columns		
year	int	year
month	int	month
basinID	int	basinID
hillID	int	hillID
zoneID	int	zoneID
patchID	bigint	patchID
stratumID	bigint	stratumID
totalc	float	totalc
total_plantc	float	total_plantc

Index on (scenarioRunId, warmingIdx, year, month, stratumID).

PatchData

Static per-patch-family spatial extents, decoded from `Pch30rip90upRN.tiff`. This reuses Big Creek's PatchData role: the PatchPointCollection contract used for the opening "where each cube comes from" animation and large-landscape terrain color/texture (config decision 8). Only the decoder is new (GeoTIFF vs text grid).

Column	Type	Meaning
common columns (no warmingIdx; spatial is climate-independent)		
zoneID	int	Patch-family id (raster

Column	Type	Meaning
		value).
data	longtext (JSON)	Serialized PatchPointCollection: data-grid location, fire-grid location, alphamap location, UTM, pixel members.

Index on (scenarioRunId, zoneID).

Decoding the raster into this contract is a later task; the table shape is fixed here.

ImportRun

One row per import execution; the batch marker referenced by importRunId.

Column	Type	Meaning
id	int identity PK	â€”
scenarioName	varchar	From config.
scenarioProfile	varchar	CentralCoastV2.
scenarioRunId	varchar	Member id.
warmingIdx	int	Member warming index.
databaseName	varchar	Target database.
sourceRoot	varchar	Resolved source bundle path.
startedUtc	datetime	Run start.
finishedUtc	datetime (nullable)	Run end.
status	varchar	e.g. running, succeeded, failed, dryrun.
filesImported	int	Count of files processed.
rowsImported	bigint	Total rows written.
notes	longtext (nullable)	Warnings/summary.

Acceptance Mapping

- **Preserves raw Central Coast structure:** each source file’s columns are preserved (cube files merged only along their shared key; aggregate, burn, stratum carbon, and rasters each retain their native columns).
- **Does not overload Big Creek v1 CubeData:** Central Coast uses its own database and its own CubeData; whole-landscape monthly data lives in separate tables (BurnData, StratumData).

- **Stores multiple future members:** scenarioRunId + warmingIdx on every data table, plus ImportRun provenance, allow many members in one database.

Follow-ups

- Model classes for these tables: CCV2-05.
- EF migrations generated from those models (authoritative DDL): after CCV2-05.
- Raster decoder for PatchData: implemented by the initial patch mapping workflow documented in Docs/CentralCoastV2/InitialPatchMapping.md.

Future Roadmap

Adding Future Scenarios

Source: Docs/AddingFutureScenarios.md

Last updated: 2026-06-16

Purpose

This guide describes the Unity-side work for adding a new Future Mountain scenario. It complements the RHESSys importer guide:

Docs/RHESSysDataImporter/ScenarioUpgradeGuide.md

The importer guide explains how to bring new RHESSys data into the database. This guide focuses on the Unity editor side: scenes, terrain assets, runtime settings, API profile selection, visual tuning, and asset isolation.

Current Scenario Pattern

Current scenario scenes live under:

Assets/Scenes/BigCreekV1/BigCreekV1.unity
Assets/Scenes/CentralCoastV2/CentralCoastV2.unity

Each scenario should have its own scene. A new scenario usually starts by duplicating the closest existing scenario scene, then replacing or tuning the scenario-specific objects and settings.

Do not turn the existing Big Creek scene into a new scenario in place. Big Creek is still the compatibility baseline.

High-Level Workflow

1. Create the importer/database/API path for the scenario, or confirm that one already exists.
2. Duplicate the closest Unity scene into a new scenario folder.

3. Duplicate scenario-owned terrain assets for the large landscape, cubes, and side cubes.
4. Duplicate or intentionally share prefabs, materials, terrain layers, and message assets.
5. Configure the scene's Settings GameObject and SimulationSettings.
6. Update scenario API routing and message resources.
7. Tune vegetation, carbon, snow, fire, water, terrain, and camera framing.
8. Run editor and web-build smoke tests.

Scene Duplication

Create a new scene folder under:

Assets/Scenes/<ScenarioName>/

Duplicate the closest existing scene into that folder and rename it clearly. After duplication, open the new scene and verify:

- The Settings GameObject exists and has SimulationSettings.
- GameController, LandscapeController, camera, canvases, timeline, message managers, cube objects, side cube objects, and aggregate cube references are assigned.
- The scene appears in build settings when it is ready for build testing.

Terrain Duplication

The scene contains more terrain-like assets than just the large landscape. When adding a scenario, duplicate and reassign terrain assets for:

- The large landscape terrain.
- Each visible sample cube.
- The aggregate cube.
- Each side cube used by Side-by-Side Mode.
- The aggregate side cube.

This matters because Unity TerrainData assets are shared assets. If two scenes reference the same terrain asset and one scene's terrain is edited, the other scene can change too. That kind of cross-scenario asset bleed is hard to notice until much later.

After duplication, inspect each terrain component and confirm it points to the new scenario-specific TerrainData asset.

Prefabs, Materials, And Shared Assets

Be deliberate about what is shared between scenarios.

Shared prefabs and materials are useful when the behavior and look should remain identical. They are dangerous when a new scenario needs region-specific tuning. If a shared prefab, material, terrain layer, or texture is edited for the new scenario, the change can bleed into older scenarios that reference the same asset.

Before editing any shared asset, decide whether to:

- Keep it shared because the change is global.
- Duplicate it into a scenario-specific folder because the change is local.

Assets that commonly need scenario-specific copies:

- Terrain materials and terrain layers.
- Large landscape terrain assets.
- Cube and side-cube terrain assets.
- Tree, shrub, roots, dead-tree, grass, and litter prefabs.
- Fire materials, fire terrain texture settings, and fire prefabs.
- Soil, groundwater, snow, and water materials.
- Message text assets.
- Camera animation controller or clips if framing changes.

Use names that make ownership obvious, such as `CentralCoastV2_...`

SimulationSettings

Each scenario scene should have its own configured `Settings GameObject` with `SimulationSettings`. The component is scene-local, so values can differ by scenario, but only if the scene has been duplicated and checked carefully.

Important fields to review:

Area	Fields
API	<code>apiProfile</code>
Scenario features	<code>SnowEnabled</code> , <code>FireEnabled</code>
Fire	<code>AutoPauseOnFire</code> , <code>MinFireFrameLength</code> , <code>MaxFireLengthInSec</code> , <code>ImmediateFireTimeThreshold</code>
Population	<code>MinFrontTrees</code> , <code>MaxTrees</code> , <code>MaxShrubs</code> , <code>WebBuildMaxVegMultiplier</code>
Carbon	<code>TreeCarbonFactor</code> , <code>RootsCarbonFactor</code> , <code>ShrubCarbonFactor</code> , <code>CubeATreeCarbonFactor</code> , <code>CubeARootsCarbonFactor</code> , <code>CubeAShrubCarbonFactor</code> ,

Area	Fields
Distribution	WebBuildCarbonMultiplier tree spacing, cube padding, shrub zone settings
Geometry	tree/shrub height and width ranges
Roots	root size, spread, offset, and variability settings
Side-by-Side	SideBySideModeXOffset, SideBySideModeXOffsetAggregate
UI	message timing

Questions to answer for every new scenario:

- Should snow visualization be enabled?
- Should fire simulation and fire data loading be enabled?
- Are the incoming carbon values on the same scale as Big Creek?
- Do tree, shrub, and root carbon factors need retuning?
- Is WebBuildCarbonMultiplier appropriate for WebGL density and performance?
- Is WebBuildMaxVegMultiplier aggressive enough for the scenario's vegetation?
- Do side-by-side cube offsets still frame correctly after terrain/cube changes?

OptimizeForWeb() runs in web builds when the WEB_VERSION scripting symbol is defined. It reduces vegetation counts and multiplies carbon factors. Tune the base carbon settings and web multipliers together, not independently.

API Profile And Runtime Data

SimulationSettings.apiProfile controls the API prefix used by WebManager.

Current profiles:

Profile	Route behavior
BigCreek	Uses legacy /api/... routes
CentralCoast	Uses /api/centralcoast/... routes

If a new scenario has its own API routes, add a new ScenarioApiProfile value and update SimulationSettings.ScenarioApiPrefix, message resource paths, and any profile-specific runtime behavior.

The Unity scene should not load importer configuration directly. The importer, database, API, and Unity DTOs still need to agree on fields, names, units, and defaults.

Messages And Narrative Content

Scenario-specific messages should live in separate Resources assets rather than editing Big Creek messages in place.

SimulationSettings currently chooses message resource paths by API profile:

- Big Creek: Messages/GeneralMessages, Messages/FireMessages
- Central Coast: Messages/CentralCoastGeneralMessages, Messages/CentralCoastFireMessages

For a new scenario, add new message assets and route them through the profile selection logic.

Visual Tuning Checklist

After data loads, compare model values to the visual result:

- Tree and shrub density.
- Root depth and width.
- Soil and groundwater material response.
- Snow accumulation and snow-to-groundwater effects.
- Fire ignition, spread, burn texture, and regrowth.
- Landscape terrain splatmap behavior.
- Cube labels and message highlighting.
- Timeline range, precipitation scaling, fire icons, and message icons.
- Camera animation framing for normal zoom and Side-by-Side Mode.

Use the model data layer when available to compare source carbon values against visible scene-derived vegetation values.

Data Contract Checklist

For each new scenario data field:

- Identify the source file and source column name.
- Define database table and column type.
- Define API JSON field name.
- Define Unity DTO field or runtime mapping.
- Define units and expected range.
- Decide whether it drives a visual change, UI display, message condition, or only metadata.
- Define default behavior when the field is missing.

Common Risks

- Shared materials or prefabs are edited for a new scenario and accidentally change Big Creek or another older scenario.
- A scene is duplicated but its large landscape or cube terrains still reference old TerrainData assets.
- Side cubes are forgotten because they are hidden until Side-by-Side Mode.
- `SimulationSettings.apiProfile` points at the wrong API route family.
- Carbon factors are tuned in the Editor but web multipliers make the WebGL build look sparse or overloaded.
- Snow or fire is enabled even though the scenario has no meaningful source data for it.
- Message resources still point at Big Creek text.
- Camera animation clips still frame the old cube/landscape positions.

Smoke Tests

Before calling a new scenario ready:

- Open the scene directly in the Unity Editor.
- Confirm there are no missing scripts, missing materials, or missing terrain references.
- Start the simulation and verify data loads from the intended API profile.
- Toggle snow/fire-relevant controls if enabled.
- Enter and exit Side-by-Side Mode for the aggregate cube and every sample cube.
- Check the timeline, messages, model data layer, and camera reset.
- Build or run the web profile and verify vegetation density after `OptimizeForWeb()`.

RHESSys Data Importer Roadmap

Source: *Specs/RHESSysDataImporter/Roadmap.md*

Last updated: 2026-06-16

Purpose

This roadmap records the remaining work and operational risks for the embedded RHESSys Data Importer. It is intentionally practical: it tracks what should be improved next now that the Central Coast v2 pre-prototype import path is in place.

The current importer supports Big Creek v1 legacy import paths and Central Coast v2 profile-specific paths. Central Coast v2 is far enough along to import the partial single-warming sample bundle, generate patch mapping, generate monthly terrain frames, and serve prototype API data. The remaining work is mostly about repeatability, production safety, reporting, and cleanup.

Current Baseline

- Big Creek v1 and Central Coast v2 data share the top-level RHESSys_Data_Importer/Data/ folder.
- Central Coast v2 uses a dedicated schema, futuremtn_central_coast, not defaultdb.
- A checked-in Central Coast schema export exists at Database/Schema/CentralCoastV2_schema.sql.
- ScenarioConfig_CentralCoastV2.json points at the partial sample bundle: RHESSysOutput-SingleWarmIdx-6-4-2026.
- Central Coast v2 dates are derived from the configured daily aggregate CSV (cubeAggregateDaily) rather than manually configured start/end dates.
- --dates exists, and full Central Coast auto import includes dates.
- Central Coast v2 patch mapping is implemented from patchFamilyRaster (Pch30rip90upRN.tiff) into PatchData.
- Central Coast v2 monthly TerrainData generation is implemented from PatchData, StratumData, and BurnData.
- CentralCoastDbContext enables MySQL retry resiliency and uses a longer command timeout for large Central Coast reads/writes.
- StratumData batch insert reports actual saved rows and recursively isolates bad rows so a single failed row does not drop an entire large batch.
- Wizard mode has Central Coast v2 coverage for dates, cube, water, burn, patch, stratum, and terrain, though auto mode remains the better-tested production path.

Near-Term Priorities

Import Resume And Idempotency

Central Coast category imports are not generally idempotent. If a production import fails midway, rerunning the same category may append duplicate rows unless the target table is cleared first.

Current manual recovery example for failed StratumData plus dependent terrain:

USE futuremtn_central_coast;

SET FOREIGN_KEY_CHECKS = 0;

TRUNCATE TABLE terraindata;

TRUNCATE TABLE stratumdata;

SET FOREIGN_KEY_CHECKS = 1;

Then rerun:

```
dotnet run -- --auto --config ScenarioConfig_CentralCoastV2.json --stratum
```

```
dotnet run -- --auto --config ScenarioConfig_CentralCoastV2.json --terrain
```

Planned improvements:

- Add explicit replace/resume modes.
- Add duplicate detection or uniqueness constraints for scenario keys.
- Print clear recovery guidance when a table must be truncated before retrying.
- Prefer import-run based cleanup once ImportRun records are actively used.

Hard-Fail On Incomplete Writes

Several DAL methods still catch exceptions and return `0`. Some import paths report saved-vs-source row counts, but orchestration should fail the process when required rows are missing, especially before dependent categories such as terrain generation.

Planned improvements:

- Throw or return a hard failure when saved $\neq$ expected.
- Prevent full import from continuing to dependent categories after incomplete upstream data.
- Apply the same standard to dates, cube, water, burn, patch, stratum, terrain, and fire-frame imports where appropriate.
- Keep the helpful bad-row isolation behavior for StratumData, but make final completeness status unambiguous.

Structured Import Reporting

The ImportRun model and table exist, but current Central Coast writers still set `importRunId = 0`; the table is not yet acting as the real batch/provenance record.

Planned improvements:

- Create an ImportRun row at the start of each import execution.
- Set status to running, succeeded, failed, or dryrun.
- Store per-category source rows read, rows saved, rows skipped/dropped, and elapsed time.
- Store target database/schema, scenario metadata, source root, and useful failure details.
- Write generated row counts into the final console summary.

Production Preflight Validation

The importer validates Central Coast source files, but target database validation is still limited.

Planned improvements:

- Confirm target schema exists.
- Confirm expected tables and columns exist.
- Confirm expected table row counts are zero or intentionally replaceable.
- Print target database/schema in the final confirmation.

- Optionally require an explicit production acknowledgement flag.

Medium-Term Improvements

Scenario Config Validation

No formal JSON schema exists for scenario config validation.

Planned improvements:

- Validate required fields by scenarioProfile.
- Reject unknown file roles when profile-specific importers require fixed roles.
- Prevent credentials from being committed in shared configs.
- Validate database name/host before import begins.
- Keep the human-readable validation report, but make profile-specific requirements machine-checkable.

Wizard Mode Smoke Testing

Wizard mode has Central Coast v2 coverage, but auto mode remains the primary tested path for production-scale imports.

Planned improvements:

- Run one end-to-end interactive Central Coast v2 wizard smoke test.
- Keep wizard prompts aligned with auto-mode import order.
- Make wizard recovery guidance match the future replace/resume behavior.

Bulk Insert Performance

Central Coast imports use chunked EF inserts. This worked locally but remains slow and sensitive to network/database interruptions at production scale.

Planned improvements:

- Evaluate MySQL bulk load or more efficient batched insert strategies for large tables.
- Reduce per-row EF overhead for StratumData, BurnData, and other large categories.
- Keep enough logging to diagnose bad rows without flooding the console.

Terrain Generation Performance

The command timeout was increased, but terrain generation still performs large aggregate scans over StratumData and BurnData.

Planned improvements:

- Confirm indexes support (scenarioRunId, warmingIdx, year, month, zoneID).
- Consider pre-aggregating monthly zone plant carbon.
- Fail cleanly if required upstream row counts are incomplete.

- Consider replacing per-month EF aggregate queries with a streaming or precomputed intermediate table if later datasets grow.

Cleanup And Lower-Priority Work

- The project builds with many nullable warnings.
- Some comments and package references still reflect older SQL Server history.
- `TextFileInput.cs` remains large and mixes parsing, conversion, and import orchestration.
- Several legacy Big Creek paths still rely on positional parsing and filename conventions.
- Climate import is recognized but not implemented.
- `--force` is recognized but does not have a well-defined safety contract beyond bypassing Central Coast validation failure.
- The create-database wizard option is simulated; it shows setup guidance rather than creating a MySQL schema.
- Automated tests are still missing for parser behavior, config loading, calendar/date derivation, database write failure behavior, and generated terrain-frame contracts.

Completed Or Improved

- README no longer describes the importer as MSSQL-only.
- Central Coast and Big Creek data folders were unified under `RHESSys_Data_Importer/Data/`.
- Central Coast v2 schema setup now documents the MySQL Workbench plus checked-in SQL export workflow.
- Visual Studio 2022+ requirement is documented for loading the importer solution.
- Central Coast v2 date table population no longer depends on Big Creek's legacy `hist` aggregate text-file path.
- Transient MySQL retry resiliency and longer command timeout were added to `CentralCoastDbContext`.
- Patch mapping and terrain generation docs were converted from completed planning notes into current implementation notes.

When To Revisit This Roadmap

Revisit after each production-style Central Coast import run, after any new scenario profile is added, or when importer failures require manual database recovery. Retire or split this roadmap once the remaining operational items are tracked in the project issue/task system.

Appendix / Miscellaneous

Technical Documentation Process

Source: Docs/TechnicalDocumentationProcess.md

Last updated: 2026-06-16

Purpose

Future Mountain keeps technical documentation in both Docs/ and Specs/. This separation is intentionally lightweight:

- Docs/ contains general, operational, onboarding, scenario, service, and higher-level process documentation.
- Specs/ contains feature-specific or subsystem-specific technical notes that are useful when implementing, reviewing, or extending a defined part of the project.

This structure is meant to support both human readers and agentic workflows. General documentation helps a new contributor understand the project and its scenarios. Specs give implementation agents and reviewers smaller, focused documents they can use as context without needing to load the whole project history.

Handbook Manifest

The generated handbook order is controlled by the repo-root file:

`handbook-manifest.txt`

This file does not move or rename documentation. It only defines handbook chapters and the order in which existing Markdown files should be assembled. The handbook is simply an organized single source for the project's docs and specs, built from the same canonical Markdown files already maintained in the repo.

Manifest conventions:

- Lines starting with # define handbook chapters.
- Lines starting with ## define handbook sub-chapters.
- Non-empty non-heading lines are relative paths to Markdown files.
- Blank lines are allowed for readability.
- Lines starting with // are comments for future build tooling.

Each Markdown file under Docs/ and Specs/ should appear in the manifest exactly once unless the handbook process is intentionally changed later.

Building The Draft Handbook

Run the handbook build script from the repo root:

`.\Build-Handbook.ps1`

By default, the script reads:

`handbook-manifest.txt`

and writes:

`FutureMountain_Handbook.md`

The generated Markdown handbook is a draft assembly of the documentation in the manifest order. The source documents remain the canonical files to edit. Ideally, the handbook should be rebuilt, exported, and refreshed after each major Future Mountain update so the shareable version stays aligned with the current project state.

Final Handbook Output

Pandoc is required to create final converted versions of the handbook, such as a PDF or Word document. The current script creates the assembled Markdown file; the final Pandoc command/process may be updated after the first full draft pass.

Until that process is finalized, install Pandoc before attempting final handbook export, and treat `FutureMountain_Handbook.md` as the intermediate build output.

RHESSys Data Importer Repository Strategy

Source: Docs/RHESSysDataImporter/RepositoryStrategy.md

Last updated: 2026-06-13

Purpose

This document records the repository/importer cleanup decision tracked as task CCV2-01 in `Tasks/CentralCoastV2_Importer_TaskGraph.md`.

It defines how the embedded RHESSys Data Importer lives inside the Future Mountain repository so the importer can be committed, cloned, and built without nested Git confusion.

Decision

The importer is **fully absorbed into the Future Mountain repository as source**.

It is not a Git submodule and does not keep its own nested `.git` directory.

Rationale:

- The importer is small, project-specific tooling, not an independently versioned library.

- A submodule would add clone/checkout complexity for a tool that is only used alongside this repo's data and docs.
- Absorbing as source keeps importer code, docs, specs, and sample data history together.

Current state confirms this decision is already in effect:

- RHESSys_Data_Importer/ has no nested .git directory.
- Its source files are tracked by the parent repository.

Project And Build Files

The importer is a real .NET 8 solution, so its solution/project files are source and must be tracked:

```
RHESSys_Data_Importer/RHESSys_Data_Importer.sln
RHESSys_Data_Importer/RHESSys_Data_Importer/RHESSys_Data_Importer.csproj
```

Problem

The repository root .gitignore uses the standard Unity ignore pattern:

```
*.csproj
*.unityproj
*.sln
```

That is correct for Unity-generated files such as Assembly-CSharp.csproj and FutureMountain.sln, which are machine-generated and vary per workstation. But it is too broad for the embedded importer, whose .sln/.csproj are real, hand-maintained source files. Before this fix they were silently untracked, so a fresh clone could not build the importer.

Fix

Keep ignoring Unity-generated project files, then explicitly un-ignore the importer's project files using narrow negation patterns in the root .gitignore:

```
## Embedded RHESSys Data Importer is a real .NET tool absorbed as source.
## Keep its solution/project files tracked despite the broad Unity rules above.
!RHESSys_Data_Importer/RHESSys_Data_Importer.sln
!RHESSys_Data_Importer/RHESSys_Data_Importer/RHESSys_Data_Importer.csproj
```

The narrow form is used (rather than RHESSys_Data_Importer/**/*.*.csproj) because only this single importer solution is expected. If more importer projects are added later, add matching narrow negations or switch to the broad form.

Generated Files

The importer's build output must never be committed. The root `.gitignore` now explicitly ignores it:

```
RHESSys_Data_Importer/**/[Bb]in/  
RHESSys_Data_Importer/**/[Oo]bj/  
RHESSys_Data_Importer/**/.vs/
```

This keeps generated local files (compiler output, NuGet/EF intermediate output, Visual Studio cache) out of source control while preserving the project files.

Large Source Data

The Central Coast v2 sample bundle is large (the embedded data totals hundreds of MB, including a ~370 MB stratum CSV). It is handled explicitly with Git LFS:

- `RHESSys_Data_Importer/.gitattributes` routes the Data tree through LFS.
- `RHESSys_Data_Importer/Data/.gitattributes` routes `*.csv` through LFS.

The raster `.tiff` sources and CSVs in `RHESSys_Data_Importer/Data/CentralCoast/RHESSysOutput-SingleWarmIdx-6-4-2026/` are tracked via LFS so the working repository stays usable.

Source vs. sample vs. generated data distinction:

- `RHESSys_Data_Importer/Data/BigCreek/` holds Big Creek v1 source/sample files.
- `RHESSys_Data_Importer/Data/CentralCoast/` holds Central Coast v2 source/sample bundles.
- `bin/`, `obj/`, and `.vs/` are generated and are not source.

Acceptance Check

- Future Mountain can commit the importer without nested Git confusion: the importer has no nested `.git`, and its `.sln/.csproj` are now tracked.
- Large source data handling is explicit: data is tracked via Git LFS.
- Generated local files are not committed: `bin/`, `obj/`, and `.vs/` under the importer are ignored.